EKNATHALABS · INTERVIEW PREP

SRE Reliability & Incident Management — EknathaLabs Interview Guide

Platform, SRE & DevOps interview guide

Questions	**173 questions · spoken answers included**
Online (always latest)	**interviewhub.eknathalabs.com/sre_reliability_interview_guide.html**
Printed	**2026-10-21**

Free to use and share for personal study. Answers are written for interview practice, so check command flags and versions against the official documentation before running anything in production.

certs.eknathalabs.com

Q1 What is SRE — how does it differ from traditional Ops and from DevOps?

Fundamentals

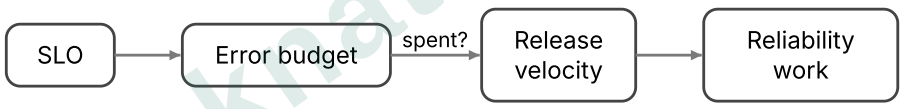
🎙 Spoken answer · ~73 s

Traditional operations is a separate team that keeps production running, usually reactive and heavy on manual work. DevOps is a culture and set of practices that breaks the wall between development and operations: shared ownership, CI/CD, automation, and fast feedback. SRE is a specific, prescriptive way of implementing those ideas. It's often described as what happens when you ask a software engineer to design an operations function. SREs treat operations as a software problem. They define reliability numerically with SLOs, use error budgets to balance reliability against feature velocity, cap toil so at least half their time goes to engineering, automate away repetitive work, run blameless postmortems, and own on-call with clear limits. So DevOps tells you to collaborate and automate, and SRE gives concrete mechanisms for how: SLIs and SLOs, error budgets, toil budgets, and defined incident practices. They are complementary rather than competing: you can say SRE is a class that implements the DevOps interface. The key difference in practice is that SRE makes reliability measurable and negotiable with data instead of opinions.

Likely follow-ups

- How does an error budget change the conversation between SRE and product?
- Can a company do DevOps without SRE?

🗺 SRE loop




```
Traditional Ops: separate team keeps prod running; reactive; manual toil
```

```
DevOps: culture + practices to break the dev/ops wall (CALMS, CI/CD)
```

```
SRE: a specific, prescriptive implementation of DevOps using SLOs + error budgets
```

SRE is "what happens when you ask a software engineer to design an operations function." It treats operations as a software problem — automating toil, defining reliability numerically with SLOs, and using error budgets to balance velocity against stability. DevOps is the philosophy; SRE is one concrete, opinionated way to do it.

Interview framing: "DevOps is a set of principles; SRE is a prescriptive engineering discipline that implements those principles with measurable reliability targets." That one sentence signals you understand both.

📚 Official docs

- Google SRE Book: Introduction (what is SRE)

certs.eknathalabs.com

Q2 What are the four golden signals of monitoring?

Fundamentals

🎙 Spoken answer · ~70 s

The four golden signals come from the Google SRE book, and they're the minimum worth instrumenting for any user-facing service. Latency is the time it takes to serve a request, and I measure the latency of successful requests and failed requests separately, because a fast error can make averages look good while hiding failures. Traffic is the demand on the system, such as requests per second or transactions per second. Errors is the rate of failing requests, including explicit failures like HTTP 500s and implicit ones, such as a 200 response with wrong content or a response that exceeded the latency target. Saturation is how full the system is: CPU, memory, disk, network, queue depth, or connection pool usage, and importantly it predicts trouble before things break, since latency often degrades as saturation approaches a limit. Together these tell me whether the service is healthy and where to look. I'd alert primarily on user-facing symptoms like errors and latency, and use saturation for capacity planning and early warning.

Likely follow-ups

- Why measure latency of failed requests separately?
- How does saturation differ from utilisation?

```
Latency  — time to serve a request (split success vs error latency)
```

```
Traffic  — demand on the system (req/s, transactions/s)
```

```
Errors   — rate of failed requests (explicit 5xx + implicit wrong answers)
```

```
Saturation — how "full" the system is (CPU, mem, queue depth, I/O)
```

From the Google SRE book. If you can only instrument four things, instrument these. Critical nuance: measure latency of failed requests separately — a fast 500 can otherwise hide in your latency percentiles and make a broken service look healthy.

RED vs USE: RED (Rate, Errors, Duration) is the request-centric view for services. USE (Utilization, Saturation, Errors) is the resource-centric view for hosts/queues. Golden signals blend both.

📚 Official docs

- Google SRE Book: Monitoring Distributed Systems

certs.eknathalabs.com

Q3 Define SLI, SLO, and SLA — how do they relate?

SLI/SLO

🎙 Spoken answer · ~73 s

An SLI, a service level indicator, is a measurement of service behavior, typically a ratio like good requests over total requests, such as the proportion of requests served successfully in under three hundred milliseconds. An SLO, a service level objective, is the target for that indicator over a window, for example ninety-nine point nine percent of requests succeed over thirty days. An SLA, a service level agreement, is the contract with customers that attaches consequences, like credits or penalties, if you miss a level, and it's usually looser than the internal SLO so you have a buffer. So the SLI is what you measure, the SLO is what you aim for, and the SLA is what you promise and pay for. SLOs are the most useful for engineering, because they drive the error budget, which is one hundred percent minus the SLO, and that tells teams how much unreliability they can spend on shipping and risk. Choosing SLIs close to user experience, like availability and latency at the load balancer, matters more than picking perfect numbers.

Likely follow-ups

- Why should the SLA be looser than the SLO?
- What makes a bad SLI?

```
SLI = a measured number: e.g. proportion of requests < 300ms that return 2xx
```

```
SLO = internal target for the SLI: e.g. 99.9% over a rolling 28 days
```

```
SLA = external contract with consequences: e.g. 99.5% or we credit the customer
```

SLI is the metric, SLO is the goal, SLA is the promise. The SLA is always looser than the SLO — you want internal alarms to fire well before you breach a customer contract. Not every SLO needs an SLA; many internal services have SLOs with no contractual SLA at all.

Common mistake: Setting the SLO equal to the SLA. If they are equal, you have zero internal safety margin — by the time you notice the SLO miss, you have already breached the contract.

📚 Official docs

- Google SRE Book: Service Level Objectives

certs.eknathalabs.com

Q4 How do you choose a good SLI — what makes a metric SLO-worthy?

SLI/SLO

🎙 Spoken answer · ~75 s

A good SLI reflects the user's actual experience, moves predictably with user happiness, and is expressed as a simple ratio of good events to valid events, between zero and one hundred percent. For example, the proportion of requests that returned a successful status and completed in under four hundred milliseconds, out of all valid requests. I measure it as close to the user as practical, ideally at the load balancer or from the client side, because server-side success means nothing if the load balancer or network in front of it is failing. I choose SLI types by service kind: availability and latency for request-driven services, freshness and correctness for data pipelines, and durability for storage. It must be something I can measure reliably and cheaply, and the team can actually influence. It should be simple enough that everyone understands what it means. Bad SLIs are internal metrics like CPU usage, which doesn't correlate directly with user experience, and averages, which hide the tail. I'd start with one or two SLIs per critical user journey and refine them as I learn.

Likely follow-ups

- Why express an SLI as a ratio of good to valid events?
- Why is CPU usage a poor SLI?

```
Good SLI: measured at the load balancer / point closest to the user
```

```
SLI form: good events / valid events  (a ratio, 0–100%)
```

```
e.g. (requests < 400ms AND status==2xx) / (all non-rate-limited requests)
```

A good SLI tracks the user's actual experience, moves predictably with user happiness, and is a simple ratio of good to total. Measure as close to the user as possible — server-side 200s mean nothing if the load balancer is timing out before requests arrive. Pick 1–3 SLIs per user journey, not one per microservice.

Per journey, not per service: Define SLIs around user journeys (checkout, search, login), not infrastructure components. Users do not care that pod-7 is healthy; they care that checkout works.

📚 Official docs

- Google SRE Book: Service Level Objectives
- Google SRE Workbook: Implementing SLOs

certs.eknathalabs.com

Q5 What do the nines actually mean in downtime per period?

SLI/SLO

🎙 Spoken answer · ~73 s

The nines translate into downtime allowances. Ninety-nine percent, two nines, allows about seven hours and eighteen minutes of downtime a month, or three point six five days a year. Ninety-nine point nine, three nines, is about forty-four minutes a month, or eight point seven seven hours a year. Ninety-nine point nine five is about twenty-two minutes a month. Ninety-nine point nine nine, four nines, is about four minutes and twenty-three seconds a month, or fifty-two minutes a year. Five nines is about twenty-six seconds a month and five minutes a year. Each additional nine is roughly ten times harder and more expensive to achieve, because it demands redundancy, automation, and faster recovery than a human can deliver. At four nines and beyond, you effectively can't rely on human response: an incident's detection and mitigation exceed the budget. So when asked how many nines a service needs, the right answer is the fewest that keep users happy, informed by user expectations, dependencies' own reliability, and cost. Your availability can't exceed that of your critical dependencies.

Likely follow-ups

- Why can't you rely on humans for five nines?
- How do dependency SLOs limit your own SLO?

```
99%     (two nines)  → 7h 18m  down / month   |  3.65 days / year
99.9%   (three)      → 43m 49s down / month   |  8.77 hours / year
99.95%               → 21m 54s down / month   |  4.38 hours / year
99.99%  (four)       → 4m 23s  down / month   |  52.6 min  / year
99.999% (five)       → 26.3s   down / month   |  5.26 min  / year
```

Each extra nine is roughly 10x harder and more expensive. The right answer to "how many nines?" is "the fewest that keep users happy." Five nines for a back-office reporting tool is waste; three nines for a payment path may be negligence.

Reliability has diminishing returns: Beyond a point, the user's network/device is less reliable than your service — extra nines become invisible to them but cost you enormously.

📚 Official docs

- Google SRE Book: Service Level Objectives
- Google SRE Book: Embracing Risk (capacity & availability math)

certs.eknathalabs.com

Q6 What is an error budget and how is it calculated?

Error Budget

🎙 Spoken answer · ~70 s

An error budget is the amount of unreliability an SLO allows. If the SLO is ninety-nine point nine percent over thirty days, the budget is point one percent, which is about forty-three minutes of full downtime, or one bad request in a thousand. You calculate it as one minus the SLO, applied to time or to request counts, and request-based is better for services with uneven traffic. The point isn't to hit zero errors; it's to make reliability a shared, measurable quantity. While there's budget left, teams ship features, run experiments, and take reasonable risks. When the budget is burning too fast or runs out, the error budget policy kicks in: slow or freeze risky launches, prioritize reliability work, and review the causes. That removes the argument between product and SRE, because the data decides. I also watch how fast we're spending the budget, the burn rate, and not only the remaining balance, because a fast burn needs a page while a slow burn can be a ticket.

Likely follow-ups

- What if product ignores the error budget policy?
- Time-based or request-based budgets: which do you pick?

```
Error budget = 100% − SLO
```

```
SLO 99.9% over 28 days → budget = 0.1%
```

```
Total requests 28d = 50,000,000
```

```
Budget = 0.001 × 50,000,000 = 50,000 allowed bad requests
```

```
Burned 30,000 so far → 60% consumed, 40% remaining
```

The error budget is the maximum unreliability you are allowed before breaching the SLO. It converts reliability from an argument into arithmetic: "we have 20,000 bad requests left this window" is a number both product and engineering can reason about.

Why it matters: The error budget reframes reliability as a shared, spendable resource. Shipping features spends it; reliability work refills it. This ends the endless "ship vs stabilize" debate with data.

📚 Official docs

- Google SRE Book: Embracing Risk (error budgets)

certs.eknathalabs.com

Q7 What is an error budget policy — what happens when the budget is exhausted?

Error Budget

🎙 Spoken answer · ~75 s

An error budget policy is the pre-agreed, written response to different states of the error budget, agreed by both product and engineering before any incident, so that decisions aren't relitigated in the heat of the moment. A typical policy: while the budget is healthy, above about twenty-five percent remaining, the team ships at normal velocity and can take reasonable risks. When the budget is low, say below twenty-five percent, risky changes get extra review and more effort goes into reliability work. When the budget is exhausted, there's a feature freeze: only reliability fixes and critical changes ship until the budget recovers. If it's overspent, meaning negative, it becomes an all-hands reliability effort and postmortem action items take priority. The policy should also cover exceptions, such as security fixes, and who can approve them, and what happens when the cause is outside the team's control, such as a dependency. It needs executive backing, otherwise it's ignored the first time a deadline looms. The value is that the trade-off between reliability and velocity becomes automatic and data-driven rather than a debate.

Likely follow-ups

- Who has to sign off on an error budget policy and why?
- What happens if leadership overrides the freeze?

```
Budget healthy (>25% left) → normal feature velocity
```

```
Budget low (<25%)          → review risky changes, add reliability work
```

```
Budget exhausted (0%)      → feature freeze; only reliability + critical fixes ship
```

```
Budget overspent (negative)→ all-hands on reliability; postmortem actions prioritised
```

The policy is the pre-agreed, written response to budget states — agreed by product AND engineering BEFORE an incident, so it is not relitigated under pressure. The classic consequence: exhausting the budget triggers a feature freeze until reliability recovers. Crucially this is a policy, not a punishment — it self-corrects velocity.

Get buy-in early: The hardest part is getting product leadership to sign the freeze clause in advance. Without that signature, the error budget is just a dashboard nobody obeys.

📚 Official docs

- Google SRE Book: Embracing Risk (error budgets)
- Google SRE Workbook: Error Budget Policy

certs.eknathalabs.com

Q8 What is burn rate — how do you alert on it?

Error Budget

🎙 Spoken answer · ~71 s

Burn rate is how fast you're consuming the error budget relative to the rate that would exactly exhaust it at the end of the window. A burn rate of one means you use the whole budget exactly at the end of the SLO period; a burn rate of ten would exhaust a thirty-day budget in three days. Alerting on burn rate beats plain error-rate thresholds because it ties urgency to impact on the SLO. The standard approach, from the Google SRE workbook, is multi-window, multi-burn-rate alerts. A fast burn, around fourteen point four times over one hour with a five-minute short window, pages immediately because it eats about two percent of the monthly budget. A slower burn, like six times over six hours, also pages. And a slow burn of about one times over three days opens a ticket. The short window confirms the problem is still happening, which cuts false positives and makes alerts reset quickly. That gives fast detection of big incidents and low noise for small ones.

Likely follow-ups

- Why use two windows instead of one?
- How would you write this alert in PromQL?

```
Burn rate = how fast you are consuming the error budget vs sustainable pace
```

```
Burn rate 1  = budget lasts exactly the SLO window (sustainable)
```

```
Burn rate 14.4 = entire 30-day budget gone in ~2 days
```

```
# Multi-window multi-burn-rate (Google SRE recommended):
```

```
Fast page: 14.4x over 1h AND 5m windows  → 2% budget in 1h
```

```
Slow page: 6x over 6h AND 30m windows    → 5% budget in 6h
```

```
Ticket:    1x over 3d AND 6h windows      → slow leak
```

Burn-rate alerting fires on how fast you are spending budget, not on raw thresholds. This gives high precision (only alert when budget is genuinely at risk) and good reset behaviour. The dual short+long window prevents flapping: the long window confirms a sustained problem, the short window confirms it is still happening right now.

Why not just alert on "error rate > 1%"? Static thresholds either page too much (noise) or too late (budget already gone). Burn rate ties the alert directly to SLO risk.

📚 Official docs

- Google SRE Workbook: Alerting on SLOs

certs.eknathalabs.com

Q9 How do you alert with PromQL on a burn rate against an SLO?

Error Budget

🎙 Spoken answer · ~81 s

I alert on burn rate by comparing the observed error ratio with the rate that would exactly consume the budget. First I compute the error ratio over a window, with PromQL: the sum of the rate of requests with a five-hundred status over the window, divided by the sum of the rate of all requests. For a ninety-nine point nine percent SLO the budget is zero point zero zero one. A fast-burn alert fires when the error ratio exceeds fourteen point four times the budget, which would consume about two percent of a thirty-day budget in one hour, and I require two windows to agree: for example the one-hour ratio and the five-minute ratio are both above the threshold. The long window shows the problem is significant, the short window confirms it is still happening, so the alert resolves quickly after recovery. A slow-burn alert uses a lower multiplier, like six over six hours or one over three days, and creates a ticket instead of a page. To keep alert evaluation cheap, I precompute the ratios with recording rules at several windows. Tools like Sloth or Pyrra can generate these rules from an SLO definition.

Likely follow-ups

- Why use two windows in a burn-rate alert?
- What does a burn rate of 14.4 mean for a 30-day SLO?

🗺 Burn-rate alert




```
# error ratio over a window
```

```
sum(rate(http_requests_total{status=~"5.."}[1h]))
```

```
  / sum(rate(http_requests_total[1h]))
```

```
# fast-burn alert (2-window): 14.4x burn on 99.9% SLO (budget=0.001)
```

```
(  job:slo_errors:ratio_rate1h > (14.4 * 0.001)
```

```
 and job:slo_errors:ratio_rate5m > (14.4 * 0.001) )
```

```
# precompute ratios with recording rules to keep alerts cheap
```

```
record: job:slo_errors:ratio_rate5m
```

Use recording rules to precompute the error ratio at several windows, then the alert just compares the ratio to burn_rate × budget. Require both the long and short window to exceed the threshold so a brief blip does not page. This is the standard SRE-workbook implementation.

certs.eknathalabs.com

📚 **Official docs**

- Google SRE Book: Service Level Objectives
- Google SRE Workbook: Alerting on SLOs
- Prometheus: Querying basics (PromQL)

Q10 What makes a healthy on-call rotation?

On-Call

🎙 **Spoken answer · ~75 s**

A healthy on-call rotation is sustainable and limited in its burden. First, enough people: ideally six to eight engineers per rotation, so each is on call about one week in six to eight. Smaller rotations burn people out. For global teams, follow-the-sun coverage avoids night pages where possible. Each shift has a primary and a secondary for escalation, with a documented path beyond that, and a clear handover process. A common guideline from Google is no more than about two incidents per shift, so people have time to handle them properly and follow up. Alert quality matters as much as headcount: every page should be urgent and actionable, with a runbook, and noisy alerts get fixed or deleted. On-call is compensated, with pay or time off, and teams respect rest after a night page. New people shadow before they go on call, and they have the training and access to act. I track metrics like pages per shift, after-hours pages, and time to acknowledge, and review them regularly. If on-call is consistently painful, that's a signal to invest in reliability.

Likely follow-ups

- Why does rotation size matter for burnout?
- How would you onboard a new engineer to on-call?

```
Rotation size: ≥ 6–8 engineers so each person is on-call ~1 week in 6–8
```

```
Follow-the-sun for global teams → no night pages where possible
```

```
Primary + secondary (escalation) on every shift
```

```
Hard cap: ≤ 2 incidents per on-call shift (Google guideline)
```

```
Compensation: time off or pay for on-call burden
```

A healthy rotation has enough people that on-call is occasional, has a documented escalation path, and limits pages per shift so responders are not exhausted. Google's rule of thumb: if a shift routinely exceeds ~2 incidents, the system is too noisy — that is an engineering bug, not an on-call problem.

Pager fatigue is a reliability risk: An exhausted, alert-numbed responder misses the real incident. Protecting on-call health directly protects uptime.

certs.eknathalabs.com

📚 **Official docs**

- Google SRE Book: Being On-Call
- Google SRE Workbook: On-Call

Q11 How do you reduce alert noise and fight pager fatigue?

On-Call

🎙 **Spoken answer · ~77 s**

Alert fatigue comes from pages that aren't urgent, actionable, or in need of human judgement. My test for any page: is it urgent, can the person do something about it, and does it need a human now? If it fails any of those, it should be a ticket or a dashboard, or be automated away. I alert on symptoms, like SLO burn rate or user-facing errors, rather than causes, such as one node's high CPU, because symptoms are what matter to users and are far fewer. I use inhibition so that when an upstream cause fires, downstream alerts are suppressed, group related alerts into a single notification, and add for-durations to avoid reacting to blips. Every alert links to a runbook. Then I measure: alerts per shift, the percentage that were actionable, and the percentage that auto-resolved, and drive those down. After each on-call week, the team reviews noisy alerts and deletes or fixes them. Alerts that nobody acts on should be removed, because they teach people to ignore the pager. Where possible, auto-remediate the common, safe cases so humans aren't woken for them.

Likely follow-ups

- What does alert inhibition do?
- Which metrics would you track to show alert quality is improving?

```
Every page must be: urgent, actionable, and require human judgement
```

```
If an alert is not all three → make it a ticket, not a page
```

```
Alert on symptoms (SLO burn), not causes (one node high CPU)
```

```
Add inhibition: silence downstream alerts when an upstream cause fires
```

```
Track: alerts per shift, % actionable, % auto-resolved → drive down
```

The test for any page: is it urgent, actionable, and does it need a human now? If a machine could fix it, automate it. If it can wait until morning, ticket it. Alert on user-facing symptoms via SLO burn rate rather than on every internal cause — one symptom alert replaces dozens of cause alerts.

Run an alert review: Monthly, review every alert that paged. Delete or downgrade any that were not actionable. A shrinking pager is a sign of a maturing SRE practice.

certs.eknathalabs.com

📚 **Official docs**

- Google SRE Book: Practical Alerting
- Google SRE Book: Being On-Call

Q12 What is the incident command system (ICS) — what are the key roles?

Incident Mgmt

🎙 **Spoken answer · ~73 s**

The incident command system, borrowed from emergency services, separates command from execution so large incidents stay coordinated. The Incident Commander owns the incident and makes the decisions, but deliberately isn't hands on the keyboard; their job is to keep the big picture, decide priorities, and make sure the right people are involved. The Operations or Tech Lead, with their team, does the investigation and fixes. The Communications Lead updates stakeholders, executives, support, and the public status page on a regular cadence, so responders aren't interrupted. The Scribe records timestamped actions and decisions in the incident channel, which produces the timeline for the postmortem. There may also be a planning lead or subject matter experts in a big incident. The principle is that the IC coordinates and does not debug, because someone deep in a debugging session loses situational awareness. In a small incident one person might take several roles, but the responsibilities are still distinct. Clear roles, a single channel as the source of truth, and explicit handoffs reduce confusion and duplicated or conflicting actions.

Likely follow-ups

- Why shouldn't the incident commander debug?
- How do roles change in a small incident?

```
Incident Commander (IC): owns the incident, makes decisions, NOT hands-on-keyboard
```

```
Operations / Tech Lead: the person(s) actually investigating and fixing
```

```
Communications Lead: updates stakeholders, status page, execs
```

```
Scribe: timestamps every action and decision in the incident channel
```

ICS (borrowed from emergency services) separates command from execution. The IC coordinates and decides but does not debug — that prevents the classic failure where the only person who understands the system is also trying to run the meeting. For small incidents one person wears several hats; for large ones the roles split.

The IC's job is clarity, not heroics: Track who is doing what, prevent duplicate work, decide when to escalate, and keep one source of truth. The IC can be junior — command is a skill, not seniority.

📚 **Official docs**

- Google SRE Book: Managing Incidents

certs.eknathalabs.com

Q13 Walk through the lifecycle of an incident from detection to closure.

Incident Mgmt

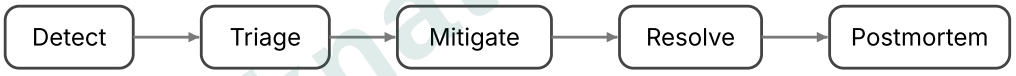
🎙 Spoken answer · ~72 s

The lifecycle goes from detection to closure in stages. First detect: an alert fires, ideally on SLO burn, or users report a problem. Then triage: assess the severity and impact, declare an incident, and assign an incident commander and roles. Then mitigate, which is the stage people get wrong: stop the bleeding by rolling back, failing over, draining traffic, or disabling a feature before I understand the root cause. Restoring service comes first; diagnosis can wait until users are no longer affected. Then resolve: confirm that the SLI has recovered and customers are unaffected, not just that the change was made. After that comes review: a blameless postmortem with a timeline, causes, and action items. Finally close: track the action items to completion in the normal backlog. Throughout, communicate on a regular cadence internally and externally. Good incident practice also records what worked and didn't, so the process improves. The sequence that trips people up is mitigate before diagnose, because engineers are tempted to dig into the root cause while the outage continues.

Likely follow-ups

- Why mitigate before finding the root cause?
- How do you confirm an incident is truly resolved?

🗺 Incident lifecycle




```
1. Detect    — alert fires (ideally SLO burn) or user report
2. Triage    — assess severity, declare incident, assign IC
3. Mitigate  — stop the bleeding (rollback, failover, drain) BEFORE root cause
4. Resolve   — confirm SLI recovered, customers unaffected
5. Review    — blameless postmortem, action items
6. Close     — actions tracked to completion
```

The order that trips people up: mitigate before you diagnose. Restoring service is the priority; understanding why it broke comes after users are safe. A rollback that fixes symptoms in 2 minutes beats a root-cause hunt that fixes it "properly" in 2 hours.

Mitigation ≠ resolution: Rolling back stops user pain (mitigation). The bug still exists (not resolved). Separating these two lets you de-escalate the page while the fix proceeds calmly.

certs.eknathalabs.com

📚 **Official docs**

- Atlassian: Incident management handbook (lifecycle)

Q14 How do you classify incident severity (SEV levels)?

Incident Mgmt

🎙 **Spoken answer · ~75 s**

Severity levels classify incidents by impact so the response is proportionate. SEV1 is critical: a major outage, data loss, or security breach, which triggers an all-hands response, an incident commander, executive communication, and frequent updates. SEV2 is high: significant degradation or a key feature down, with an incident commander and the on-call engaged. SEV3 is moderate: partial or minor impact where a workaround exists, handled during business hours or by the on-call without a full process. SEV4 is low: cosmetic or internal issues handled as a ticket, with no page. The definitions should be based on customer impact and ideally tied to the SLO, such as how fast the error budget is burning, plus things like data integrity and security, not on which system is affected. Severity drives who gets paged, whether an incident commander is assigned, how often stakeholders are updated, and whether the work can wait until morning. It can change during an incident, and it's better to declare high and downgrade than the other way around. Everyone should be able to declare an incident without fear.

Likely follow-ups

- Why is it better to over-declare severity than under-declare?
- What should a SEV1 definition include?

```
SEV1 — critical: major outage, data loss, security breach; all-hands, exec comms
```

```
SEV2 — high: significant degradation, key feature down; IC + on-call engaged
```

```
SEV3 — moderate: partial/minor impact, workaround exists; handled in business hours
```

```
SEV4 — low: cosmetic or internal; ticket, no page
```

Severity drives response: who is paged, whether an IC is assigned, how often stakeholders are updated, and whether it can wait until morning. Define the levels concretely (by user impact, revenue, data risk) so the on-call engineer is not guessing at 3am. Under-classifying delays response; over-classifying burns people out.

When unsure, declare higher: It is cheap to downgrade a SEV2 to SEV3 once you understand it. It is expensive to discover a "minor" issue was a SEV1 an hour too late.

📚 **Official docs**

- Google SRE Workbook: Incident Response
- Atlassian Incident Management Handbook

certs.eknathalabs.com

Q15 What is MTTR — and why decompose it into MTTD / MTTA / MTTF?

Incident Mgmt

🎙 Spoken answer · ~75 s

These metrics measure different parts of incident response. MTTD is mean time to detect: from the incident starting to an alert or detection. MTTA is mean time to acknowledge: from the alert to a human responding. MTTR is mean time to recover or repair: from detection to the service being restored. MTBF is mean time between failures, a measure of how reliable the system itself is. Saying improve MTTR is too blunt, so I decompose it to find where time goes. If detection is slow, I invest in better monitoring and SLO-based alerts. If acknowledgement is slow, the problem is paging, escalation, or on-call practices. If the repair phase is slow, I invest in runbooks, automation, safer rollbacks, observability for faster diagnosis, and practice. A caveat is that averages hide the distribution: a few long incidents dominate the mean, so I look at percentiles and individual incidents, and the metrics are noisy with small sample sizes. They're also useful as trends, not targets, because turning them into targets invites gaming. The user-facing outcome is better captured by the SLO.

Likely follow-ups

- Why can MTTR be a misleading metric?
- Which part of MTTR would you attack first and why?

```
MTTD — Mean Time To Detect    (incident start → alert)
```

```
MTTA — Mean Time To Acknowledge (alert → human responding)
```

```
MTTR — Mean Time To Recover/Repair (detect → service restored)
```

```
MTBF — Mean Time Between Failures (reliability of the system itself)
```

"Improve MTTR" is too blunt. Decompose it: if detection (MTTD) is slow, invest in monitoring; if acknowledgement (MTTA) is slow, fix the pager/escalation; if repair is slow, invest in runbooks, rollback tooling, and practice. You cannot improve what you cannot localise.

MTTR is a distribution, not a point: A single 12-hour outage can dominate the mean. Track P50 and P90 of recovery time, and look at the long tail — that is where catastrophic incidents hide.

📚 Official docs

- DORA: The four key metrics

certs.eknathalabs.com

Q16 Production is down. Walk me through your first 10 minutes.

Incident Mgmt

🎙 Spoken answer · ~74 s

First I confirm and size the problem: is it real, who is affected, how bad. That sets the severity. I declare the incident, take or assign the incident commander role, and open a single channel so communication lives in one place. In the first minutes I post a short status update, because stakeholders care that someone has it. Then I go for mitigation before root cause: what changed recently, such as deploys, config, feature flags, or traffic? A recent change is the cause most of the time, and rollback is the fastest fix. I check the golden signals and dashboards to scope the blast radius, and I split roles, with someone investigating, someone communicating, and someone handling logistics, so I'm not doing everything myself. If I can stop the bleeding by rolling back, failing over, shedding load, or disabling a feature, I do that first and investigate afterwards. If I'm not making progress in about fifteen minutes, I escalate and bring in more people. And I keep a timeline as I go, because it feeds the postmortem.

Likely follow-ups

- You cannot roll back. What are your next options?
- How do you decide the severity level?

```
0–2m: Acknowledge page. Declare incident. Become/assign IC. Open a channel.
```

```
2–4m: Assess blast radius — which SLO, which users, how many. Set SEV.
```

```
4–6m: Check recent changes — last deploy? config push? feature flag? (80% of incidents)
```

```
6–8m: Mitigate — rollback / disable flag / failover. Stop the bleeding.
```

```
8–10m: Confirm SLI recovering. Post first stakeholder update. Scribe logging.
```

The structure interviewers want: coordinate first (IC + channel), quantify impact, suspect recent change, mitigate before diagnosing, communicate. Most outages correlate with a recent deploy or config change — "what changed in the last hour?" is the single highest-yield question.

Say this out loud: "My first instinct is to restore service, not to find root cause. I would roll back the most recent change and confirm the SLI recovers before investigating why." That is exactly the SRE reflex.

📚 Official docs

- Google SRE Workbook: Incident Response
- Google SRE Book: Effective Troubleshooting

certs.eknathalabs.com

Q17 How do you communicate during an incident — internal and external?

Incident Mgmt

🎙 Spoken answer · ~72 s

Communication is a core part of incident response and has its own owner, the communications lead, in a big incident. Internally, there's one incident channel as the single source of truth, with a pinned status summary. Responders stay in it, and stakeholders follow it rather than asking responders directly. Updates go out on a fixed cadence, for example every thirty minutes for a SEV1, even if the message is that there's no change, so stakeholders stop interrupting the team. Externally, I update a public status page quickly, acknowledging the problem even before the root cause is known, because silence erodes trust faster than bad news. I use a simple template: what's impacted, since when, what we're doing, and when the next update will be. I avoid speculation, blame, and jargon, and keep to confirmed facts. Customer support gets internal guidance and talking points. After resolution I post a final update, and for significant incidents follow up with a summary and a postmortem. Honest, predictable, plain-language communication matters as much as the technical fix.

Likely follow-ups

- Why post updates even when there's nothing new?
- What should a public status update never contain?

```
Internal: one incident channel = single source of truth; pinned status
```

```
Cadence: stakeholder updates on a fixed clock (e.g. every 30 min for SEV1)
```

```
External: public status page — acknowledge fast, even before root cause
```

```
Template: what's impacted | since when | what we're doing | next update time
```

Communication is the Comms Lead's full-time job in a big incident. Update on a predictable cadence so stakeholders stop interrupting responders to ask "any news?". Externally, acknowledging an issue quickly (even "we're investigating") builds more trust than silence followed by a perfect explanation.

Always promise a next-update time: "Next update by 14:30" stops the flood of status pings and buys the responders space to work.

📚 Official docs

- Atlassian: Incident management handbook (lifecycle)

certs.eknathalabs.com

Q18 What is a blameless postmortem and why is blamelessness essential?

Postmortem

🎙 Spoken answer · ~69 s

A blameless postmortem focuses on how the system and its processes allowed the failure, rather than on who made the mistake. The reasoning is that people rarely cause incidents through carelessness; they act on the information and tools they had, and a human error is a symptom that something made that error easy, like missing guardrails, unclear docs, or misleading alerts. If people fear punishment, they hide mistakes, avoid declaring incidents, and sanitize timelines, and you lose exactly the information you need to prevent a repeat. Blameless doesn't mean no accountability; it means accountability for fixing the system. In practice I ask what made this the reasonable thing to do at that moment, rather than who did it. I write the timeline factually, identify contributing factors rather than a single root cause, and produce concrete action items with owners and due dates. Leadership has to model it too: if the first reaction to an incident is to find someone to blame, the culture won't hold.

Likely follow-ups

- How do you handle a genuinely negligent act?
- How do you make sure action items get done?

```
Blameless = focus on systems and contributing factors, never on punishing individuals
```

```
Assume everyone acted reasonably with the info they had at the time
```

```
Ask "how did the system allow this?" not "who broke it?"
```

Blamelessness is not about being nice — it is about getting the truth. If engineers fear punishment, they hide details, and you lose the data needed to actually fix the system. The person who "caused" the outage usually did exactly what the system invited them to do; the fix is changing the system, not the person.

Human error is a symptom: "Someone ran the wrong command" is the start of the investigation, not the end. Why did a dangerous command have no guardrail, no confirmation, no dry-run? That is the real finding.

📚 Official docs

- Google SRE Book: Postmortem Culture
- Google SRE Workbook: Postmortem Culture
- Atlassian: How to run a blameless postmortem

certs.eknathalabs.com

Q19 What goes into a good postmortem document?

Postmortem

🎙 Spoken answer · ~73 s

A good postmortem exists to drive change, not to be filed and forgotten, and it's blameless: it focuses on systems and processes, not individuals. The sections I include: a summary of what happened in two sentences; impact, covering users affected, duration, SLO and error budget consumed, and cost if known; a detailed timeline with timestamps from first symptom through detection, mitigation, and full recovery; the root cause or, better, causes and contributing factors, both technical and organisational; what went well, what went poorly, and where we got lucky, which often reveals hidden risks; and action items, each with an owner, a priority, and a ticket link. The most important parts are the contributing factors and the action items. Actions should address detection, mitigation, and prevention, and be specific and verifiable. The review should be shared widely so other teams learn, and the actions tracked to completion. Also record the lessons learned and any questions that remain open. A good document is factual, free of blame, and written soon after the incident while memories are fresh.

Likely follow-ups

- Why is 'where we got lucky' a valuable section?
- What makes an action item good rather than vague?

```
Summary — what happened, in two sentences
```

```
Impact — users affected, duration, SLO/budget burned, $ if known
```

```
Timeline — timestamped, from first symptom to full recovery
```

```
Root cause(s) — technical + contributing factors
```

```
What went well / what went poorly / where we got lucky
```

```
Action items — each with an owner, priority, and ticket link
```

The document exists to drive change, not to file and forget. The most important sections are the timeline (builds shared understanding) and the action items (the only part that prevents recurrence). "Where we got lucky" is underrated — it surfaces the next incident before it happens.

Action items without owners die: Every action needs a named owner, a priority, and a tracked ticket. An action item with no owner is a wish, not a fix.

📚 Official docs

- Google SRE Book: Postmortem Culture
- Google SRE Workbook: Postmortem Culture
- Atlassian: How to run a blameless postmortem

certs.eknathalabs.com

Q20 How do you do root cause analysis — what is the Five Whys, and its trap?

Postmortem

🎙 Spoken answer · ~79 s

Root cause analysis tries to find why an incident happened so the underlying issue can be fixed. The Five Whys is a simple technique: start with the symptom, such as checkout returned errors for twenty minutes, and keep asking why. Because the database connections were exhausted; because a deploy increased both connections per pod and the pod count; because there was no connection pool limit relative to the database maximum; because capacity review wasn't part of the deploy checklist; because no automated check connects the pool size to the database limit. This drills past the surface to systemic causes. The trap is that it implies a single linear chain with one root cause. Real incidents usually have multiple contributing factors, and the answer depends on who is asking and where they stop, which can end at blaming a person. So I treat it as a starting point and use broader techniques: timelines, contributing factor analysis, and looking at what in the system made the failure possible and what would have limited its impact, such as detection, safeguards, and recovery. And I aim at fixable system causes, not human error.

Likely follow-ups

- Why is 'human error' a poor root cause?
- How would you find contributing factors beyond the Five Whys?

```
Symptom: checkout returned 500s for 20 minutes
```

```
Why? DB connections exhausted
```

```
Why? A deploy raised connections-per-pod and pod count together
```

```
Why? No connection-pool limit relative to DB max_connections
```

```
Why? Capacity review not part of the deploy checklist
```

```
Why? No automated check links pool size to DB limits
```

Five Whys drills past the surface symptom to systemic causes. The trap: it implies a single linear chain. Real incidents usually have multiple contributing factors that combined — a latent bug plus a config change plus a missing alert. Treat it as a tree of contributing factors, not one root.

Prefer "contributing factors" over "the root cause": Complex systems fail from combinations. Naming one root cause often hides the others and gives false confidence that one fix is enough.

📚 Official docs

- Google SRE Book: Postmortem Culture

certs.eknathalabs.com

Q21 What is toil — and how is it different from overhead?

Toil

🎙 Spoken answer · ~70 s

Toil, in SRE terms, is work that is manual, repetitive, automatable, tactical, produces no lasting value, and scales linearly with the size of the service. Examples are restarting a stuck service by hand, applying the same fix every week, manually provisioning accounts, or clicking through a deployment. If the service doubles, the toil doubles. Not everything unpleasant is toil, and overhead is different: meetings, planning, training, and administrative tasks are necessary but aren't toil. Engineering work, like writing automation, design reviews, capacity planning, and improving on-call strategy, produces lasting value and isn't toil either. The distinction matters because SRE aims to keep toil below a cap and invest the remaining time in engineering that reduces future toil. Identifying toil involves tracking where time goes, perhaps with tagging in tickets or surveys, and asking whether a task needs human judgement. If a machine could do it, it's toil. Left unchecked, toil crowds out the engineering work that would reduce it, creating a downward spiral and burning out people.

Likely follow-ups

- Give an example of work that is overhead but not toil.
- How would you measure the toil in your team?

```
Toil is work that is: manual, repetitive, automatable, tactical,
```

```
        devoid of lasting value, and scales linearly with service size
```

```
e.g. manually restarting a stuck service, applying the same fix weekly
```

```
NOT toil: design reviews, writing automation, on-call strategy (overhead/eng work)
```

Toil is the manual, repetitive operational work that grows as your service grows and produces no durable improvement. It is distinct from overhead (meetings, planning) and from real engineering. The danger of toil is that it scales linearly with the system — so without automation, growth eventually consumes the whole team.

The test: "If the service doubled in size, would this work double too?" If yes, it is toil and a candidate for automation.

📚 Official docs

- Google SRE Book: Eliminating Toil

certs.eknathalabs.com

Q22 What is the SRE toil budget — the 50% rule?

Toil

🎙 Spoken answer · ~75 s

Google's guideline is that SREs should spend at most fifty percent of their time on toil, such as tickets and manual operations, and at least fifty percent on engineering: automation, reliability projects, and tooling. The cap is a structural defence against a team drowning in operational work, because if all the time goes to firefighting, nothing is left to fix the causes, and the load only grows with the service. If toil consistently exceeds fifty percent, that's a signal to act: either automate it away, or push the operational load back to the product development team until the service is made more operable, or add headcount if the work is legitimate and unavoidable. It gives SRE teams a basis to say no to taking on a service that would be pure toil. Measuring it matters: I track how time is spent, using ticket categories or periodic surveys, and review it each quarter. It's a guideline, not a law; the principle is that sustainable operations need engineering time to reduce operational load, and the number gives a concrete trigger for conversations.

Likely follow-ups

- What do you do if toil is consistently above 50%?
- Why can the cap help SREs push back on taking a new service?

```
Google SRE cap: ≤ 50% of an SRE's time on toil (ops)
```

```
≥ 50% on engineering: automation, reliability projects, tooling
```

```
If toil exceeds 50% → overflow goes back to the dev team, or hire/automate
```

The 50% rule is a structural defence against the team drowning in operations. If toil consistently exceeds half the team's time, that is a signal — either automate it away or push the operational load back to the product team that generates it. Measuring toil (surveys, ticket categorisation) is how you know you are over the line.

Why the cap matters: Without it, an SRE team gradually becomes a manual-ops team, loses its engineering capacity, and can never dig out — the automation that would save them never gets written.

📚 Official docs

- Google SRE Book: Eliminating Toil

certs.eknathalabs.com

Q23 How do you decide what toil to automate first?

Toil

🎙 Spoken answer · ~74 s

I prioritise automation by value, not by annoyance. A simple scoring approach is frequency times time per occurrence times error-proneness, which captures how much time it consumes and how risky it is to do by hand. A task that happens daily and is dangerous to do manually, like a production failover or a database promotion, ranks high. I estimate payback as the cost of building the automation divided by the time saved per year, which tells me whether it's worth doing. Not all toil is worth automating, because automation has costs: building, maintenance, and the risk of bugs, so a rare task that takes ten minutes may simply deserve a good runbook. I'd start with a couple of quick wins to build momentum and credibility, then tackle the large recurring drains. Also consider whether to remove the toil entirely, by fixing the root cause or eliminating the need, which beats automating a workaround. Prefer self-service for requests from other teams, with guardrails. And measure the toil reduction afterwards, so the team sees the benefit and keeps investing.

Likely follow-ups

- When is a runbook better than automation?
- How do you calculate the payback of an automation project?

```
Score by: frequency × time-per-occurrence × error-proneness
```

```
Prioritise: high-frequency + risky (e.g. manual prod failover)
```

```
"Is it worth the time?" — payback = automation cost / time saved per year
```

```
Quick wins first to build momentum, then the big recurring drains
```

Not all toil is worth automating — automation has a cost too. Rank toil by how often it happens, how long it takes, and how dangerous it is to do by hand. A weekly 2-hour manual failover that occasionally causes outages is a far better target than a yearly 10-minute task. Compute the payback period honestly.

Don’t automate a broken process: If the manual process is itself wrong, automating it just produces failures faster. Fix the process, then automate it.

📚 Official docs

- Google SRE Book: Eliminating Toil

certs.eknathalabs.com

Q24 What is graceful degradation — give a concrete pattern.

Reliability Patterns

🎙 Spoken answer · ~75 s

Graceful degradation means keeping the core user journey alive when a non-critical dependency fails, by serving a reduced experience instead of an error. Concrete patterns: if the recommendations service is down, show generic popular items rather than an error page; if personalisation times out, serve cached or default content; if a non-critical dependency is failing, drop that feature but keep the main flow working. A checkout must work even if recommendations, reviews, and personalisation are unavailable. To implement it, I classify dependencies as critical or non-critical, set tight timeouts and circuit breakers on the non-critical ones, provide fallbacks and cached responses, and use feature flags that can shed optional load under pressure. I also design the UI to handle missing pieces. It must be tested, because fallbacks that are never exercised often fail when needed; chaos experiments and game days validate them. And I define in the SLO what degraded service means: it should count as available for the core journey but I track degradation separately so it's not hidden. The key principle is that partial service beats a total outage.

Likely follow-ups

- How do you decide which dependencies are critical?
- How do you test that fallbacks actually work?

```
Recommendations service down → show generic popular items, not an error page
```

```
Personalisation timeout → serve cached/default content
```

```
Non-critical dependency failing → drop the feature, keep the core flow working
```

```
Feature flags to shed optional load under pressure
```

Graceful degradation keeps the core user journey alive when a non-critical dependency fails. The checkout must work even if recommendations, reviews, and personalisation are all down. Design so that a failing dependency degrades the experience rather than breaking it — fall back to cached or default responses.

Rank dependencies by criticality: Map which dependencies are load-bearing for the core journey and which are optional. Optional ones should fail open (degrade); critical ones need redundancy.

📚 Official docs

- Google SRE Book: Handling Overload
- AWS Builders' Library: Avoiding fallback in distributed systems

certs.eknathalabs.com

Q25 Explain the circuit breaker pattern and why it prevents cascading failure.

Reliability Patterns

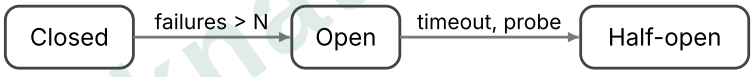
🎙 Spoken answer · ~73 s

A circuit breaker protects a caller from a failing downstream service and prevents cascading failure. It has three states. Closed: requests flow normally and failures are counted. Open: once the failure rate crosses a threshold, the breaker trips and calls fail immediately without touching the sick service, which gives it time to recover. Half-open: after a cooldown, the breaker lets a few probe requests through; if they succeed it closes again, and if they fail it re-opens. Without a breaker, callers keep retrying a dead service, piling up threads, connections, and requests until they exhaust their own resources, and then they fail too, so the failure propagates upward. With a breaker, the caller fails fast, frees resources, and can return a fallback or degraded response. Practical points: tune the thresholds and cooldown, combine it with timeouts, bounded retries with backoff and jitter, and bulkheads, and expose breaker state in metrics and alerts. Libraries like Resilience4j, or service meshes and proxies like Envoy, provide it. It's one layer of a resilience strategy, not a replacement for capacity.

Likely follow-ups

- How does a circuit breaker differ from a retry?
- How would a service mesh implement circuit breaking?

🗺 Circuit breaker




```
Closed  — requests flow normally; failures counted
```

```
Open    — failure threshold tripped → fail fast, stop calling the sick service
```

```
Half-open — after a cooldown, allow a few probes; success → close, failure → re-open
```

A circuit breaker stops a failing downstream from dragging callers down with it. Without it, callers pile up retrying a dead service, exhaust their own threads/connections, and the failure cascades upward. The breaker "trips" after N failures and fails fast, giving the downstream room to recover and protecting the caller's resources.

Retries make cascades worse: Naive retries multiply load on an already-struggling service. Pair circuit breakers with exponential backoff and jitter, and budget retries.

📚 Official docs

- Google SRE Book: Addressing Cascading Failures
- Microsoft: Cloud design pattern – Circuit Breaker

certs.eknathalabs.com

Q26 What are timeouts, retries with backoff, and jitter — why all three together?

Reliability Patterns

🎙 Spoken answer · ~70 s

They solve three different problems and are dangerous without each other. Timeouts bound how long you wait; without them, a slow dependency ties up threads and connections, and the slowness spreads upstream. Retries recover from transient failures, like a dropped packet or a brief overload, but they add load, so naive immediate retries during an outage multiply traffic and can turn a brownout into a full outage, a retry storm. Exponential backoff spaces retries out progressively, giving the dependency room to recover. Jitter adds randomness to those delays, because if a thousand clients fail at once and back off on the same schedule, they all retry at the same moments and hit the service in synchronized waves. So the combination is a sensible timeout, a small number of retries, only on idempotent or safe operations, exponential backoff, and full jitter. I'd also add a retry budget, like retries capped at ten percent of requests, and a circuit breaker so clients stop calling something that is clearly down.

Likely follow-ups

- Which operations are safe to retry?
- How does a circuit breaker interact with retries?

```
Timeout: never wait forever — bound every network call (e.g. 2s)
```

```
Retry: transient failures may succeed on retry — but cap attempts
```

```
Backoff: wait longer each retry (1s, 2s, 4s) to let the dependency recover
```

```
Jitter: randomise the wait so all clients don't retry in lockstep
```

```
delay = min(cap, base * 2**attempt) * random(0.5, 1.0)
```

Timeouts bound how long you wait. Retries handle transient blips. Exponential backoff stops retries from hammering a recovering service. Jitter is the subtle one — without it, every client retries at the same instant after an outage, creating a synchronised thundering herd that knocks the service back down. All four work as a set.

Retry budgets: Cap retries as a fraction of total traffic (e.g. ≤10%). This prevents retry storms from doubling load during partial outages.

📚 Official docs

- AWS Builders' Library: Timeouts, retries and backoff with jitter
- Microsoft: Cloud design pattern – Retry

certs.eknathalabs.com

Q27 What is a thundering herd / cache stampede and how do you prevent it?

Reliability Patterns

🎙 Spoken answer · ~74 s

A thundering herd or cache stampede happens when many clients act at the same instant. A classic case is a popular cache key expiring, so thousands of simultaneous requests miss the cache and all hit the database to regenerate it, which can overload or take down the backend. Another is all clients reconnecting at once after an outage. Prevention techniques: request coalescing, or singleflight, where one request fetches the value and the others wait for its result instead of each hitting the backend. Staggered expiry times with random jitter so keys don't all expire together. Refresh-ahead or background refresh, where the value is renewed before it expires. Serve-stale-while-revalidate, returning the old value while one request refreshes it, and also serving stale data if the backend is failing. A lock or lease on regeneration. For reconnects, clients use exponential backoff with jitter. At the system level, rate limiting and load shedding protect the backend. I'd also warm caches before launches. The theme is spreading load out in time and making sure only one request does the expensive work.

Likely follow-ups

- What does jitter do in a retry or expiry strategy?
- How does singleflight work?

```
Cache key expires → thousands of requests miss simultaneously → all hit the DB
```

```
Fixes:
```

```
  request coalescing / singleflight — one fetch, others wait for it
```

```
  staggered TTLs + jitter on expiry
```

```
  early/async refresh before expiry (refresh-ahead)
```

```
  serve-stale-while-revalidate on backend error
```

A thundering herd happens when many clients act at the same instant — a popular cache key expiring, or all clients reconnecting after an outage. The defence is to break the synchronisation: coalesce duplicate work so only one request rebuilds the cache, jitter expiry times, and serve stale data while a single background refresh runs.

📚 Official docs

- Google SRE Book: Addressing Cascading Failures
- AWS Builders' Library: Caching challenges and strategies

certs.eknathalabs.com

Q28 What is the difference between health checks: liveness vs readiness vs startup?

Reliability Patterns

🎙 Spoken answer · ~77 s

These three probes answer different questions, and confusing them causes outages. Liveness asks whether the process is alive and not deadlocked; if it fails, the kubelet restarts the container. Readiness asks whether the pod can serve traffic right now; if it fails, the pod is removed from Service endpoints but is not restarted, so traffic stops going to it until it recovers. A startup probe is for slow-starting applications: it gates the other two probes until the app has finished booting, so liveness doesn't kill it during a long startup. My guidance: readiness should check what's needed to serve requests, including critical dependencies, so an instance that can't reach its database stops receiving traffic. Liveness should be minimal and should not depend on external services, otherwise a database outage makes every pod fail liveness and restart together, turning a dependency problem into a restart storm. I also set sensible timeouts, thresholds, and periods, and make the endpoints cheap. Using a startup probe avoids having to set a huge initial delay on liveness. The most common mistake is a liveness probe that checks downstream dependencies.

Likely follow-ups

- Why should liveness not check dependencies?
- When do you need a startup probe?

```
Liveness  — is the process alive? Fail → kubelet restarts the container
```

```
Readiness — can it serve traffic now? Fail → removed from Service endpoints
```

```
Startup   — has a slow app finished booting? Gates the other two probes
```

```
readiness should check critical deps; liveness should NOT (avoid restart loops)
```

Confusing these causes outages. Liveness restarts a hung process. Readiness pulls a pod out of rotation without killing it (e.g. while a dependency is briefly unavailable). The classic mistake: putting dependency checks in the liveness probe — a downstream blip then triggers mass restarts, turning a minor degradation into a full outage.

Liveness must be cheap and local: If liveness depends on the database, a DB hiccup restarts every pod simultaneously — the opposite of resilience.

📚 Official docs

- Kubernetes: Liveness, readiness and startup probes
- AWS Builders' Library: Implementing health checks

certs.eknathalabs.com

Q29 How do load shedding and rate limiting protect a service under overload?

Reliability Patterns

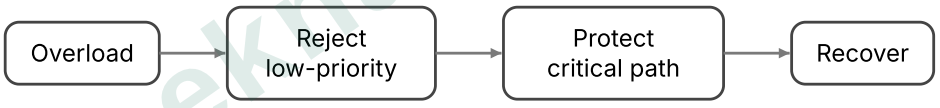
🎙 Spoken answer · ~75 s

Both protect a service when demand exceeds capacity, and the principle is that doing some work well beats doing all work badly. Rate limiting caps the requests per client, key, or tenant, using algorithms like token bucket or leaky bucket, so a single abusive or buggy client can't consume everything. Load shedding works at the service level: when saturated, it drops the lowest-priority work to protect the rest. For example, it sheds batch jobs, health-check noise, and background requests before user-facing traffic, often based on request priority or the queue length and latency. When rejecting, the service should return a fast, explicit response such as 429 for rate limiting or 503 with a Retry-After header, so clients back off, rather than letting requests queue until everything times out and the service collapses. Related techniques include bounded queues, adaptive concurrency limits, admission control, and graceful degradation. Clients should cooperate with exponential backoff and jitter. I'd also test overload behaviour with load tests, because services often fail badly at a cliff just beyond their capacity. Metrics for shed and limited requests give visibility.

Likely follow-ups

- Why is a fast rejection better than queueing under overload?
- What do 429 and 503 communicate to clients?

🗺 Load shedding




```
Rate limiting: cap requests per client/key (token bucket, leaky bucket)
```

```
Load shedding: drop lowest-priority work when saturated to protect the rest
```

```
Prioritise: shed health-check noise & batch jobs before user-facing traffic
```

```
Return 429 / 503 with Retry-After rather than collapsing
```

When demand exceeds capacity, doing some work well beats doing all work badly. Rate limiting protects against individual abusive clients; load shedding protects the system as a whole by dropping low-value requests so high-value ones still succeed. A service that sheds 10% of load and serves 90% fast is far better than one that accepts 100% and serves all of it slowly or crashes.

Fail fast and explicitly: Returning 429 with Retry-After lets well-behaved clients back off. Silently queueing requests until you OOM helps no one.

certs.eknathalabs.com

📚 **Official docs**

- RFC 6585: HTTP 429 Too Many Requests
- Google SRE Book: Handling Overload
- AWS Builders' Library: Using load shedding to avoid overload

---

Q30 What is a bulkhead and how does it limit blast radius?

Reliability Patterns

🎙 **Spoken answer · ~76 s**

A bulkhead isolates resources so that one failure can't sink the whole system, named after a ship's watertight compartments. In software that means giving each dependency, or each tenant, its own pool of resources, such as separate thread pools and connection pools per downstream service. Without bulkheads, if every dependency shares a single thread pool and one dependency becomes slow, its requests consume all the threads, and the whole service hangs even for requests that didn't need that dependency. With bulkheads, only calls to the slow dependency are affected, and the rest keep working. The same idea applies at larger scale: cell-based architecture, where traffic is partitioned into independent cells, each a complete copy of the stack serving a subset of users, so a failure or bad deploy is limited to one cell; shard per tenant, so one noisy customer can't starve others; separate clusters or accounts; and resource quotas. The cost is lower resource efficiency, since isolated pools can't share spare capacity, and more complexity. I use bulkheads together with timeouts, circuit breakers, and load shedding, and size pools by measured need.

Likely follow-ups

- What is a cell-based architecture?
- What's the trade-off of bulkheads in terms of efficiency?

```
Isolate resources so one failure can’t sink the whole ship
```

```
Separate thread/connection pools per dependency
```

```
Cell-based / shard-per-tenant architecture
```

```
One noisy tenant or one slow dependency cannot starve the others
```

Named after a ship’s watertight compartments. If every dependency shares one thread pool, a single slow dependency consumes all threads and the whole service hangs. Bulkheads give each dependency (or each tenant, or each shard) its own isolated resources, so a failure is contained to one compartment instead of flooding the entire service.

📚 **Official docs**

- AWS: Disaster recovery & availability concepts (Fault isolation boundaries)
- Google SRE Book: Addressing Cascading Failures
- AWS Builders' Library: Workload isolation using shuffle sharding

certs.eknathalabs.com

Q31 What is chaos engineering — what is the experiment loop?

Chaos/Game Day

🎙 Spoken answer · ~75 s

Chaos engineering proactively injects failure to find weaknesses before real outages do, and it's run as a disciplined experiment, not random breakage. The loop is: first define steady state, using a measurable signal that means the system is healthy, such as an SLI. Second, form a hypothesis, for example killing one availability zone will not breach the SLO. Third, inject the failure, in production or a production-like environment, starting with a small blast radius, such as a single instance or a small percentage of traffic. Fourth, observe whether steady state held, and abort immediately if the experiment causes real harm, with a clear stop condition ready. Fifth, learn and fix: if the system broke, I address the weakness, then widen the blast radius or move to the next failure and repeat. Prerequisites matter: good observability, so I can see the effect, a rollback plan, and stakeholder awareness. Examples are killing instances or pods, adding latency, dropping a dependency, or exhausting a resource. Tools include Chaos Mesh, Litmus, Gremlin, and AWS Fault Injection Service. Over time experiments are automated and run continuously.

Likely follow-ups

- What makes a good steady-state definition?
- How do you limit the blast radius of a chaos experiment?

```
1. Define steady state (the SLI that means "healthy")
```

```
2. Hypothesise: "killing one AZ will NOT breach the SLO"
```

```
3. Inject the failure in production (or close to it), small blast radius first
```

```
4. Observe — did steady state hold? abort if it breaks
```

```
5. Fix what broke; widen the blast radius; repeat
```

Chaos engineering proactively injects failure to find weaknesses before they cause real outages. It is a disciplined experiment, not random breakage: you start from a hypothesis about steady state, inject one fault with a tight blast radius and a kill switch, and verify the system behaves as you believed. The value is converting unknown failure modes into known, tested ones.

Start in staging, then prod: The real value is in production (that's where the real failure modes live), but earn trust first with small, well-instrumented experiments and an always-ready abort.

📚 Official docs

- Principles of Chaos Engineering

certs.eknathalabs.com

Q32 What is a game day, and how does it differ from a chaos experiment?

Chaos/Game Day

🎙 Spoken answer · ~78 s

A game day is a scheduled, rehearsed failure scenario run by the whole team, and its goal is to test people, process, and tooling, not only the technical resilience of the system. For example, we simulate that the primary region is gone and ask the on-call team to execute the failover runbook, timing it, checking that the runbook is accurate, that people have the access they need, that communication works, and that monitoring shows what we expect. A chaos experiment, by contrast, is often automated and focuses on a technical hypothesis about system behaviour, like what happens when a node is killed. They complement each other. Game days reveal gaps that automation can't: outdated runbooks, missing permissions, unclear ownership, slow escalation, and the stress of coordinating under pressure. To run one well, I define the scenario and objectives, brief participants, prepare safety measures and a way to stop the exercise, run it, observe and record, and then hold a debrief with action items, like any postmortem. Starting in a staging environment and progressing to controlled production exercises builds confidence. Regular practice means real incidents feel familiar.

Likely follow-ups

- What do game days reveal that automated chaos tests don't?
- How do you keep a production game day safe?

```
Game day: scheduled, rehearsed failure scenario run by the whole team
```

```
Goal: test people + process + tooling, not just the system
```

```
e.g. "the primary region is gone — execute the failover runbook, timed"
```

```
Chaos experiment: often automated, tests the system’s technical resilience
```

A game day is a planned drill that exercises the human side — runbooks, on-call response, communication, and the failover procedure — under realistic but controlled failure. Where an automated chaos experiment asks "does the system survive?", a game day also asks "does the team know what to do, and does the runbook actually work?". Both build confidence; game days build muscle memory.

Runbooks rot: Game days catch the stale runbook, the expired credential, and the failover step that silently broke three months ago — before a real 3am incident does.

📚 Official docs

- Principles of Chaos Engineering

certs.eknathalabs.com

Q33 How do you do capacity planning for a service?

Capacity

🎙 **Spoken answer · ~75 s**

Capacity planning matches resources to forecast demand with deliberate headroom. I start with the per-unit cost: how many requests per second one replica can serve while meeting the latency target, which I measure with load tests, not guess. Then I forecast demand: organic growth from historical trends, plus known events like launches, sales, seasonality, and marketing campaigns. Next I add headroom: redundancy so that losing a node or an availability zone still leaves enough capacity, commonly N plus one or N plus two, and a buffer for spikes and for the time autoscaling needs to react. I also consider lead times: provisioning hardware, quota increases, reservations, or approvals can take weeks, so planning must start earlier. I validate with load tests and re-check regularly, because performance changes with each release. And I monitor leading indicators such as saturation, queue depth, and growth rate, with alerts that warn well before a limit is reached. Cost matters too: too much headroom wastes money, so I balance it with autoscaling and right-sizing. I document assumptions, so the plan can be updated as reality diverges.

Likely follow-ups

- Why must the per-unit capacity be load tested rather than estimated?
- How much headroom would you keep and how do you decide?

```
1. Establish per-unit cost: requests/s served per replica at target latency
```

```
2. Forecast demand: organic growth + known events (launch, sale, season)
```

```
3. Add headroom: N+1 / N+2 redundancy, plus buffer for spikes
```

```
4. Load test to validate the per-unit number, don’t guess it
```

```
5. Plan lead time for provisioning (esp. on-prem / quota increases)
```

Capacity planning matches resources to forecast demand with deliberate headroom. The foundation is a measured per-unit capacity (load test it; do not assume), a demand forecast that includes known spikes, and redundancy headroom so losing a node or an AZ does not tip you over. Always include provisioning lead time — cloud quota increases and hardware both take time.

Plan for the peak, not the average: A service sized for mean traffic will fall over at the daily peak. Size for peak demand plus the loss of one failure domain.

📚 **Official docs**

- Google SRE Book: Software Engineering in SRE (capacity planning)
- Azure Well-Architected: Performance efficiency

certs.eknathalabs.com

Q34 What is autoscaling — and what are its failure modes?

Capacity

🎙 **Spoken answer · ~73 s**

Autoscaling adjusts capacity to demand automatically. In Kubernetes, the HPA scales the pod count on CPU, memory, or custom metrics like requests per second or queue depth; the VPA right-sizes pod requests and limits; and the cluster autoscaler or Karpenter adds and removes nodes when pods can't be scheduled. The failure modes are important. Scale-up may be too slow for sudden spikes, because metrics lag, pods need time to start, and new nodes take minutes to provision. Flapping occurs when scale-up and scale-down thrash without stabilisation windows. Scaling on the wrong metric, such as CPU for an I/O-bound service, means it doesn't scale when needed. Missing resource requests break utilisation-based scaling. Limits can block it: max replicas, quotas, and cloud capacity. Scaling can also overload dependencies: more app instances means more database connections, so the bottleneck simply moves. And autoscaling cannot fix a retry storm. So I keep headroom for spikes, pre-scale for known events, scale on a metric that reflects load, set sensible min and max, and protect downstream systems with limits and backpressure.

Likely follow-ups

- Why might autoscaling make a database problem worse?
- How would you prepare for a known traffic spike?

```
HPA — scale pods on CPU/mem/custom metric (e.g. req/s, queue depth)
```

```
VPA — right-size pod requests/limits
```

```
Cluster Autoscaler — add/remove nodes when pods can’t schedule
```

```
Failure modes: scale-up too slow for spikes; flapping; metric lag;
```

```
  node provisioning latency; scaling on the wrong metric
```

Autoscaling adjusts capacity to demand automatically, but it is not magic. It reacts with a lag — for sudden spikes you may still need pre-provisioned headroom or pre-warming. Scaling on CPU when the bottleneck is really queue depth scales the wrong dimension. And aggressive thresholds cause flapping (scale up, down, up) that wastes resources and churns connections.

Scale on the bottleneck signal: For a queue worker, scale on queue depth or age, not CPU. The right metric is the one that predicts user-facing latency.

📚 **Official docs**

- Google SRE Book: Software Engineering in SRE (capacity planning)

certs.eknathalabs.com

Q35 Why do retries and naive autoscaling sometimes make outages worse? Reliability Patterns

🎙 Spoken answer · ~78 s

Some outages are self-sustaining, which is called a metastable failure. A brief trigger, such as a traffic spike or a short dependency blip, pushes the system into a bad state where its own behaviour keeps it down even after the original trigger is gone. The classic mechanism is a retry storm: failures cause clients to retry, which adds load to an already struggling service, causing more failures, and more retries, a positive feedback loop. Naive autoscaling can make it worse: adding more instances during the storm sends even more traffic to the overloaded dependency, for instance the database, and new instances start with cold caches which increases the load further. Capacity isn't the missing ingredient; the system is trapped. Recovery requires reducing load, not adding capacity: shed load, drop retries, open circuit breakers, block or throttle clients, and bring traffic back gradually. Prevention is by design: retries with exponential backoff, jitter, and retry budgets, so retries are capped as a fraction of traffic; timeouts; circuit breakers; load shedding; and avoiding fragile, load-dependent modes, such as cold-start behaviour. Testing overload and recovery scenarios finds these traps before production does.

Likely follow-ups

- What is a retry budget?
- Why does adding capacity not fix a metastable failure?

```
Metastable failure: system stuck in a bad state even after the trigger is gone
```

```
Retry storm: failures → retries → more load → more failures (positive feedback)
```

```
Scale-up under retry storm just feeds the overloaded dependency more traffic
```

```
Recovery requires shedding load, not adding capacity
```

Some outages are self-sustaining: a brief trigger pushes the system into a state where its own retries keep it down even after the original cause is gone. Adding capacity or retrying harder pours fuel on the fire. The escape is counter-intuitive — shed load, drain queues, disable retries temporarily — let the system drain to a healthy state, then restore traffic gradually.

This is the metastable failure trap: The instinct to "add more servers" during an overload-driven outage often deepens it. Recovery is about reducing load, not increasing supply.

📚 Official docs

- AWS Builders' Library: Timeouts, retries and backoff with jitter
- Microsoft: Cloud design pattern – Retry

certs.eknathalabs.com

Q36 What are the trade-offs between availability and consistency (CAP) for an SRE?

Reliability Patterns

🎙 Spoken answer · ~77 s

CAP says that when a network partition happens, a distributed system must choose between consistency and availability. If I choose consistency, a CP system refuses writes or reads on the side that can't confirm the latest state, to avoid divergence. That fits configuration stores, financial ledgers, and anything where wrong data is worse than no data. If I choose availability, an AP system keeps accepting writes on both sides of the partition and reconciles later, accepting temporary inconsistency. That fits shopping carts, session data, and social feeds where staleness is tolerable. Real systems are often tunable per operation: quorum settings let you pick consistency or availability per request. PACELC extends the idea: even without a partition, there's a trade-off between latency and consistency. As an SRE, the practical question is per data type: what's the cost to the user and business of unavailability versus of stale or conflicting data? Then I design SLOs accordingly. A payment ledger prioritises correctness, so its availability SLO acknowledges that during failures it will reject operations, while a product catalogue prioritises staying up. It also guides failure testing and runbooks.

Likely follow-ups

- Give an example of a CP and an AP choice in the same product.
- What does PACELC add to CAP?

```
During a network partition you must choose:
```

```
  CP — refuse writes to stay consistent (e.g. config store, ledgers)
```

```
  AP — accept writes, reconcile later (e.g. shopping cart, sessions)
```

```
Most real systems are tunable per-operation (PACELC: also latency vs consistency)
```

CAP says under a network partition you trade availability against consistency. As an SRE the practical question is per-data-type: a payment ledger must stay consistent (reject the write if unsure), while a shopping cart can stay available and reconcile conflicts later. PACELC extends this: even without partitions, you trade latency for consistency on every read.

Map it to the data: Don’t pick CP or AP for "the system" — pick per data flow. Money: consistency. Engagement features: availability. This nuance signals senior judgement.

📚 Official docs

- Jepsen: Consistency models

certs.eknathalabs.com

Q37 How do you set an SLO for a brand-new service with no historical data?

SLI/SLO

🎙 Spoken answer · ~77 s

Without historical data I can't set a credible SLO, so I follow a sequence. First, start from what users expect: what does a good experience look like for this journey? Then instrument the SLIs before choosing a target, and run for a few weeks to measure the real baseline, including normal variation and any incidents. Then set the initial SLO close to the measured baseline, perhaps slightly below, rather than an aspirational number, so it's achievable and the error budget is meaningful. Setting ninety-nine point nine nine percent on day one for a new service is a mistake: you'd immediately blow the budget, or you'd over-invest. I involve product and stakeholders to confirm the target matches business needs and dependencies' reliability, since I can't be more available than the services I depend on. Then I review the SLO regularly, perhaps quarterly, tightening it iteratively as I learn and as the service matures, or loosening if it's costing too much for little benefit. For launch, I may use a lenient, provisional SLO labelled as such. The aim is to start measuring and learning, rather than guess.

Likely follow-ups

- Why is an aspirational SLO on day one a bad idea?
- How do dependencies constrain the SLO you can promise?

```
1. Start from user expectation, not current performance
```

```
2. Instrument SLIs first, run for a few weeks, measure the baseline
```

```
3. Set an achievable initial SLO near the measured baseline
```

```
4. Tighten iteratively as you learn; never set aspirational nines on day one
```

Without data you cannot set a credible SLO, so first instrument the SLIs and observe. Set the initial SLO close to the measured baseline rather than to an aspirational number — an SLO you breach every week trains everyone to ignore it. Treat the SLO as a living target you ratchet tighter as reliability work pays off.

An ignored SLO is worse than none: Set it too strict and the team learns the alert means nothing. Start realistic, earn the right to tighten it.

📚 Official docs

- Google SRE Book: Service Level Objectives
- Google SRE Workbook: Implementing SLOs

certs.eknathalabs.com

Q38 How do you make sure postmortem action items actually get done?

Postmortem

🎙 Spoken answer · ~77 s

The graveyard of reliability is the postmortem whose action items were never completed, and then the same incident recurs. To prevent that, every action has a named owner, a priority, and a linked ticket in the same tracker as feature work, so it competes for time visibly rather than living in a document. High-priority actions, especially from SEV1 incidents, are scheduled with a deadline, not best-effort. I limit the number of actions to those that really matter, because long lists of vague items don't get done; each should be specific, verifiable, and address detection, mitigation, or prevention. I track the completion rate as a team health metric, review overdue actions in a regular meeting, and escalate when they slip. When a similar incident happens again, the review explicitly traces it back to unfinished prior actions, which creates accountability for prioritisation. Leadership support is crucial: engineering managers need to protect time for this work, and the error budget policy can mandate it when the budget is burned. Finally, I close the loop by verifying the fix worked, for example with a test or a chaos experiment.

Likely follow-ups

- How do you keep a postmortem action list short and meaningful?
- What metric tells you the postmortem process is working?

```
Each action: named owner + priority + linked ticket in the normal backlog
```

```
High-priority (esp. SEV1) actions get scheduled, not "best effort"
```

```
Track completion rate as a team health metric
```

```
Tie repeat incidents back to unfinished prior actions in the review
```

The graveyard of reliability is the postmortem whose actions were never done — and then the same incident recurs. Put actions in the same tracker as feature work with owners and priorities, report on their completion rate, and when an incident repeats, explicitly trace it to the action item that was skipped. That feedback loop is what gives postmortems teeth.

Protect the time: If reliability actions always lose to feature work, the error budget policy is the lever — budget exhaustion forces those actions to the top of the queue.

📚 Official docs

- Google SRE Book: Postmortem Culture
- Google SRE Workbook: Postmortem Culture
- Atlassian: How to run a blameless postmortem

certs.eknathalabs.com

Q39 How do you balance reliability against feature velocity in practice?

Fundamentals

🎙 Spoken answer · ~75 s

The error budget is the balancing mechanism, so the debate doesn't have to happen meeting by meeting. When the budget is healthy, the team ships quickly and takes reasonable risks, because the budget exists to absorb those risks. When the budget is low or exhausted, effort shifts to reliability work, with risky changes slowed or frozen until the budget recovers. Importantly, one hundred percent reliability is the wrong target: it means you've been shipping too slowly and spending more than users need, and users can't tell the difference between 99.99 and 100 when their own network and devices are less reliable. A service that never burns any of its budget is probably too conservative. This makes the trade-off data-driven and shared between product and engineering, which removes the adversarial dynamic where operations says no and developers push. Practically it needs agreed SLOs that reflect user needs, a written policy approved by both sides, visible dashboards, and consistent enforcement. It also encourages investing in safer delivery, like canaries and automated rollback, which let you move fast while spending less budget.

Likely follow-ups

- Why is 100% reliability the wrong target?
- What makes teams ignore error budget policies?

```
The error budget IS the balancing mechanism
```

```
Budget healthy → ship fast, take risks, the budget absorbs them
```

```
Budget low/gone → shift effort to reliability until it refills
```

```
100% reliability is the wrong target — it means you shipped too slowly
```

Reliability and velocity are not enemies to be argued about meeting-by-meeting — the error budget makes the trade-off automatic and data-driven. A service that never burns its budget is over-invested in reliability and under-shipping. One that constantly overspends is reckless. The budget keeps you in the productive middle without politics.

Say "100% is the wrong target": Interviewers love this line. The right reliability is the level that keeps users happy and leaves room to ship — usually a specific number of nines, never all of them.

📚 Official docs

- Google SRE Book: Embracing Risk (error budgets)

certs.eknathalabs.com

Q40 Design the reliability practice for a 30-engineer startup moving to SRE.

Fundamentals

🎙 Spoken answer · ~78 s

For a thirty-engineer startup, I'd sequence this to fit the organisation's size and avoid process overload. Phase one, instrumentation: golden signals for the key services and one or two SLIs per critical user journey, since you can't manage what you can't measure. Phase two, set realistic SLOs and error budgets from the measured baselines, agreeing them with product. Phase three, alerting and on-call: replace cause-based noise with SLO burn-rate alerts, delete unactionable alerts, and set up a sustainable rotation with a primary and secondary, which at this size may be shared between developers. Phase four, incident process: lightweight roles with an incident commander, severity levels, a status page, and blameless postmortems with tracked action items. Phase five, error budget policy that the leadership actually backs. Then ongoing: toil tracking and automation, safer deployments with canaries and rollbacks, runbooks for the top failure modes, load testing and capacity planning, and chaos or game days later. Culturally, I'd start by embedding reliability into the teams rather than creating a separate gatekeeping SRE team too early, and show quick wins to earn trust. Measure progress by incident trends and developer experience.

Likely follow-ups

- Would you hire a dedicated SRE team first or embed practices in dev teams?
- What would you do in the first 30 days?

Principal-level question — culture, process, and tooling, sequenced sensibly for a small org.

```
Phase 1: Instrument golden signals; define 1–2 SLIs per critical journey
```

```
Phase 2: Set realistic SLOs + error budgets from measured baselines
```

```
Phase 3: SLO burn-rate alerting; delete cause-based noise; sane on-call rotation
```

```
Phase 4: Incident process — IC roles, SEV levels, status page, blameless postmortems
```

```
Phase 5: Error budget policy signed by product; track toil; automate the worst
```

```
Phase 6: Game days + chaos experiments once the basics are solid
```

For a small org, do not stand up a separate SRE silo on day one — embed SRE practices into the existing teams. Sequence matters: you cannot have error budgets without SLOs, and you cannot have SLOs without instrumentation. Get measurement and a humane on-call in place first, then process, then the advanced proactive work.

Culture is the hard part: The error budget policy only works if product leadership signs the feature-freeze clause in advance, and postmortems only work if they are genuinely blameless. Tooling is the easy 20%; the cultural buy-in is the 80%.

certs.eknathalabs.com

📚 **Official docs**

- Google SRE Book: Service Level Objectives
- Google SRE Book: Embracing Risk (error budgets)

Q41 What are the main SLI types (availability, latency, throughput, correctness, freshness, durability)?

SLI/SLO

🎙 **Spoken answer · ~74 s**

There are several main SLI types, chosen to match the kind of service. Availability is the proportion of valid requests served successfully. Latency is the proportion of requests faster than a threshold, expressed as a ratio, not an average. Throughput is how many requests per second the system sustains, often more of a capacity indicator. Correctness is the proportion of responses that are actually right, which matters for data pipelines and computation. Freshness is the proportion of data that is younger than some age, relevant for caches, replicas, and batch pipelines. Durability is the proportion of stored data that isn't lost over time, important for storage systems. Quality and coverage can also be useful, for example the proportion of data processed. I pick by service type: request-driven services use availability and latency; pipelines use freshness, correctness, and throughput; storage uses durability and availability. Each is expressed as good events over valid events so it can feed an SLO and an error budget. The thing to avoid is picking whatever's easy to measure instead of what reflects user experience.

Likely follow-ups

- Which SLIs would you choose for a data pipeline?
- Why express every SLI as a ratio?

```
Availability — proportion of valid requests served successfully
```

```
Latency      — proportion of requests faster than a threshold (a ratio, not an avg)
```

```
Throughput   — requests/s the system sustains (often a capacity SLI)
```

```
Correctness  — proportion of responses that are actually right (data pipelines)
```

```
Freshness    — proportion of data younger than X (caches, replicas, pipelines)
```

```
Durability    — proportion of stored data not lost over time (storage systems)
```

Different services need different SLI types. A request/response API cares about availability + latency; a data pipeline cares about freshness + correctness; a storage system cares about durability. Picking the right SLI *type* for the service is the first design decision — measuring latency on a batch pipeline misses the point entirely.

Match SLI type to the user harm: Ask "what does it mean for this service to fail the user?" A stale recommendation (freshness), a wrong total (correctness), and a slow page (latency) are different failures needing

certs.eknathalabs.com

different SLIs.

📚 **Official docs**

- Google SRE Book: Service Level Objectives
- Azure Well-Architected: Performance efficiency

Q42 Why are averages dangerous for latency SLIs, and how do percentiles fix it?

SLI/SLO

🎙 **Spoken answer · ~78 s**

An average blends the fast majority with the slow tail into one misleading number. An average latency of two hundred milliseconds could hide ninety-five percent of requests at fifty milliseconds and five percent at three seconds, which is a terrible experience for those users. A single extreme outlier can also swing the average, while a thousand slow requests can hide inside it. Percentiles describe the distribution. P50, the median, is the typical experience. P95 and P99 describe the unhappy tail, where users are most likely to complain and churn; P99 means ninety-nine percent of requests are faster than this. P99.9 matters at scale, because one in a thousand requests is still many users, and a page that makes several backend calls is more likely to hit a slow one. For SLOs, I express the latency SLI as the proportion of requests faster than a threshold, for example ninety-nine percent under three hundred milliseconds, which works directly with error budgets. A caveat: percentiles can't be averaged across instances, so I use histograms and compute percentiles from aggregated buckets, and I keep in mind that high percentiles need enough samples.

Likely follow-ups

- Why can't you average percentiles across instances?
- How would you define a latency SLO using a threshold?

```
Avg latency 200ms can hide: 95% at 50ms, 5% at 3s (a terrible tail)
```

```
P50 (median) — typical experience
```

```
P95 / P99 — the unhappy tail; where real users churn
```

```
P99.9 — rare but visible at scale (1 in 1000 requests)
```

An average blends the fast majority with the slow tail into one misleading number, and a single slow outlier can swing it while a thousand can hide in it. Percentiles describe the distribution: P99 = "99% of requests were at least this fast." Users remember the slow requests, so SLOs are set on tail percentiles (P95/P99), not the mean.

The fan-out trap: If one page makes 100 backend calls, a 1% (P99) per-call slowness means most *pages* hit at least one slow call. Tail latency compounds dramatically with fan-out — which is why P99 of dependencies matters so much.

certs.eknathalabs.com

📚 **Official docs**

- Google SRE Book: Service Level Objectives
- Prometheus: Histograms and summaries

Q43 What's the difference between time-based and event/request-based SLO windows?

SLI/SLO

🎙 **Spoken answer · ~77 s**

They count different things. A request-based, or event-based, SLO is the ratio of good requests to valid requests over the window. A ninety-nine point nine percent target means at most zero point one percent of individual requests may fail. It naturally weights by traffic, so an outage during peak hours burns much more budget than the same outage at three in the morning, which matches the real impact on users. A time-based SLO measures the proportion of time intervals that were good: was each minute good or bad? Ninety-nine point nine percent allows about forty-three minutes of bad minutes in thirty days. It's easier to explain and reason about, and suits services with low or uneven traffic, where request counts are tiny, and when you need a simple uptime commitment, as in contracts. The drawback is that it treats a minute with one failed request the same as a minute of total outage unless you define a bad minute carefully, and it ignores traffic volume. I usually prefer request-based SLOs for user-facing services, and time-based ones where request counts are low or for external agreements.

Likely follow-ups

- Why does a request-based SLO weight peak-time outages more?
- When would a time-based SLO be a better fit?

```
Request-based: good_requests / valid_requests over the window (a ratio of events)
```

```
  99.9% = at most 0.1% of individual requests may fail
```

```
Time-based:   good_minutes / total_minutes (was each minute "good"?)
```

```
  99.9% uptime = at most ~43m of "bad minutes" per 30 days
```

Request-based SLOs count individual events and naturally weight by traffic — an outage during peak burns more budget than the same outage at 3am. Time-based SLOs ask "was this minute healthy?" regardless of how many requests it held, which can over- or under-count impact. Request-based is usually preferred for user-facing APIs because it tracks actual user harm.

Traffic weighting matters: A 10-minute outage that hits your busiest hour harms far more users than one at the quietest. Request-based SLOs capture that automatically; time-based ones treat every minute equally.

📚 **Official docs**

- Google SRE Book: Service Level Objectives

certs.eknathalabs.com

Q44 How do you handle SLIs across a multi-step user journey with fan-out?

SLI/SLO

🎙 Spoken answer · ~79 s

Reliability multiplies across a chain of dependencies, so a user journey with several steps can be much less reliable than any single step. Take a journey of login, search, add to cart, and checkout. If each step is ninety-nine point nine percent reliable, the journey is only about ninety-nine point six percent reliable, since zero point nine nine nine to the fourth power is roughly zero point nine nine six. There are two approaches. One is per-step SLIs, which give a clear picture of where problems are, with the journey counted good only if all critical steps succeed. The other is to measure the end-to-end journey success directly where I can, for example with synthetic journeys or by tracing sessions, which reflects what users experience. Often I use both: journey SLIs for the SLO and error budget, and per-step SLIs for diagnosis. Fan-out adds complexity: a request that calls many backends fails if any critical one fails, and tail latency compounds, so per-dependency SLOs need to be tighter than the journey's. I also distinguish critical from non-critical steps, so degradation of optional features doesn't count as a failed journey.

Likely follow-ups

- Why does reliability get worse as you chain dependencies?
- How would you measure an end-to-end journey SLI?

```
Journey = login → search → add-to-cart → checkout (each a critical step)
```

```
Option A: per-step SLIs, journey "good" only if ALL critical steps succeed
```

```
Compound availability: 0.999 × 0.999 × 0.999 × 0.999 ≈ 0.996 (worse!)
```

```
Option B: measure the end-to-end journey success directly when possible
```

Reliability multiplies across a chain of dependencies, so a journey of four 99.9% steps is only ~99.6% reliable end-to-end. Either hold each critical step to a tighter SLO so the product meets target, or instrument the journey outcome directly. The key insight: you cannot set every microservice to the product SLO and expect the product to meet it.

Dependencies set your ceiling: Your service can never be more available than the product of its hard dependencies. Map the critical path and budget each step's SLO so the multiplied result hits the journey target.

📚 Official docs

- Google SRE Book: Service Level Objectives
- Go concurrency patterns: Pipelines and cancellation

certs.eknathalabs.com

Q45 What is the 'valid events' denominator and why does excluding events matter? SLI/SLO

🎙 Spoken answer · ~79 s

An SLI is the ratio of good events to valid events, not to all events, and the denominator defines what you're held accountable for. Some events are legitimately excluded from the denominator: client errors like 400 responses caused by malformed input, which are not the service's fault; rate-limited requests returning 429, because the system worked as designed; health checks and synthetic probes, which are not real users; and requests during announced planned maintenance, if the policy says so. What must never be excluded are real server errors, timeouts, and your own bugs, including responses that return a success code with wrong data, since the user still got a bad outcome. Excluding too much is a way of fooling yourself: a team that classifies every failure as client error will have a great-looking SLO and unhappy users. So I define the valid events carefully and in writing, review exclusions, and sanity-check them with user complaints and support tickets. A caveat: a spike in 4xx may still signal a problem, such as a broken client release or confusing API, so I monitor those separately even if they don't count against the SLO.

Likely follow-ups

- Why shouldn't 5xx errors ever be excluded?
- How do you stop teams gaming the denominator?

```
SLI = good events / VALID events  (not / all events)
```

```
Legitimately excluded from the denominator:
```

```
  4xx client errors (bad input — not your fault), rate-limited 429s,
```

```
  health checks & synthetic probes, requests during planned maintenance
```

```
NEVER exclude: real 5xx, timeouts, your own bugs returning 200-with-wrong-data
```

The denominator defines what you are held accountable for. A user sending malformed input (400) is not your reliability failing, so it should not burn budget. But excluding too much is gaming the SLO — if you quietly drop "slow" requests from the denominator, the SLI looks great while users suffer. Define exclusions explicitly and honestly.

Watch the 429 nuance: Rate-limiting *your own* capacity shortfall as 429 and excluding it can hide a real capacity problem. Exclude abusive-client throttling, but count throttling caused by your under-provisioning.

📚 Official docs

- RFC 6585: HTTP 429 Too Many Requests
- Microsoft: Cloud design pattern – Throttling

certs.eknathalabs.com

Q46 How should you treat planned maintenance and dependency outages in an SLO?

SLI/SLO

🎙 Spoken answer · ~77 s

Users don't care whether an outage was planned or caused by a third party; they experienced downtime. So the mature position is that both normally count against the SLO. For planned maintenance, there are two options: announce the windows and either explicitly exclude them or budget for them. But the better long-term answer is to aim for zero-downtime deployments and maintenance, with rolling updates, blue-green, and online migrations, so there's nothing to exclude. If exclusions are used, they should be rare, communicated, and agreed in the SLO document, otherwise exclusions become a loophole. For dependency outages, the failure usually counts against you, because the user still sees your service failing. The exception is a contractual carve-out for a hard dependency outside your control, and even then, I'd track the impact internally. The better engineering response is to reduce coupling to dependencies with timeouts, fallbacks, caching, and graceful degradation, and to hold dependencies to their own SLOs, so your SLO is not capped by theirs. When my SLO is higher than the product of my dependencies' availability, that is a signal the design needs redundancy.

Likely follow-ups

- Why should a vendor's outage still count against your SLO?
- How do you reach an SLO higher than a dependency's availability?

```
Planned maintenance: announce windows; either exclude or budget for them explicitly
```

```
  – better long-term: aim for zero-downtime deploys so there's nothing to exclude
```

```
Dependency outage: usually COUNTS against you (the user still sees failure)
```

```
  – unless you have a contractual hard dependency carve-out
```

Users do not care whether the outage was "planned" or caused by a third party — they experienced downtime. The mature stance: count dependency failures against your SLO because that pressure drives you to add fallbacks and redundancy. For maintenance, prefer engineering away the need (rolling/blue-green deploys) over carving out exclusion windows that erode user trust.

Counting dependency failures is a feature: If a flaky third party burns your budget, that is the system correctly telling you to add a circuit breaker, cache, or fallback — not an unfair penalty to exclude.

📚 Official docs

- Google SRE Book: Service Level Objectives

certs.eknathalabs.com

Q47 What is an SLO dashboard and what should it show at a glance?

SLI/SLO

🎙 Spoken answer · ~78 s

An SLO dashboard turns reliability into something a non-engineer can read in five seconds: are we meeting the target, and how much room do we have left? At a glance it should show the current SLI against the SLO target, so we know right now whether we're compliant. It shows the error budget remaining over the rolling window, both as a percentage and in absolute terms such as minutes or number of requests. It shows burn rate over several windows, for example one hour, six hours, and three days, so we can see whether we're burning faster or slower and whether the trend is up or down. It shows the projected time to exhaustion at the current burn rate. And it shows the history of the SLI across the full window with the SLO line drawn on it, plus annotations for deploys and incidents so spikes can be explained. I'd keep it simple, one page per service, linked to alerts and runbooks, with drill-downs for engineers. The audience matters: leadership reads the budget status, engineers use the burn rate and breakdown by endpoint or region to investigate.

Likely follow-ups

- Why include deploy annotations on an SLO dashboard?
- What does time to exhaustion tell you?

```
Current SLI vs SLO target (are we meeting it right now?)
```

```
Error budget remaining (% and absolute) for the rolling window
```

```
Burn rate over multiple windows (1h, 6h, 3d) — trending up or down?
```

```
Time-to-exhaustion at current burn rate
```

```
History: SLI over the full window with the SLO line drawn
```

An SLO dashboard turns reliability into something a non-engineer can read in five seconds: are we okay, how much budget is left, and how fast are we spending it. The most actionable single number is "days until budget exhausted at current burn" — it converts a trend into an urgency. Product and engineering should look at the same dashboard.

Make it shared and boring: The SLO dashboard should be the neutral arbiter in ship-vs-stabilize conversations. If product and engineering trust the same numbers, the debate becomes arithmetic instead of opinion.

📚 Official docs

- Google SRE Book: Service Level Objectives
- Google SRE Workbook: Implementing SLOs
- Grafana documentation

certs.eknathalabs.com

Q48 Rolling window vs calendar window for SLOs — what's the trade-off?

SLI/SLO

🎙 **Spoken answer · ~76 s**

The window defines the period over which the SLO is evaluated, and the trade-off is between smoothness and simplicity. With a rolling window, such as the trailing twenty-eight days, the budget continuously recovers as old errors age out. It's smooth, with no artificial reset day that people can game, and it reflects recent user experience at any moment. The drawback is that it's harder to explain to stakeholders, because there's no clear month end, and a bad incident keeps affecting the budget for the entire window length. A calendar window, such as each month, resets the budget on the first. It's simple to communicate, aligns with business reporting and contracts, and gives a clean fresh start. But it creates perverse incentives: late in a good month, teams may take bigger risks because they have budget to spare, and early in a month after a reset, a bad end of the previous month seems forgotten. I generally prefer rolling windows for operational decisions and alerting, and use calendar reporting for communication to leadership or for contractual SLAs. Whatever I choose, I document it clearly.

Likely follow-ups

- Why can calendar windows encourage risky behaviour at month end?
- Which would you use for contractual SLAs?

```
Rolling (e.g. trailing 28 days): budget continuously recovers as old errors age out
```

```
  + smooth, no artificial "reset day" gaming; − harder to explain to stakeholders
```

```
Calendar (e.g. each month): budget resets on the 1st
```

```
  + simple to communicate, aligns with reporting; − incentivises risky behaviour
```

```
    late in a good month and a "fresh start" hides a bad end-of-month
```

A rolling window is generally preferred operationally: an incident's budget impact fades gradually over the trailing period rather than vanishing at an arbitrary reset. Calendar windows are easier for business reporting but create perverse incentives — teams "spend" leftover budget recklessly before a reset, or a disaster on the 31st is forgiven on the 1st.

Common hybrid: Use a 28-day rolling window for alerting/operations (no gaming, smooth) and report a calendar-month summary to leadership for alignment with business cycles.

📚 **Official docs**

- Google SRE Book: Service Level Objectives

certs.eknathalabs.com

Q49 How do SLOs change for batch/data pipelines vs request/response services? SLI/SLO

🎙 Spoken answer · ~75 s

Batch and data pipelines need different SLIs because they don't serve individual requests with latency percentiles. You can't sensibly put a P99 latency SLO on a nightly ETL job. For pipelines I use freshness: the output is no older than a given age, meaning the data lands by its deadline. Completeness: all the expected records were processed, with no silent drops. Correctness: the outputs match validation or reconciliation checks. And coverage: the percentage of input successfully processed per run. Throughput can matter too, to make sure the job keeps up with growth. Request-based services use availability and latency per request. A good pipeline SLO might say ninety-nine percent of daily runs complete before six in the morning, with ninety-nine point nine percent of records reconciled. The tricky part is that failures can be silent: a job that succeeds but produces wrong or partial data looks healthy, so correctness checks and data quality monitoring are essential. Alerts focus on missed deadlines and growing lag, not on CPU. Error budgets still apply, counted per run or per record, depending on the SLI.

Likely follow-ups

- Why are silent failures the biggest risk for pipelines?
- How would you define a freshness SLO?

```
Request/response: availability + latency percentiles (per request)
```

```
Batch / pipeline:
```

```
  Freshness  — output is no older than X (data lands by the deadline)
```

```
  Completeness — all expected records processed (no silent drops)
```

```
  Correctness — outputs match validation / reconciliation checks
```

```
  Coverage   — % of input successfully processed per run
```

You cannot put a P99-latency SLO on a nightly ETL job. Pipelines are judged on whether fresh, complete, correct data arrives by the time downstream consumers need it. The user-facing SLI is often "is the data in the dashboard/feature no more than N minutes/hours stale?" — freshness — plus correctness checks that catch silent data corruption.

Silent data errors are the worst failures: A pipeline that runs "successfully" but emits wrong numbers does more damage than one that visibly fails. Correctness/reconciliation SLIs catch the failures that availability metrics never will.

📚 Official docs

- Google SRE Book: Service Level Objectives

certs.eknathalabs.com

Q50 What is error budget burn during a deploy, and how does it gate releases?

SLI/SLO

🎙 Spoken answer · ~78 s

Error budgets and deployments are connected in two ways. First, during a rollout, I use progressive delivery with SLO gates: deploy to a small percentage, such as one percent of traffic, watch the SLI and burn rate on the canary, then ramp to ten, fifty, and a hundred percent, with automatic rollback if the burn rate spikes above a threshold. That catches a bad release while it affects few users and spends little budget. Tools like Argo Rollouts or Flagger automate the analysis against Prometheus metrics. Second, a release gate based on the remaining budget: if the budget is below a threshold, non-critical deploys are blocked or need extra approval, in line with the error budget policy, while reliability fixes still ship. This ties release cadence to actual reliability. It works best when deploys are small and frequent, because each change then has a small blast radius and an easy rollback. To make it trustworthy, the SLI must be sensitive enough to detect regressions quickly, with enough traffic on the canary to be statistically meaningful, and I need an accurate way to attribute budget burn to a deploy.

Likely follow-ups

- What traffic level makes a canary statistically meaningful?
- Which changes should bypass the budget gate?

```
Progressive delivery gated by SLO/budget:
```

```
  canary 1% → watch SLI + burn rate → 10% → 50% → 100%
```

```
  automatic rollback if burn rate spikes above threshold during rollout
```

```
Release gate: if remaining budget < threshold, block non-critical deploys
```

Tie deploys to the budget in two ways. First, *during* a rollout, watch the SLI on the canary and auto-rollback if burn rate jumps — catching a bad release before it hits everyone. Second, *before* a deploy, if the budget is nearly exhausted, the error budget policy blocks risky changes. This makes the release pipeline reliability-aware rather than blindly shipping.

The budget becomes a release valve: Healthy budget → ship freely and fast. Depleted budget → the pipeline itself refuses risky changes. Reliability enforcement moves from meetings into automation.

📚 Official docs

- Google SRE Book: Embracing Risk (error budgets)

certs.eknathalabs.com

Q51 How do you split or allocate an error budget across teams and dependencies?

Error Budget

🎙 Spoken answer · ~78 s

When many services compose a product, the product-level error budget must be divided so each team knows its share. For example, a ninety-nine point nine percent product SLO leaves zero point one percent of budget, which I allocate along the critical path: perhaps zero point zero two to the frontend, zero point zero three to the API, zero point zero two to the database, and zero point zero three to a third-party dependency, with the parts summing to the total. Each owner is accountable for a sub-budget with its own SLO, which should be tighter than the product's since failures compound along a serial path. Shared dependencies are trickier: their failures debit the consuming teams' budgets, which is realistic since users see them, but I also need the platform team to have its own SLO to stand behind. The allocation is a negotiation informed by the measured reliability of each component and the cost to improve it, not an equal split. I revisit it regularly, and use it to decide where reliability investment goes. Distributed tracing and per-component SLIs help attribute the budget burn to the right owner.

Likely follow-ups

- Why must sub-SLOs be tighter than the product SLO?
- How do you attribute budget burn to the right team?

```
Product SLO 99.9% → allocate the 0.1% budget across the critical path:
```

```
  frontend 0.02% + API 0.03% + DB 0.02% + 3rd-party 0.03%  (sums to budget)
```

```
Each owner gets a sub-budget they're accountable for
```

```
Shared dependencies: their failures debit the consuming team's budget
```

When many services compose a product, the product budget must be divided so each team knows its share. Allocate sub-budgets along the critical path that multiply to the product target, and make each team accountable for theirs. This turns a vague "the product is unreliable" into "the payments service overspent its sub-budget by 3x this month."

Attribution drives ownership: Without per-team budget attribution, reliability is everyone's job and therefore no one's. Sub-budgets make the responsible team visible and accountable.

📚 Official docs

- Google SRE Book: Embracing Risk (error budgets)

certs.eknathalabs.com

Q52 What is a 'silver bullet' / budget exception, and when is overriding the policy justified?

Error Budget

🎙 Spoken answer · ~73 s

A budget exception, sometimes called a silver bullet, is a defined and rare escape hatch from the error budget policy. When the budget is exhausted, the policy says feature freeze. But without any exception mechanism, the first time the business genuinely needs to ship, someone will ignore the policy altogether, destroying its credibility. Legitimate overrides are narrow: a security fix, a legal or compliance deadline that must be met, or a change that itself improves reliability. Each override is logged, approved by a named owner who has the authority, such as a director, time-boxed, and reviewed afterwards to see whether it was justified. It's important to keep it rare: if exceptions are frequent, the SLO may be wrong, too strict for the business reality, or the policy lacks support, and that deserves a conversation about adjusting the SLO instead of quietly overriding it. I also watch for overrides becoming routine, track how many were used, and report them. The goal is for the policy to bend under pressure without breaking, while still being taken seriously.

Likely follow-ups

- What does frequent use of exceptions tell you?
- Who should be allowed to approve an override?

```
Budget exhausted → policy says feature freeze
```

```
Legitimate override (rare, documented, time-boxed):
```

```
  – a security fix or legal/compliance deadline that must ship
```

```
  – a fix that itself improves reliability
```

```
Each override is logged, approved by a named owner, and reviewed later
```

The error budget policy must have a defined, rare escape hatch — otherwise the first time the business genuinely must ship during a freeze, people simply ignore the whole policy and it loses all authority. A documented, approved, time-boxed exception ("silver bullet") preserves the policy's credibility while acknowledging reality. The key is that overrides are visible and accountable, not silent.

Unlimited exceptions kill the policy: If overrides are easy and unrecorded, the freeze never actually happens. Make them rare, named, and reviewed — so using one has real friction and visibility.

📚 Official docs

- Google SRE Workbook: Error Budget Policy (exceptions)

certs.eknathalabs.com

Q53 How do you choose burn-rate thresholds and windows for multi-window alerts?

Error Budget

🎙 Spoken answer · ~81 s

The goal of multi-window burn-rate alerts is to balance how quickly I detect a problem, how much budget is spent before I'm alerted, and how quickly the alert resets. A common set, from the Google SRE workbook for a thirty-day SLO: a fast page when the burn rate is fourteen point four times, checked over one hour and confirmed with a five-minute window, which means about two percent of the budget is consumed in an hour. A medium page at six times, over six hours with a thirty-minute confirmation, about five percent of the budget. A ticket for a slow leak at one times, over three days with a six-hour window, which doesn't need a page. The long window confirms the problem is sustained and significant, avoiding noise from blips; the short window confirms it's still happening right now, so the alert clears quickly after recovery rather than staying red for the whole long window. The thresholds come from how much budget I'm willing to lose before a human is notified. I tune them to the service's traffic and SLO, since low-traffic services produce noisy ratios and might need longer windows or time-based SLIs.

Likely follow-ups

- What does a 14.4 burn rate mean in terms of budget consumed?
- Why is the short window needed?

```
Goal: balance detection time, budget spent before alert, and reset time
```

```
Page (fast):  burn 14.4x, alert if 1h AND 5m windows exceed → ~2% budget in 1h
```

```
Page (med):   burn 6x,   alert if 6h AND 30m windows exceed → ~5% budget in 6h
```

```
Ticket (slow):burn 1x,  alert if 3d AND 6h windows exceed → slow leak, no page
```

```
Short window confirms "still happening"; long window confirms "sustained"
```

The Google SRE workbook recipe: pick a small number of (burn-rate, long-window, short-window) tiers that map to how much budget you are willing to spend before being alerted. The high burn rate pages fast (an acute outage), lower burn rates open tickets (slow leaks). The short window is an AND-condition that auto-resets the alert quickly once the problem stops, preventing flapping.

Why two windows per alert: The long window gives precision (real, sustained burn). The short window gives fast reset and confirms the issue is current — together they cut both false pages and stuck alerts.

📚 Official docs

- Google SRE Workbook: Alerting on SLOs

certs.eknathalabs.com

Q54 How do you account for an error budget when traffic is highly variable or seasonal?

Error Budget

🎙 Spoken answer · ~77 s

It depends on whether the budget is request-based or time-based. Request-based budgets self-adjust to variable traffic, because the budget is a percentage of requests: zero point one percent of a hundred million requests is a bigger absolute allowance than zero point one percent of ten million. Peak periods therefore carry more headroom, and an outage at peak burns more budget than the same outage in a quiet hour, which matches the user impact. Time-based budgets don't adjust: a quiet-hour outage costs the same budget as a peak one. A caution for request-based SLOs is that a flood of cheap or abusive traffic, like bots or health checks, can inflate the denominator and mask real errors, so I define valid events carefully and exclude non-user traffic. For seasonal businesses, I consider setting different SLO targets or stricter policy around peak events, such as holiday sales, and planning change freezes and capacity around them. With very low traffic, ratios become volatile and a few errors blow the budget, so longer windows or synthetic probes help. I review SLOs after seasonal peaks to check they still reflect reality.

Likely follow-ups

- How can bot traffic distort a request-based SLO?
- How would you handle a low-traffic service's SLO?

```
Request-based budgets self-adjust: more traffic → bigger absolute budget
```

```
  (0.1% of 100M req > 0.1% of 10M req) — peak periods get more headroom
```

```
Time-based budgets do NOT adjust → a quiet-hour outage costs the same as peak
```

```
Watch: a flood of cheap/abusive traffic can inflate the denominator and mask errors
```

Request-based error budgets naturally handle variable traffic because the budget scales with volume — but that cuts both ways. A spike of low-value traffic enlarges the denominator and can hide real failures in a percentage, so monitor absolute error counts too. For seasonal businesses (e.g. retail peaks), consider tightening SLOs or pre-provisioning headroom during the critical window.

Track both ratio and absolute: A 0.05% error rate sounds fine until it is 0.05% of a billion requests. Alert on burn rate (ratio) but also watch raw error counts so volume changes don't fool you.

📚 Official docs

- Google SRE Book: Embracing Risk (error budgets)

certs.eknathalabs.com

Q55 What's the relationship between error budgets, change freezes, and release cadence?

Error Budget

🎙 **Spoken answer · ~75 s**

Most incidents are triggered by change, so the rate and size of change is the main lever on error budget burn, which is how budgets, freezes, and release cadence connect. With a healthy budget, a high release cadence is fine: frequent releases give fast feedback, and the budget absorbs occasional failures. With a burned budget, I slow down or freeze changes until reliability recovers. The counter-intuitive part is that small, frequent releases usually burn less budget than large, infrequent ones. Each small change has a limited blast radius, is easy to understand and to roll back, and it's clear which change caused a regression. Big batched releases are riskier, harder to debug, and tend to cause the large incidents. So when the budget is low, the answer isn't necessarily to release less often with bigger batches; it's to make changes safer: smaller diffs, canaries, feature flags, automatic rollback, and better testing. Calendar freezes, like around holidays, are a blunt tool, and they can increase risk by accumulating changes beforehand. I'd rather tie restrictions to budget state and invest in safe delivery.

Likely follow-ups

- Why do smaller releases usually cause fewer incidents?
- When is a calendar change freeze justified?

```
Most incidents follow change → change rate is the main budget-burn lever
```

```
Healthy budget → high release cadence is fine (fast feedback, small diffs)
```

```
Burned budget → slow/freeze changes until reliability recovers
```

```
Counter-intuitive: SMALL frequent releases usually burn LESS budget than
```

```
  big infrequent ones (smaller blast radius, easier rollback)
```

Because change is the dominant cause of incidents, the error budget and your release process are tightly coupled — the budget effectively governs how aggressively you may change production. But "slow down" does not mean "big-bang releases": small, frequent, easily-reversible deploys typically burn less budget than large risky ones. A freeze is the emergency brake, not the steady state.

Frequency reduces risk: Counter to intuition, deploying more often (with small diffs + canaries) is usually *safer* than deploying rarely. The freeze is for recovery, not a model for normal operation.

📚 **Official docs**

- Google SRE Book: Embracing Risk (error budgets)

certs.eknathalabs.com

Q56 How do you measure and report on error budget to non-technical leadership?

Error Budget

🎙 Spoken answer · ~77 s

Leadership doesn't need PromQL; they need the error budget framed as a finite, shared resource with clear consequences, in business language. So instead of reporting a ninety-nine point eight percent availability, I say we have thirty-eight percent of this month's reliability budget left, and at the current rate it lasts eleven more days. I translate incidents into budget and business terms: the checkout incident consumed twenty-two percent of the budget, which affected a number of orders. I tie the budget state to the agreed policy consequence, for instance with a healthy budget we ship as planned, and if it falls below a threshold we shift effort to reliability, so leadership sees why a decision is being made. A simple traffic-light status per service, a trend line, and the top contributors to budget burn work well in a monthly review. I also report the cost side: what it would take to move from three nines to four, and what that's worth. And I highlight actions, such as the reliability work planned and its expected effect. The aim is shared understanding so leaders can make trade-offs knowingly.

Likely follow-ups

- How would you explain 99.9% versus 99.99% to an executive?
- What would you put on a one-page reliability report?

```
Translate to business language:
```

```
  "We have 38% of this month's reliability budget left"
```

```
  "At the current rate it lasts 11 more days"
```

```
  "The checkout incident cost 22% of the budget = N affected orders"
```

```
Tie budget state to the agreed policy consequence (ship vs freeze)
```

Leadership does not need PromQL — they need the budget framed as a finite, shared resource with clear consequences. Report remaining budget, time-to-exhaustion, and the business impact of recent incidents in plain terms. The point is to make the ship-vs-stabilize trade-off a calm, numeric conversation that product owns alongside engineering, rather than a recurring argument.

Connect budget to outcomes they care about: "This outage burned a quarter of the budget and delayed X feature" lands far better than a graph of nines. Speak in orders, revenue, and shipping capacity.

📚 Official docs

- Google SRE Book: Embracing Risk (error budgets)

certs.eknathalabs.com

Q57 What belongs in a good runbook, and how do you keep runbooks from rotting? On-Call

🎙 Spoken answer · ~78 s

A runbook turns a three a.m. page into a checklist instead of a research project. For each alert, a good runbook states what the alert means, the user impact, how to confirm it's real, how to mitigate, and how to escalate. It uses concrete, copy-pasteable commands and links to dashboards, with expected output, not prose that requires interpretation. The best ones are specific to the alert, linked directly from the alert's annotation, so the responder finds it in one click, and written for someone who is tired and unfamiliar with the system. Include the safe first actions, how to check whether a mitigation worked, and when to stop and escalate. Runbooks rot because systems change, so I keep them fresh by updating after every incident that touched them, testing them in game days, putting a last-reviewed date on them, and assigning an owner, ideally storing them in version control alongside the service. A good test is whether a new on-call engineer can follow one without help. And where a runbook is just a sequence of commands that never needs judgement, I automate it and link the automation instead.

Likely follow-ups

- How do you keep runbooks from going stale?
- When should a runbook become automation?

```
Per alert: what it means, user impact, how to confirm, how to mitigate, how to escalate
```

```
Concrete commands/links, not prose — copy-pasteable, with expected output
```

```
Link the runbook directly from the alert annotation
```

```
Keep fresh: update after every incident; test in game days; date-stamp
```

A runbook turns a 3am page into a checklist instead of a research project. The best ones are specific and actionable — exact commands, dashboard links, and decision points — and are linked straight from the alert so the responder is not hunting. Runbooks rot the moment the system changes, so the discipline is updating them in every postmortem and exercising them in game days to catch the stale step before a real incident does.

A wrong runbook is worse than none: A confidently-stated stale command can make an incident worse. Treat runbooks as code: review, test, and version them — and prefer linking to automation that can't drift.

📚 Official docs

- Atlassian: Incident management handbook (lifecycle)

certs.eknathalabs.com

Q58 How do you onboard someone to on-call so their first shift isn't terrifying?

On-Call

🎙 Spoken answer · ~73 s

Throwing a new engineer onto the primary on-call rotation cold is how you create panic and mistakes, so I use a graduated path. First, shadow shifts: they ride along with a veteran, seeing real pages and how they are handled, without responsibility. Then reverse shadowing: they drive the response while the experienced engineer observes and coaches. Next, they take the secondary role, as an escalation backup, before becoming primary. Alongside that, I provide the essentials: up-to-date runbooks, an architecture overview, verified access to all the systems and dashboards they'll need, tested ahead of time and not discovered at three a.m., and practice through game days or simulated incidents. I make clear that asking for help and escalating is expected, not a failure, and that incident roles and the escalation path are documented. After their first shifts, I review how it went with them and fix gaps in documentation. I also make sure they know the incident process, severity levels, and communication expectations. The goal is that the first real shift feels familiar, not terrifying.

Likely follow-ups

- What access do you verify before someone goes on call?
- How do you make escalation feel safe for a new engineer?

```
Shadow shifts: ride along on a veteran's rotation before going solo
```

```
Reverse-shadow: they drive, the veteran observes and coaches
```

```
Start as secondary (escalation backup) before primary
```

```
Provide: runbooks, architecture overview, access verified ahead of time, game-day reps
```

Throwing a new engineer onto primary on-call cold is how you create panic and mistakes. A graduated path — shadow, then reverse-shadow, then secondary, then primary — builds confidence and surfaces gaps (missing access, unclear runbooks) while a mentor is present. Verifying their access and tooling *before* the shift avoids the classic "I can't even log in" failure during a real page.

Psychological safety matters: Make explicit that escalating early is encouraged, not a failure, and that no one is blamed for paging for help. A responder who's afraid to escalate is a slower incident.

📚 Official docs

- Google SRE Workbook: On-Call

certs.eknathalabs.com

Q59 What on-call metrics should you track, and what do they tell you?

On-Call

🎙 **Spoken answer · ~74 s**

On-call health is measurable, and the metrics also double as reliability signals. Pages per shift or per week shows workload and noise, and I want it low and sustainable. The percentage of actionable pages shows alert quality: a low percentage means noisy alerts to fix or delete. Pages outside business hours show the burnout risk and can justify follow-the-sun coverage. Time to acknowledge and time to mitigate show process health: slow acknowledgement suggests paging or escalation problems, slow mitigation suggests weak runbooks or tooling. Repeat pages on the same cause reveal a missing permanent fix, which should become an engineering priority. I also track the proportion of pages that are auto-resolved, the distribution of load across engineers, so some aren't carrying much more than others, and qualitative feedback through on-call retrospectives. I review them regularly with the team and set targets, such as fewer than two incidents per shift, and use them to argue for investment in reliability or headcount. The caution is not to turn any single metric into a target that people game; they're diagnostic.

Likely follow-ups

- What does a high repeat-page rate tell you?
- How would you use these metrics to justify engineering time?

```
Pages per shift / per week — workload + noise (target: low, sustainable)
```

```
% actionable pages — alert quality (low % = noisy, fix alerts)
```

```
Pages outside business hours — burnout risk (drives follow-the-sun)
```

```
Time-to-ack, time-to-mitigate — process health
```

```
Repeat pages on the same cause — missing permanent fix
```

On-call health is measurable, and the metrics double as a reliability signal. A high page volume or a low actionable-percentage means your alerting is broken, not that your people need to try harder. Repeat pages on the same root cause reveal action items that were never completed. Tracking off-hours pages protects against the burnout that quietly degrades incident response.

Noise is an engineering bug: Frame a noisy pager as a defect to be fixed (delete/tune the alert, fix the flaky system), not as the cost of doing business. A shrinking, more-actionable pager is a maturing practice.

📚 **Official docs**

- Google SRE Workbook: On-Call

certs.eknathalabs.com

Q60 How should escalation policies be designed?

On-Call

🎙 Spoken answer · ~75 s

An escalation policy guarantees that a page is never silently dropped. The structure: the primary on-call has a set time, for instance five to ten minutes, to acknowledge; if they don't, the system automatically escalates to the secondary. If the secondary doesn't respond, it goes to the team lead or manager, and for the most serious incidents to a wider incident escalation and executive contacts. Severity changes the path: a SEV1 escalates faster and more widely than a SEV3, with an incident commander assigned early. The policy should also define who can pull in specialists, such as database, network, or security engineers, and how to reach them, including their on-call schedules. I test it, because misconfigured escalation, outdated phone numbers, or schedule gaps are common and only found during real incidents. Practicalities: multiple notification channels, such as push, SMS, and phone calls; override schedules for holidays and swaps; and clear documentation so responders know when to escalate. Culturally, escalating early should be encouraged, never penalised. The policy lives in the paging tool, like PagerDuty or Opsgenie, and is reviewed regularly.

Likely follow-ups

- How would you test an escalation policy?
- When should an on-call engineer escalate instead of continuing alone?

```
Primary acks within N minutes → else auto-escalate to secondary
```

```
Secondary → team lead / manager → incident escalation / exec for SEV1
```

```
Severity-based: SEV1 escalates faster and wider than SEV3
```

```
Define WHO can pull in specialists (DBA, security, network) and HOW
```

An escalation policy guarantees a page is never silently dropped: if the primary doesn't acknowledge within a timeout, it automatically moves to the secondary, then up the chain. Tie escalation speed and breadth to severity — a SEV1 should reach leadership fast, a SEV3 should not wake anyone. Crucially, define the path to *specialists* (database, security, networking) so responders aren't stuck when the problem is outside their domain.

Escalating is a strength: The goal is the fastest path to the right expertise, not heroically soldiering alone. A well-designed policy makes "get the right person" a one-click action.

📚 Official docs

- Google SRE Workbook: Incident Response

certs.eknathalabs.com

Q61 What is follow-the-sun on-call and when is it worth the coordination cost?

On-Call

🎙 Spoken answer · ~77 s

Follow-the-sun on-call has teams in two or three time zones each covering their own daytime, so pages are routed to whichever region is awake and nobody is paged at night. The benefit is large: it largely eliminates night pages and the burnout, attrition, and poor decisions they cause, and responders are alert and productive. But it's expensive to coordinate. It needs at least two staffed regions with enough engineers in each, shared tooling and access, and a solid handoff process at each shift change, with a documented state of ongoing incidents, open issues, and risky changes in flight. Knowledge must be shared so each region can handle the full system, which means consistent runbooks, training, and often rotating people across regions. There's also the cost of the headcount and the risk of handoff errors or fragmentation of ownership. It's worth it when the service is truly global with a demanding SLO, pages at night are frequent, and the team is large enough. For smaller teams, alternatives include better alerting hygiene to cut night pages, a sustainable single-region rotation with time off, or compensation and automation.

Likely follow-ups

- What must a good handoff contain?
- When is follow-the-sun not worth the cost?

```
Teams in 2–3 time zones each cover their daytime → no one is paged at night
```

```
Needs: ≥2 staffed regions, shared tooling, solid handoff process
```

```
Handoff: documented state of ongoing incidents at each shift change
```

Follow-the-sun routes pages to whichever region is currently awake, largely eliminating night pages and the burnout they cause. It is powerful but expensive: it requires genuinely staffed teams in multiple time zones and disciplined handoffs so an in-flight incident isn't dropped at the boundary. It is worth it when you have the global headcount and night-page volume is harming retention or response quality.

Handoffs are the failure point: An incident mid-investigation at shift change can stall if state isn't transferred cleanly. A written handoff of open incidents + current hypotheses is mandatory, not optional.

📚 Official docs

- Google SRE Book: Being On-Call
- Google SRE Workbook: On-Call

certs.eknathalabs.com

Q62 How do you handle a flapping alert that fires and resolves repeatedly?

On-Call

🎙 Spoken answer · ~75 s

A flapping alert fires and resolves repeatedly, usually because its threshold sits right at the normal operating boundary of the metric. First I add hysteresis: fire above one threshold and resolve only when the value drops below a lower one, instead of using the same line for both, so small oscillations don't toggle the alert. Second, require a duration, such as a for clause of five minutes in Prometheus, so a momentary blip doesn't page. Third, evaluate over a longer window, or smooth the signal with an average or a rate over time, or better, use a burn-rate alert on an SLO instead of an instantaneous value, since it's inherently more stable. I can also group and deduplicate in Alertmanager and use inhibition to suppress dependent alerts. But I also ask why it's flapping: if the underlying system genuinely oscillates, for example a service restarting in a loop or an autoscaler thrashing, the alert is just the messenger, and the real fix is in the system. And if the alert is not actionable, delete it. Flapping alerts erode trust in paging.

Likely follow-ups

- What is hysteresis in alerting?
- What does flapping often reveal about the underlying system?

```
Add hysteresis: fire above X, resolve only below Y (Y < X) — not the same line
```

```
Require a duration: "for: 5m" so a momentary blip doesn't page
```

```
Evaluate over a longer window / use burn rate instead of instantaneous value
```

```
If the underlying system flaps, fix the system — the alert is a messenger
```

A flapping alert usually means the threshold sits right at the normal operating boundary, so noise crosses it constantly. Fixes: add a `for:` duration so the condition must persist, use hysteresis (separate fire/resolve thresholds), or evaluate over a longer window. But a chronically flapping *metric* often signals a real instability (a service near its limit, GC thrash) — silencing the alert without investigating hides a brewing problem.

Don't just widen until silent: Suppressing a flapping alert can mask a system that's genuinely teetering at the edge of its capacity. Ask whether the flap is the alert's fault or the system's.

📚 Start here (official docs)

- Google SRE Book

certs.eknathalabs.com

Q63 What is the difference between symptom-based and cause-based alerting, and why prefer symptoms?

On-Call

🎙 Spoken answer · ~75 s

Cause-based alerting pages on every internal condition that might be bad: high CPU, disk at eighty percent, a pod restart. That produces dozens of pages, many non-actionable, because those causes often have no user impact, and during a real incident it creates an alert storm that buries the useful signal. Symptom-based alerting pages on user-facing impact: SLO burn rate, error rate, or latency. It produces one page that means users are hurting, and the causes become part of the investigation rather than separate pages. There are good reasons to prefer symptoms. They have far fewer alerts to maintain. They page on what matters to the business, including problems I didn't anticipate, since I don't need to predict every cause. And they're robust to infrastructure changes. Cause-based signals still have a place: as dashboards for diagnosis, as low-urgency tickets for things like a disk filling slowly, and as early warnings for saturation, but not as pages unless the condition is imminent and actionable. The principle is: page on symptoms, investigate causes. That keeps pager fatigue down and focuses attention on user pain.

Likely follow-ups

- Where do cause-based signals still belong?
- How does symptom-based alerting handle failures you didn't anticipate?

```
Cause-based: alert on every internal cause — high CPU, disk 80%, pod restart
```

```
  → dozens of pages, many non-actionable, alert storm during real incidents
```

```
Symptom-based: alert on user-facing impact — SLO burn, error rate, latency
```

```
  → one page that means "users are hurting"; causes become investigation, not pages
```

Cause-based alerting tries to page on everything that *might* be bad and drowns responders in noise — and when a real outage hits, fifty cause-alerts fire at once. Symptom-based alerting pages only when users are actually affected (SLO burn rate), then you diagnose the cause. This collapses many alerts into a few meaningful ones and is the core of the Google SRE alerting philosophy.

Page on the symptom, dashboard the causes: One "checkout error budget burning fast" page replaces a wall of CPU/disk/restart alerts. The causes still live on dashboards for diagnosis — they just don't wake anyone.

📚 Official docs

- Google SRE Book: Monitoring Distributed Systems

certs.eknathalabs.com

Q64 How do you balance automation vs human judgement in incident response (auto-remediation)?

On-Call

🎙 Spoken answer · ~75 s

Auto-remediation removes toil and shrinks recovery time for known failure modes, but it needs boundaries. It's safe to automate things that are well understood, low risk, and frequent: restarting a hung worker, clearing a full temporary directory, scaling out on a clear signal, or failing over a dead replica. I keep a human in the loop for actions that are ambiguous, rare, high blast radius, or that affect data, such as deleting data, failing over a primary with replication lag, or large-scale restarts. Any automation must be observable, so I can see what it did and when; rate-limited, so it can't loop and make things worse, for example repeatedly restarting a service; and have a kill switch to disable it quickly. It should also be idempotent, tested, and alert a human when it fires repeatedly, since repeated remediation signals an underlying problem that needs fixing, not endless patching. I start with runbooks, automate the safe and common steps, and use human approval for risky ones. The risk of over-automation is that it hides problems and causes cascading failures when assumptions break.

Likely follow-ups

- Why must auto-remediation be rate limited?
- What should happen when auto-remediation fires repeatedly?

```
Safe to automate: well-understood, low-risk, frequent (restart hung worker,
  clear a full temp dir, scale out on clear signal, failover a dead replica)
Keep human-in-loop: ambiguous, high-blast-radius, rare, data-affecting actions
Always: auto-remediation must be observable, rate-limited, and have a kill switch
```

Auto-remediation removes toil and shrinks MTTR for known failure modes, but blind automation can amplify a problem — an auto-restart loop that masks a crash, or an auto-scale that feeds an overloaded dependency. Automate the safe, well-understood, frequent responses; keep humans in the loop for novel or destructive ones. Critically, automated actions must be logged, bounded (rate-limited so they can't run away), and overridable.

Automation can hide and amplify: An auto-healer that silently restarts a crashing service every 30s turns a visible incident into an invisible chronic failure. Make auto-actions loud, capped, and easy to disable.

📚 Official docs

- Google SRE Book: Managing Incidents
- Google SRE Workbook: Incident Response
- Google SRE Book: The Evolution of Automation at Google

certs.eknathalabs.com

Q65 How do you decide when to mitigate vs when to keep investigating?

Incident Mgmt

🎙 Spoken answer · ~77 s

The SRE reflex is to mitigate before diagnosing. Users are hurting now, and a rollback that restores service in two minutes beats a root cause analysis that takes an hour while the outage continues. So the default is to mitigate first if a safe mitigation exists: roll back the change, fail over, turn off a feature flag, or shed load. I only investigate before mitigating when there's no safe mitigation, or when the mitigation would destroy evidence I can't capture another way, or when acting blindly could make things worse, such as actively corrupting data where I need to stop it quickly but carefully. Even then I try to capture evidence cheaply first. I time-box the investigation: for instance, fifteen minutes to find the cause, and if not found, we mitigate broadly with the biggest available lever. The incident commander makes this call and keeps the team from falling down a diagnostic rabbit hole. After mitigation, the root cause analysis happens calmly, using logs, metrics, and captured data. This also means being prepared: rollbacks and flags should be ready and tested, so mitigation is cheap.

Likely follow-ups

- When might you investigate before mitigating?
- Why time-box the investigation?

```
Default: mitigate first if a safe mitigation exists (rollback, failover, flag off)
```

```
Investigate-first only when: no safe mitigation, or mitigating destroys evidence
```

```
  needed to stop active data loss/corruption
```

```
Time-box investigation: "15 min to find it, else we mitigate broadly"
```

The SRE reflex is mitigate-before-diagnose: a rollback that restores service in two minutes beats a root-cause hunt that fixes it "properly" in two hours while users suffer. The exception is when the safest mitigation would erase the evidence you need (e.g. wiping a corrupted node) or when no mitigation is known — then you investigate, but under a strict time box so you don't drift into an open-ended debugging session while the page burns.

State your bias out loud: "Is there a safe rollback? Then we roll back now and investigate after." Interviewers want to hear restore-service-first as the instinct, with a clear exception for data-loss scenarios.

📚 Official docs

- AWS Builders' Library: Ensuring rollback safety during deployments

certs.eknathalabs.com

Q66 What is blast radius and how do you limit it during an incident and by design?

Incident Mgmt

🎙 Spoken answer · ~77 s

Blast radius is how much of the system, and how many users, a failure can affect. During an incident, the goal is to contain the spread: drain the bad node, isolate the failing shard or region, turn off the offending feature flag, stop the in-progress rollout, and block the problematic client or tenant. By design, I limit blast radius ahead of time. Partition the system into cells or shards so a failure affects only a fraction of users; isolate tenants so one can't affect others; make regions and availability zones independent; roll out changes progressively, starting at one percent, then a canary, then the fleet, with automatic rollback; use bulkheads to separate resource pools; use circuit breakers and timeouts to stop failures propagating; and separate accounts or environments with scoped permissions. Configuration and global control-plane changes deserve extra care, because they bypass the progressive rollout of code and have affected everything at once in many large outages. The metric I use is, if this component fails or this change is bad, what percentage of users are affected, and I try to push that number down.

Likely follow-ups

- Why are global config pushes especially risky?
- How does cell-based architecture reduce blast radius?

```
During an incident: contain spread — drain the bad node, isolate the failing
  shard/region, disable the feature flag, stop the rollout
By design: cells/shards, per-tenant isolation, region/AZ independence,
  progressive rollout (1%→canary→fleet), bulkheads, circuit breakers
```

Blast radius is how much of the system (and how many users) a failure can affect. In the moment, you shrink it by isolating the failing component — drain it, fail it over, flag it off. By design, you architect so failures are contained from the start: cell-based architecture, per-tenant/per-shard isolation, and independent failure domains mean one broken cell can't take down the whole fleet. Progressive rollouts limit the blast radius of bad *changes*.

Cells contain failure: If users are partitioned into independent cells, a catastrophic bug hits one cell, not everyone. The cost is operational complexity; the payoff is that "total outage" becomes structurally impossible.

📚 Official docs

- AWS: Disaster recovery & availability concepts (Fault isolation boundaries)
- AWS Builders' Library: Workload isolation using shuffle sharding

certs.eknathalabs.com

Q67 How do you run an incident across multiple teams without chaos?

Incident Mgmt

🎙 Spoken answer · ~73 s

A multi-team incident fails when everyone investigates in parallel without coordination, producing duplicated effort and conflicting changes. To avoid that, there is one incident commander for the whole incident, not one per team, as the single decision-maker. There is one incident channel as the single source of truth, with sub-threads or breakout channels per workstream so noise is contained. The commander assigns clear workstreams to team leads, for example one team investigating the database, another handling the customer-facing mitigation, and another communications, so work isn't duplicated and nothing is dropped. A regular sync cadence, say every fifteen minutes, has each lead report status to the commander, who maintains the overall picture. A scribe records the timeline and decisions. Changes to production are announced in the channel before they're made, to avoid people stepping on each other. A dedicated communications lead handles stakeholder updates so responders aren't interrupted. Roles and handoffs are explicit, particularly in long incidents where people rotate. After the incident, a joint postmortem covers cross-team issues such as unclear ownership and escalation paths.

Likely follow-ups

- Why announce production changes in the incident channel?
- How do you hand off command during a long incident?

```
One IC owns the whole incident (not one per team) — single decision-maker
```

```
One incident channel = single source of truth; sub-threads per workstream
```

```
IC assigns clear workstreams to team leads; no duplicated effort
```

```
Regular sync cadence (e.g. every 15 min): each lead reports status to IC
```

Multi-team incidents fail when everyone investigates in parallel with no coordination — duplicated work, conflicting fixes, and no one with the full picture. The fix is a single Incident Commander who owns the whole thing (not one IC per team), one shared channel as the source of truth, and explicit workstream ownership so each team knows its lane. A short, regular sync keeps the IC's mental model current without constant interruption.

Two people fixing the same thing is dangerous: Without coordination, one engineer's rollback can collide with another's hotfix. The IC's core job is preventing conflicting simultaneous actions.

📚 Official docs

- Atlassian: Incident management handbook (lifecycle)

certs.eknathalabs.com

Q68 What's the difference between mitigation, resolution, and recovery?

Incident Mgmt

🎙 Spoken answer · ~75 s

These are distinct states, and conflating them leads to premature all clear calls. Mitigation is stopping the user pain now, with a rollback, failover, or traffic shift: the symptom is gone, but the underlying cause may still be there. Recovery is the service and its SLI being back to normal, with users unaffected, including backlogs drained and caches warm, so it's verified from the user's perspective. Resolution is the underlying cause actually being fixed: the bug is removed, the capacity is added, the configuration is corrected. You can be recovered but not resolved, for example after rolling back a bad release: users are fine, but the bug is still in the codebase and will return when the change is redeployed. The incident process should reflect this. We declare mitigated when impact stops, recovered when the SLI confirms normal operation, and the incident is closed after follow-up work is tracked, with resolution handled through postmortem actions. Distinguishing them also helps with metrics, such as time to mitigate, and with communication, since customers care about mitigation and recovery, while engineers care about resolution.

Likely follow-ups

- Give an example of being recovered but not resolved.
- Which of the three do customers care most about?

```
Mitigation — stop user pain now (rollback, failover) — symptom gone
```

```
Recovery   — service/SLI back to normal, users unaffected
```

```
Resolution — the underlying cause is actually fixed (the bug is gone)
```

```
You can be recovered but not resolved (rolled back, bug still in the code)
```

These are distinct states and conflating them causes premature "all clear" calls. Mitigation removes the immediate user impact; recovery means the system is healthy again; resolution means the root cause is genuinely fixed so it can't recur. A rollback mitigates and recovers but does not resolve — the bug still exists in the codebase. Tracking them separately lets you de-escalate the page (recovered) while the real fix proceeds calmly (toward resolution).

De-escalate on recovery, close on resolution: Once the SLI recovers you can stand down the war room, but the incident isn't truly closed until the cause is fixed and action items are tracked.

📚 Official docs

- Atlassian: Incident management handbook (lifecycle)

certs.eknathalabs.com

Q69 How do you handle a 'gray failure' where the system is partially/intermittently broken?

Incident Mgmt

🎙 Spoken answer · ~77 s

A gray failure is a partial or intermittent failure where the system reports itself healthy while users experience errors or slowness: for example the health check passes but real requests fail, one availability zone is degraded, or five percent of pods are bad. These are the hardest incidents because monitoring says green while customers are unhappy. Detection must come from the user's perspective, through SLO-based measurement, synthetic journeys, and real user monitoring, not just internal health checks. I compare behaviour across dimensions, such as per instance, zone, version, and customer, to find the unhealthy subset, since averages hide it. To mitigate, I shift traffic away from the degraded subset by draining the bad zone or removing suspect instances, and I widen probes so health checks exercise real functionality, including dependencies, not just whether the process responds. Techniques such as outlier detection in load balancers or service meshes automatically eject instances with elevated error rates. For prevention, I use deeper health checks that mimic real requests, per-instance error-rate monitoring, and redundancy so partial failures are tolerated. The key lesson: trust user-facing signals over component self-reports.

Likely follow-ups

- Why do internal health checks miss gray failures?
- How does outlier detection help?

```
Gray failure: system reports healthy but users see errors/slowness
```

```
  e.g. health check passes but real requests fail; one AZ degraded; 5% of pods bad
```

```
Detect: measure from the USER's perspective (SLO), not internal health checks
```

```
Mitigate: shift traffic away from the degraded subset; widen probes to match real usage
```

Gray failures are the hardest incidents: monitoring says green while users suffer, because the health checks test something different from what users actually do. The defense is measuring success from the user's vantage point (end-to-end SLIs, synthetic transactions that mimic real journeys) so the discrepancy surfaces. Mitigation often means draining the subtly-broken subset (a flaky AZ, a bad deploy cohort) even before you fully understand why.

Trust the user signal over the green dashboard: When internal metrics say healthy but users report failure, your monitoring has a blind spot. Believe the users and instrument what they experience.

📚 Official docs

- Gray Failure: The Achilles' Heel of Cloud-Scale Systems

certs.eknathalabs.com

Q70 What are the most common incident causes, and what does that imply for prevention?

Incident Mgmt

🎙 Spoken answer · ~72 s

The dominant cause of incidents is change: deploys, config pushes, feature flag flips, and infrastructure edits. After that come capacity and overload problems, dependency failures, expired certificates or credentials, bad data or migrations, DNS issues, and the classic full disk. The implication for prevention is to invest in safe-change tooling: progressive delivery with canaries, fast automatic rollback, configuration validation and staged rollout, feature flags with kill switches, and pre-deployment checks. Since so many incidents follow a change, the first question during an incident should be what changed recently, so I want a change log that is easy to search and shows deploys and config changes next to the metrics. For the other categories I add targeted controls: capacity planning, load testing and autoscaling for overload; timeouts, retries, and graceful degradation for dependencies; certificate and credential expiry monitoring and automated renewal; disk space alerts with trend-based warnings; and testing of migrations. Looking at the actual distribution of the team's own incidents each quarter shows where reliability investment pays off most, instead of guessing.

Likely follow-ups

- Why is 'what changed?' the first question in an incident?
- How would you analyse your own incident data for patterns?

```
Dominant cause: CHANGE — deploys, config pushes, feature flags, infra edits
```

```
Then: capacity/overload, dependency failure, expired certs/credentials,
```

```
  bad data/migrations, DNS, and the classic full disk
```

```
Implication: invest in safe-change tooling (canary, rollback, config validation)
```

The large majority of incidents are triggered by a change humans made, which is why "what changed in the last hour?" is the highest-yield first question. This directly shapes prevention: the biggest reliability wins come from making change safe — progressive rollouts, automatic rollback, config validation/linting, and pre-deploy checks — plus guarding the boring recurring killers (cert/credential expiry, disk space, DNS) with proactive monitoring.

Reliability is mostly change management: Since change causes most incidents, a great deploy pipeline (canary + auto-rollback + validation) prevents more outages than almost any other single investment.

📚 Official docs

- Kubernetes: Deployments

certs.eknathalabs.com

Q71 How do you preserve evidence during an incident without slowing recovery?

Incident Mgmt

🎙 Spoken answer · ~78 s

There's a real tension: the fastest mitigation, such as killing and replacing a bad node, can destroy the evidence needed to find the cause. The answer is to capture evidence cheaply before destructive mitigation, in seconds or a minute, not hours. For example, snapshot the bad VM or node disk, grab logs, heap dumps and thread dumps, capture process lists and network state, save dashboard screenshots, and note exact timestamps. Then mitigate by draining or replacing, knowing I have a copy to investigate later. Where possible I mitigate in a way that preserves the original: take the instance out of rotation and keep it running isolated, rather than terminating it, if capacity allows. The scribe's timeline is evidence too, recording actions, times, and decisions. Automation helps, for example a standard script or tooling that collects diagnostics on demand or automatically before replacing a node. If the cost of capturing is too high for the severity, such as an active outage with major user impact, restoring service takes priority. For security incidents, the balance shifts toward preservation. The goal is both fast recovery and a good postmortem.

Likely follow-ups

- What would you capture from a bad JVM service before restarting it?
- How can you mitigate without destroying the failing instance?

```
Capture cheaply BEFORE destructive mitigation: snapshot the bad node/VM,
  grab logs/heap/thread dumps, screenshot dashboards, note timestamps
Then mitigate (drain/replace) — you kept a copy to investigate later
The scribe's timeline IS evidence — actions, times, decisions
```

There is a real tension: the fastest mitigation (kill and replace the bad node) can destroy the evidence needed for the postmortem. Resolve it by capturing volatile state cheaply first — snapshot the VM, pull thread/heap dumps and logs, screenshot the dashboards at peak — which takes seconds, then mitigate freely against the copy. Recovery still comes first; you just don't throw away the corpse.

One person captures, the rest recover: Assign someone to grab snapshots/dumps in parallel while others mitigate. You rarely have to choose between speed and evidence if you delegate.

📚 Official docs

- NIST SP 800-61: Incident handling (security incidents)

certs.eknathalabs.com

Q72 When and how do you declare a security incident differently from a reliability incident?

Incident Mgmt

🎙 Spoken answer · ~76 s

Security incidents invert several reliability instincts. In a reliability incident, the aim is to restore service fast, communicate openly, and use the public status page. In a security incident, the first priority is to preserve evidence and limit who knows, using need-to-know channels, because an attacker may be watching, and open discussion could tip them off or leak. I involve security and legal teams early, because there may be regulatory notification obligations, and external communication must be carefully worded and approved. I'd trigger a security incident on suspected breach, data exfiltration, unauthorised access, ransomware, or compromised credentials. Containment must be thought through: abruptly cutting off the attacker might destroy evidence or cause them to cover their tracks or escalate, so the response is coordinated by the security lead. I use separate, secure communication channels, assume that normal ones may be compromised, rotate credentials and secrets carefully, and keep a precise timeline. The roles map to the incident command system but with security specialists leading investigation. Afterwards, a postmortem focuses on how access was gained, what was exposed, and how to detect it earlier.

Likely follow-ups

- Why limit who knows during a security incident?
- What additional teams do you involve and why?

```
Reliability incident: restore service fast, communicate openly, public status page
```

```
Security incident: PRESERVE evidence first, LIMIT who knows (need-to-know),
```

```
  involve security/legal, do NOT tip off the attacker, careful external comms
```

```
Trigger: suspected breach, data exfiltration, unauthorized access, ransomware
```

Security incidents invert several reliability instincts. Where a reliability incident favors broad communication and fast service restoration, a security incident favors confidentiality (need-to-know to avoid alerting the attacker), evidence preservation for forensics and legal, and a different escalation path (security team, legal, possibly law enforcement). Rebooting or "cleaning" a compromised host can destroy forensic evidence and may not remove the attacker. Recognizing which type you're in changes the entire playbook.

Don't run a breach like an outage: Powering off or wiping to "recover" can erase evidence and leave backdoors. Escalate to security, contain at the network layer, and follow the IR plan — not the reliability runbook.

📚 Official docs

- NIST SP 800-61: Incident handling (security incidents)

certs.eknathalabs.com

Q73 What are the three pillars of observability, and how do they work together?

Fundamentals

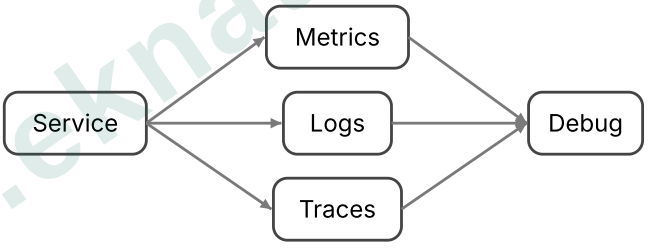
🎙 Spoken answer · ~80 s

Metrics, logs, and traces answer different questions and work together. Metrics are cheap, aggregatable numbers over time, such as rates, latencies, and counts, and they tell me that something is wrong, which is why alerts are built on them. Logs are discrete timestamped events with detail, and they tell me what exactly happened at a point in time. Traces follow a single request across services, with spans showing where time was spent, and they tell me where the problem is in a distributed system. The usual workflow is: a metric alert tells me something is wrong, a trace shows me where in the call chain the latency or error is, and the logs at that span tell me why, with the detailed error message. To make this work, they need correlation: consistent labels and service names, trace IDs in the log lines, and exemplars linking metrics to traces. Each has costs and trade-offs: metrics are cheap but lose detail, logs are rich but expensive at volume, traces need instrumentation and sampling. Observability is more than the three pillars: it's the ability to ask new questions of the system without deploying new code.

Likely follow-ups

- How do you link a metric spike to a specific trace?
- What does high-cardinality data enable that metrics don't?

🗺 Observability pillars




```
Metrics — cheap, aggregatable numbers over time (rates, latencies, counts)
```

```
Logs    — discrete timestamped events with detail (what exactly happened)
```

```
Traces  — the path of one request across services (where time went)
```

```
Workflow: metrics tell you SOMETHING is wrong → traces tell you WHERE
```

```
  → logs tell you WHY (the detailed error at that span)
```

Metrics, logs, and traces answer different questions and are strongest together. Metrics are cheap and great for alerting and dashboards ("error rate is up"), but can't explain a single request. Traces follow one request across service boundaries to localize the slow/failing hop. Logs give the granular detail at that point. The mature flow: alert on metrics, drill with traces, confirm with logs.

certs.eknathalabs.com

Observability vs monitoring: Monitoring answers known questions ("is CPU high?"); observability lets you ask new questions about unknown failure modes after the fact, via high-cardinality data you didn't have to predict in advance.

📚 **Official docs**

- OpenTelemetry: Signals (traces, metrics, logs)

Q74 What is cardinality in metrics, and why does it cause monitoring outages? Fundamentals

🎙 **Spoken answer · ~81 s**

Cardinality is the number of unique combinations of label values on a metric. A request counter with labels for method, status, and endpoint has a small, manageable cardinality. But adding a label like user ID or request ID creates a separate time series for every value, potentially millions, which causes a cardinality explosion. Each series consumes memory and storage in the time-series database, so Prometheus memory blows up, ingestion slows, queries time out, and in the worst case the monitoring system itself falls over, usually during the incident when it's needed most. It also drives cost in hosted services that bill per series. Prevention: review labels before adding them; never put unbounded values like IDs, emails, URLs with parameters, or timestamps in labels; normalise paths into route templates; limit label values; and set limits on series per metric and per scrape in the tooling. When I need per-user or per-request detail, I use logs or traces instead, which are designed for high cardinality. I monitor series counts and growth, and alert on spikes, and find offenders using the cardinality explorer tools. A deploy that adds a bad label is a common cause of monitoring outages.

Likely follow-ups

- Where should per-user detail go if not in metric labels?
- How would you find which metric exploded your series count?

```
Cardinality = number of unique label/tag combinations on a metric
```

```
http_requests{method, status, endpoint} → low, fine
```

```
…{user_id} or {request_id} → MILLIONS of series → explosion
```

```
Effect: Prometheus/TSDB memory blows up, ingestion slows, queries time out
```

Each unique combination of label values creates a separate time series. Adding a high-cardinality label like user ID, request ID, or raw URL with IDs can create millions of series and OOM your metrics backend — ironically taking down the monitoring during an incident. The rule: labels must be bounded, low-cardinality dimensions (status code, method, region), never unbounded identifiers. Unbounded detail belongs in logs/traces, not metric labels.

certs.eknathalabs.com

Cardinality kills monitoring systems: A single innocent `{user_id}` label can OOM Prometheus. Keep metric labels bounded; push per-request detail to traces and logs where high cardinality is expected.

📚 **Official docs**

- Prometheus: Metric and label naming

Q75 **What is distributed tracing, and what are spans, trace context, and sampling?** Fundamentals

🎙 **Spoken answer · ~80 s**

Distributed tracing reconstructs the path of a single request across many services as a tree. A trace represents the whole journey, and it's made of spans, each representing one operation, such as an HTTP call or a database query, with start time, duration, and metadata, nested to show the parent and child relationships. The trace context, containing the trace ID and span ID, propagates between services in headers, typically the W3C traceparent, so each service can attach its spans to the right trace. If any hop drops the header, including through message queues, the trace breaks. Spans are sent to a collector and a backend such as Tempo or Jaeger, where I see a waterfall view showing where time went. Because tracing every request is expensive at scale, sampling is used. Head sampling decides at the start of a request to keep a fraction, which is simple but may drop the interesting ones. Tail sampling decides after the trace completes, so it can keep all slow or error traces and drop boring fast ones, but it requires buffering. OpenTelemetry is the standard for instrumentation. Traces are most useful for latency and dependency analysis.

Likely follow-ups

- Why does tail sampling need buffering?
- What breaks trace propagation across a message queue?

```
Trace = one request's journey; made of nested SPANS (one per operation)
```

```
Trace context (trace_id, span_id) propagates via headers across services
```

```
Sampling: keep only a fraction (head sampling) or decide after (tail sampling)
```

```
  tail sampling can keep all SLOW/ERROR traces, drop boring fast ones
```

Distributed tracing reconstructs a single request's path across many services as a tree of timed spans, so you can see exactly which hop consumed the latency. It works by propagating a trace context (IDs) through request headers so each service stamps its span onto the same trace. Because tracing every request is expensive, you sample — and tail-based sampling is powerful because it can retain all the error/slow traces (the interesting ones) while discarding the routine fast ones.

certs.eknathalabs.com

Tail sampling for the tail latency: Head sampling might drop the rare slow request you most need. Tail sampling decides after the trace completes, so you keep the errors and P99 outliers and throw away the cheap successes.

📚 **Official docs**

- OpenTelemetry: Signals (traces, metrics, logs)
- OpenTelemetry: Sampling

Q76 How do you avoid metrics, logging, and tracing costs spiraling out of control?

Fundamentals

🎙 **Spoken answer · ~75 s**

Observability data grows with traffic and can rival or exceed the cost of the infrastructure itself, so I manage it deliberately. For logs: sample high-volume info and debug logs, keep all errors, use structured logs so I can filter early, drop noisy or useless lines at the collector, and apply tiered retention. For metrics: control cardinality, which is the biggest cost driver, drop unused series and labels, aggregate at the edge or with recording rules, and lengthen scrape intervals where fine resolution isn't needed. For traces: sample, using tail sampling to keep errors and slow requests, and keep routine traces for a short time. Use tiered storage: hot storage for the last few days, with fast queries, then warm, then cold or archive in cheap object storage, retained for compliance. Review usage regularly by team and service, show costs back to teams to create accountability, and set budgets and quotas. Ask for each signal who uses it and why, and delete what nobody reads. Importantly, don't cut the signals that matter for SLOs and incidents; cut waste, not visibility.

Likely follow-ups

- Which of the three signals usually drives the largest cost?
- How would you decide what to stop collecting?

```
Logs: sample high-volume INFO/debug; keep all ERROR; structured + tiered retention
```

```
Metrics: control cardinality; drop unused series; aggregate at the edge
```

```
Traces: sample (tail-sample for errors/slow); short retention for routine traces
```

```
Tiered storage: hot (days, fast) → warm → cold/archive (cheap, slow)
```

Observability data volume grows with traffic and can rival or exceed infra cost. Control it deliberately: sample verbose logs while keeping all errors, cap metric cardinality and prune dead series, sample traces (tail-based to keep the interesting ones), and use tiered retention so recent data is fast/expensive and old data is cheap/archived. The goal is keeping the signal you'd actually query during an incident, not hoarding everything forever.

certs.eknathalabs.com

Observability can cost more than the service: Unbounded log/metric/trace ingestion is a common runaway cloud bill. Treat telemetry volume as a budgeted resource: sample, aggregate, and expire deliberately.

📚 **Start here (official docs)**

- Google SRE Book

Q77 What is structured logging and why does it matter at scale?

Fundamentals

🎙 **Spoken answer · ~77 s**

Unstructured logging writes free-text sentences like user one two three failed checkout timeout, which are hard to query because you need regular expressions to extract anything. Structured logging emits machine-parseable key-value data, usually JSON, with fields such as event, user ID, action, error, and trace ID. At scale, that matters because I can filter, aggregate, and join: count failures by error type, filter by action, break down by customer, and link a log line to its trace through the trace ID. It also makes log pipelines more reliable, since parsing is straightforward, and supports alerting on log-derived fields. I make sure every log line carries a consistent set of fields: timestamp, level, service name, version, trace ID, and request ID, and use consistent field names across services. Practical cautions: avoid logging sensitive data like passwords, tokens, and personal information; keep log levels meaningful; avoid very high-cardinality fields where the backend indexes them, which affects cost; and keep log messages stable so they can be searched. Structured logs turn the logging system into a queryable dataset rather than a text dump, which speeds up debugging significantly.

Likely follow-ups

- Which fields should every log line contain?
- What should never be written to logs?

```
Unstructured: log.info("user 123 failed checkout: timeout")  → hard to query
```

```
Structured (JSON): {event, user_id, action:"checkout", error:"timeout", trace_id}
```

```
Now queryable: count by error, filter by action, JOIN to a trace_id
```

```
Always include: timestamp, level, service, trace_id, request_id
```

Structured logs are machine-parseable key/value (usually JSON) instead of free-text prose. At scale this is the difference between grepping millions of lines and running a query like "count checkout failures by error type in the last hour." The single highest-value field is a correlation ID (trace_id/request_id) that links a log line to its distributed trace and to every other log for the same request — turning three separate tools into one investigation.

Correlation IDs tie it together: Propagate a request/trace ID into every log line and span. Then one ID pivots you across logs, traces, and metrics for a single user's failing request — the fastest path from symptom to cause.

certs.eknathalabs.com

📚 **Official docs**

- OpenTelemetry: Logs

Q78 What is the RED method vs the USE method, and when do you use each?

Fundamentals

🎙 **Spoken answer · ~75 s**

RED and USE are two complementary lenses. RED is for request-driven services: Rate, the number of requests per second; Errors, the number or proportion that fail; and Duration, how long they take. It answers how the service is serving its users. USE is for resources, like hosts, disks, networks, and queues: Utilisation, the percentage of time the resource is busy; Saturation, the amount of extra work it can't service, such as queue length; and Errors, the count of error events. It answers whether the resource is healthy and where the bottleneck is. I use RED for the top-level health of each service and for SLOs, and USE when investigating infrastructure and finding bottlenecks, going resource by resource: CPU, memory, disk, network. The Google golden signals of latency, traffic, errors, and saturation blend both views. A typical workflow: RED shows a service's latency has increased, then USE on the hosts or dependencies it relies on reveals the saturated resource, such as a disk or connection pool. Both give a systematic checklist so I don't rely on random metric hunting during an incident.

Likely follow-ups

- When would you apply USE rather than RED?
- How do golden signals relate to RED and USE?

```
RED (services / request-driven): Rate, Errors, Duration
```

```
  → "how is this service serving requests?"
```

```
USE (resources / hosts): Utilization, Saturation, Errors
```

```
  → "is this resource (CPU, disk, queue) healthy?"
```

```
Golden Signals (Latency, Traffic, Errors, Saturation) blend both views
```

RED is the request-centric lens for services — request rate, error rate, and duration (latency) tell you how a service is treating its callers. USE is the resource-centric lens for infrastructure — utilization, saturation, and errors of a CPU, disk, or queue. They're complementary: RED tells you the service is slow; USE tells you the disk it depends on is saturated. The four golden signals are essentially the union of both.

Two lenses on one system: Use RED to monitor every service uniformly and USE to drill into the resource behind a struggling service. Saturation appears in both because a saturating resource is the leading indicator of trouble.

📚 **Official docs**

- Google SRE Book: Monitoring Distributed Systems

certs.eknathalabs.com

Q79 What are SLO-based alerts vs traditional threshold alerts in practice?

Fundamentals

🎙 Spoken answer · ~83 s

A traditional threshold alert says page if the error rate exceeds one percent. It's a fixed line with no notion of how much unreliability I can afford. That has two problems: it pages on harmless brief blips that consume almost no budget, and it can miss a slow, steady leak that stays just under the line but eventually exhausts the budget. An SLO-based alert pages when I'm burning the error budget fast enough that the SLO is at genuine risk of being breached. It uses burn rate over multiple windows, so a short severe spike and a longer moderate problem are both caught appropriately, and the alert resets properly when the problem ends. The result is higher precision, with pages only when users are meaningfully affected, and fewer false alarms. It also gives a direct way to prioritise: a fast burn pages immediately, a slow burn creates a ticket. Cost: it requires well-defined SLIs and SLOs, recording rules for efficiency, and the team to understand the model, and low-traffic services need special handling. In practice I keep a few simple threshold alerts for things that can't wait, like a total outage, but base the main alerting on SLO burn.

Likely follow-ups

- What kinds of problems do static thresholds miss?
- How do you handle SLO alerting for low-traffic services?

```
Threshold: "page if error rate > 1%" — fixed line, no notion of budget
```

```
  → pages on harmless brief blips; or misses a slow steady leak under the line
```

```
SLO-based: "page if burning budget fast enough to breach the SLO"
```

```
  → precision (only when SLO is genuinely at risk) + good reset behaviour
```

A static threshold doesn't know how much unreliability you can actually afford. It pages on a 30-second spike that costs almost no budget, yet can stay silent during a sustained sub-threshold leak that quietly drains the whole month's budget. SLO/burn-rate alerts tie the page directly to risk of breaching the target, so you're woken when (and only when) the SLO is genuinely threatened — far higher precision and far less noise.

Alert on consequence, not number: "We will breach our reliability target if this continues" is more actionable than "a metric crossed an arbitrary line." That's why mature teams alert on burn rate.

📚 Official docs

- Google SRE Book: Service Level Objectives

certs.eknathalabs.com

Q80 How do synthetic monitoring and real user monitoring (RUM) complement each other? Fundamentals

🎙 Spoken answer · ~78 s

Synthetic monitoring and real user monitoring answer different questions and complement each other. Synthetic monitoring runs scripted probes against critical journeys on a schedule from fixed locations. It gives a consistent baseline, catches outages even at three in the morning when no real users are online, works before a launch or in pre-production, and lets me test specific journeys such as login and checkout from different regions. Its limit is that it covers only what I scripted, from a handful of vantage points. Real user monitoring collects telemetry from actual users' browsers and apps: real devices, networks, geographies, browsers, and behaviours. It captures what synthetics miss, such as a problem affecting a particular mobile carrier, a slow third-party script, or an issue on one browser version, and it reflects true user-perceived performance, like page load and interaction timing. Its limit is that it depends on traffic, so it can't detect problems when no one is using the service, and the data is noisier. I use synthetics for availability alerting and baselines, and RUM for understanding the real experience and feeding SLIs. Together they give both control and realism.

Likely follow-ups

- What can RUM see that synthetic monitoring cannot?
- Why do you still need synthetics if you have RUM?

```
Synthetic: scripted probes hit critical journeys on a schedule from fixed locations
```

```
  + consistent baseline, catches outages even at 3am with zero traffic, pre-prod
```

```
RUM: telemetry from ACTUAL users' browsers/apps
```

```
  + real devices, networks, geographies; catches what synthetics miss
```

Synthetic monitoring runs controlled, scripted transactions so you have a steady signal independent of traffic — it detects a broken checkout at 3am before any real user hits it, and gives a clean baseline for trends. RUM captures the messy reality of real users on real devices and networks, surfacing problems synthetics never script. Together: synthetics for consistent, always-on detection of critical paths; RUM for the true breadth of user experience.

Synthetics detect, RUM explains: A synthetic probe tells you the path is down even with no traffic; RUM tells you that 2% of users on a specific mobile network see failures. You want both.

📚 Official docs

- Google SRE Book: Distributed tracing & logs (Monitoring chapter)
- Prometheus Blackbox exporter

certs.eknathalabs.com

Q81 What is idempotency and why is it essential for retries and reliability?

Reliability Patterns

🎙 Spoken answer · ~77 s

An operation is idempotent if doing it twice has the same effect as doing it once. A PUT that sets a resource to a given state is idempotent; a POST that charges a card is not, because repeating it charges twice. This is essential for reliability because in distributed systems you can never be sure that a request that timed out didn't actually succeed. Retries, at-least-once message delivery, and network ambiguity are everywhere, so the same operation will sometimes be delivered more than once. If operations aren't idempotent, retries cause duplicates: double charges, duplicate orders, repeated emails. To make non-idempotent operations safe, the client sends an idempotency key, a unique identifier for the logical operation, and the server records the key with the result and returns the stored result on repeats instead of processing again. This is how payment APIs work. For message consumers I deduplicate by message ID or design handlers to be naturally idempotent, using upserts and state-based updates rather than increments. Idempotency lets me use retries freely, which is the basis for resilience patterns, and makes recovery from failures simpler and safer.

Likely follow-ups

- How does an idempotency key work end to end?
- Why are retries dangerous without idempotency?

```
Idempotent op: doing it twice == doing it once (no extra side effect)
```

```
PUT /resource/123 {state} — idempotent. POST /charge — NOT (double charge!)
```

```
Make POST safe: client sends an Idempotency-Key; server dedupes by key
```

```
Critical because: retries, at-least-once delivery, and network ambiguity are everywhere
```

In distributed systems you can never be sure a request that timed out didn't actually succeed — so clients retry, and message queues deliver at-least-once. If the operation isn't idempotent, that retry double-charges the card or duplicates the order. Idempotency keys let the server recognize a repeat and return the original result instead of re-executing. This is foundational: safe retries (and thus most reliability patterns) depend on idempotent operations.

The timeout ambiguity: A timeout doesn't mean the request failed — it might have succeeded and the response was lost. Without idempotency, the "safe" retry causes the very damage you feared. Design write paths to dedupe.

📚 Official docs

- RFC 9110: HTTP Semantics (status codes, methods)
- AWS Builders' Library: Timeouts, retries and backoff with jitter
- Microsoft: Cloud design pattern – Retry

certs.eknathalabs.com

Q82 What's the difference between failover, fallback, and failing open vs closed?

Reliability Patterns

🎙 Spoken answer · ~80 s

These are distinct resilience choices. Failover is switching to a standby when the primary dies, such as promoting a database replica or moving to a secondary region, restoring the same capability on redundant capacity. Fallback is serving a degraded alternative when a dependency fails: cached or default data instead of the real thing, so the user gets something rather than an error. Fail open and fail closed describe what the system does when a control or check can't be performed. Fail open means that on failure it allows: if the authentication or rate-limit service is down, let traffic through. Fail closed means on failure it denies: if the payment fraud check is down, reject the transaction. The right choice depends on the risk. Availability-critical, low-risk checks, like a feature flag service or a recommendation filter, should usually fail open. Security and financial controls usually fail closed, because letting everything through could cause serious harm. Some fail partially: allow reads but block writes. I make the choice explicitly per dependency, document it, and test it, because the default behaviour of a library or timeout is often an accident rather than a decision.

Likely follow-ups

- Which would you choose for a failing authorization service, and why?
- How do you test fail-open or fail-closed behaviour?

```
Failover: switch to a standby/replica when primary dies (DB replica, secondary region)
```

```
Fallback: serve a degraded alternative when a dependency fails (cached/default data)
```

```
Fail open: on failure, ALLOW (e.g. auth check down → let traffic through)
```

```
Fail closed: on failure, DENY (e.g. payment check down → reject)
```

These are distinct resilience choices. Failover swaps to redundant capacity; fallback degrades gracefully to a cheaper answer. Fail-open vs fail-closed is the security/safety decision for what to do when a check can't run: a non-critical feature flag service should fail open (don't break the site), but an authorization or payment check should fail closed (deny rather than risk a breach or fraud). Choosing wrong is a classic outage or security hole.

The choice depends on the harm: Fail open when blocking causes more harm than allowing (availability-first); fail closed when allowing causes more harm than blocking (safety/security-first). Decide this deliberately per check.

📚 Official docs

- AWS Builders' Library: Avoiding fallback in distributed systems

certs.eknathalabs.com

Q83 How do you design for multi-region/multi-AZ resilience, and what are the trade-offs?

Reliability Patterns

🎙 Spoken answer · ~76 s

Multi-AZ means spreading a service across availability zones within one region. It survives the loss of a zone, has low latency between zones, allows synchronous replication, and is the baseline for high availability, relatively cheap and simple. Multi-region survives the loss of an entire region, which is rare but severe, but it brings real costs and complexity: cross-region latency forces asynchronous replication, which means data lag, possible data loss on failover, and conflict resolution if both regions accept writes. It also costs more, because of duplicated capacity and data transfer, and demands harder operations. The strategies form a spectrum. Backup and restore is cheapest and slowest. Pilot light keeps minimal core components running. Warm standby runs a scaled-down copy that can scale up. Active-active serves traffic from multiple regions simultaneously, offering the lowest recovery time but the highest complexity, especially for data. I choose based on RTO and RPO targets, the SLO, the cost of downtime, and the team's operational maturity. For most services multi-AZ is enough, and multi-region is reserved for the most critical ones. Whatever I pick, I test failover regularly.

Likely follow-ups

- How do RTO and RPO drive the choice of strategy?
- What makes active-active so hard for data?

```
Multi-AZ: spread across availability zones in one region — survives an AZ loss,
  low latency, synchronous replication feasible — the baseline for HA
Multi-region: survives a whole-region outage — but cross-region latency forces
  async replication → data lag, conflict resolution, much higher cost/complexity
Active-active vs active-passive (warm standby) vs pilot-light vs backup-restore
```

Multi-AZ is the standard high-availability baseline: zones are close enough for synchronous replication, so you survive a datacenter failure with no data loss. Multi-region defends against losing an entire region but the physics of distance forces asynchronous replication — meaning potential data loss (nonzero RPO), conflict handling, and steep cost. The DR spectrum (backup-restore → pilot-light → warm standby → active-active) trades cost against RTO/RPO; pick the cheapest tier that meets the business's recovery objectives.

Multi-region is not free HA: It introduces async replication lag, split-brain risk, and data-conflict resolution. Most workloads need multi-AZ; reserve multi-region for those whose region-outage cost truly justifies the complexity.

📚 Official docs

- Amazon RDS: Multi-AZ deployments
- AWS Builders' Library: Static stability using Availability Zones

certs.eknathalabs.com

Q84 What is backpressure and how does it differ from load shedding?

Reliability Patterns

🎙 Spoken answer · ~79 s

Both protect an overloaded system but in different ways. Backpressure pushes the constraint upstream: when a component is full, it signals producers to slow down, using bounded queues, flow control, TCP windowing, or reactive streams, so the whole pipeline paces itself to the slowest part. It says I'm full, send less. Load shedding drops work that can't be handled, returning errors like 429 or 503 quickly: I refuse this one. Backpressure propagates the limit through the system and avoids waste, but it only works when upstream callers can and will slow down, for example in controlled pipelines or between internal services. For open, user-facing traffic, you cannot make the internet slow down, so you need shedding at the edge. In practice I combine them: bounded queues and concurrency limits that apply backpressure internally, and prioritised shedding at the boundary, dropping low-priority work first. The thing to avoid is unbounded queues, which hide overload until memory runs out or latency becomes enormous, since the queued requests are often already abandoned by clients. Clients should cooperate with backoff and jitter. Both need metrics, so I can see when limits are being hit.

Likely follow-ups

- Why are unbounded queues dangerous?
- Where does backpressure not work?

```
Backpressure: signal UPSTREAM to slow down (bounded queues, flow control,
  TCP windowing, reactive streams) — "I'm full, stop sending so fast"
Load shedding: DROP work you can't handle (return 429/503) — "I'll refuse this"
Backpressure propagates the limit; shedding discards at the limit
```

Both protect an overloaded system but differently. Backpressure pushes the constraint upstream so producers slow to the rate the consumer can sustain — bounded queues that block or signal when full, rather than growing without limit until OOM. Load shedding instead drops excess work at the overloaded service (fast 429/503) to protect throughput for the requests it does accept. Often you want both: backpressure where the pipeline allows producers to slow, shedding at the edge for traffic you can't push back on.

Unbounded queues are a trap: A queue with no limit doesn't prevent overload — it hides it until memory runs out, turning a slowdown into a crash. Bound queues so they exert backpressure or shed.

📚 Official docs

- Google SRE Book: Handling Overload
- AWS Builders' Library: Using load shedding to avoid overload

certs.eknathalabs.com

Q85 How do you make deployments safe: blue-green, canary, and rolling — trade-offs?

Reliability Patterns

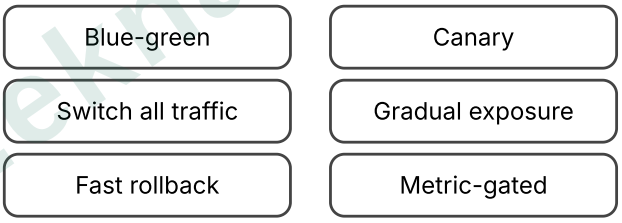
🎙 Spoken answer · ~75 s

All three reduce deployment risk with different trade-offs. A rolling deployment replaces instances batch by batch. It needs no extra capacity and is simple, but old and new versions run simultaneously, so they must be compatible, and rollback is slower because it's another rolling update. Blue-green runs a full parallel new environment and flips traffic atomically. Rollback is instant, just flip back, and I can test the green environment before switching, but it needs roughly double the capacity during the switch, and the cutover exposes all users at once, plus database changes need care. Canary routes a small percentage of traffic to the new version, watches the SLOs, and ramps up gradually. It offers the best risk control, because problems are caught while they affect few users, but it needs good metrics, traffic-splitting capability, and automation to be effective. I often combine them: a canary stage followed by a rolling or progressive rollout, with automatic rollback on SLO regression, and feature flags to decouple deploy from release. Database migrations must be backward-compatible with both versions in any of these strategies.

Likely follow-ups

- When would blue-green be better than a canary?
- How do feature flags separate deploy from release?

🗺 Safe deployment




```
Rolling: replace instances batch by batch — no extra capacity, but mixed versions
  run simultaneously; slower rollback
Blue-green: full parallel new env, flip traffic atomically — instant rollback,
  but ~2x capacity during the switch
Canary: route a small % to the new version, watch SLOs, then ramp — best risk
  control, needs good metrics + automation
```

All three reduce deploy risk differently. Rolling is resource-efficient but you briefly run two versions and rollback means rolling back. Blue-green gives an instant, clean cutover and rollback (just flip the router) at the cost of double capacity. Canary is the safest for catching real-world problems: expose 1% of users,

certs.eknathalabs.com

watch the SLO/burn rate, and auto-rollback on regression before most users are affected. Canary + automated analysis is the gold standard for high-risk changes.

Canary catches what staging can't: Real traffic, real data, real scale expose bugs staging misses. A metrics-gated canary with auto-rollback limits the blast radius of those bugs to a tiny fraction of users.

📚 **Official docs**

- Kubernetes: Deployments
- Google SRE Workbook: Canarying Releases
- Argo Rollouts

Q86 What database reliability patterns matter most: replication, failover, connection pools?

Reliability Patterns

🎙 **Spoken answer · ~75 s**

Databases are usually the hardest part of reliability because they hold state. Replication copies data to other nodes. Synchronous replication means no data loss on failover but adds write latency and can stall writes if a replica is down; asynchronous replication is faster but has lag, and a failover can lose recent writes. Failover is the automated promotion of a replica when the primary fails; the dangers are split-brain, where two nodes both believe they are primary and accept writes, causing divergence, so I need proper fencing and quorum-based leader election, and testing of failover timings. Connection pools cap the number of database connections; without them, an application surge opens too many connections and overwhelms the database, and with them, pool exhaustion makes the app stall under load, so I size pools carefully, set timeouts, and monitor pool saturation. Read replicas scale reads, but replication lag breaks read-after-write consistency, so I route reads that need fresh data to the primary. I add backups with tested restores and point-in-time recovery, monitoring of lag and connections, and I rehearse failover regularly.

Likely follow-ups

- What is split-brain and how do you prevent it?
- How do you handle read-after-write consistency with read replicas?

```
Replication: sync (no data loss, higher latency) vs async (lag, possible loss on failover)
```

```
Failover: automated promotion of a replica; beware split-brain (two primaries)
```

```
Connection pools: cap connections; pool exhaustion = app-wide stall under load
```

```
Read replicas for scale; mind replication lag for read-after-write consistency
```

Databases are usually the hardest part of reliability because they're stateful. Synchronous replication avoids data loss but adds write latency; async is faster but a failover can lose the un-replicated tail (nonzero RPO). Automated failover must guard against split-brain (a fencing/quorum mechanism so two nodes never both

certs.eknathalabs.com

accept writes). And the silent killer under load is connection-pool exhaustion: too many app instances × pool size exceeding the DB's max connections stalls everything — a frequent root cause in postmortems.

Split-brain corrupts data: If a network partition lets two nodes both believe they're primary, you get divergent writes that are painful to reconcile. Proper failover needs quorum/fencing, not just "promote the replica."

📚 **Official docs**

- PgBouncer

Q87 How do you safely run zero-downtime schema migrations on a live database?

Reliability Patterns

🎙 **Spoken answer · ~78 s**

You can't take a busy database offline to change its schema, so I use the expand and contract pattern, also called parallel change. Step one, expand: add the new column or table in a backward-compatible way, nullable or with a default, so the old code keeps working. Step two, migrate: write to both old and new, and backfill existing data in small batches so I don't hold long locks or overload the database. Step three, switch: deploy code that reads and writes the new shape, usually progressively, and verify. Step four, contract: once nothing uses the old column, and after a safe period, drop it in a later release. Each step is independently deployable and reversible, so I can roll back at any point. Things to avoid are blocking locks and big ALTER statements on huge tables during peak traffic; I use online schema change tools, add indexes concurrently, and set lock timeouts so a migration fails fast instead of queuing behind a long transaction and blocking everything. Migrations run as a separate controlled step, not at every pod's startup, and I test them against production-sized data first.

Likely follow-ups

- Why set a lock timeout on migrations?
- What is the risk of renaming a column in one step?

```
Expand/contract (parallel change):
```

```
  1. Expand: add new column/table, backward-compatible (old code still works)
```

```
  2. Migrate: dual-write + backfill data in batches
```

```
  3. Switch: deploy code that reads/writes the new shape
```

```
  4. Contract: drop the old column once nothing uses it
```

```
Avoid: blocking locks, big ALTERs on huge tables during peak
```

certs.eknathalabs.com

You can't take a busy database down to change its schema, and a change must be compatible with both the old and new application code during the rollout. The expand/contract pattern decouples schema change from code change: add the new structure first (non-breaking), backfill and dual-write, switch the code over, then remove the old structure later. This avoids the two classic disasters: a long-locking `ALTER` that freezes the table, and a migration that breaks whichever code version isn't deployed yet.

Schema and code rollouts must overlap safely: Never ship a migration that only works with code that isn't fully deployed. Expand/contract guarantees every intermediate state is runnable and rollback-able.

📚 **Official docs**

- gh-ost: GitHub's online schema migration
- Flyway documentation

Q88 What is a dead man's switch / heartbeat alert, and why monitor for silence?

Reliability Patterns

🎙 **Spoken answer · ~78 s**

A normal alert fires when something bad happens, such as the error rate going up. A dead man's switch, or heartbeat alert, fires when something expected stops happening: a cron job didn't run, an agent hasn't sent a heartbeat in five minutes, or the monitoring pipeline itself went silent. Most alerting watches for bad signals, but what if the thing that emits the signal has died? Then there's no bad signal, just silence, and silence looks like health. To implement it, a job or system pings a watchdog on a schedule, and a missing ping triggers the alert. In Prometheus, an always-firing watchdog alert is routed to an external service, and if that service stops receiving it, it pages, which verifies the whole alerting pipeline end to end. For batch jobs, services like Healthchecks.io or Cronitor expect a check-in and alert on absence. Also use absent() functions for metrics that should always be present. This catches silent failures: backups that stopped, queues that stopped being consumed, data pipelines that halted, or an exporter that crashed. It's an important complement to threshold alerts, especially for monitoring the monitoring.

Likely follow-ups

- How do you verify your alerting pipeline itself works?
- What would you use to detect a silently failing backup job?

```
Normal alert fires when something BAD happens (error rate up)
```

```
Dead man's switch fires when something EXPECTED stops happening
```

```
  e.g. "the cron didn't run", "no heartbeat from the agent in 5m",
```

```
       "the monitoring pipeline itself went silent"
```

```
Implement: a job pings a watchdog on a schedule; missing ping = alert
```

cons.eknathalabs.com

Most alerting watches for bad signals — but what if the thing that emits signals dies? A crashed monitoring agent, a cron that silently stopped, or a broken metrics pipeline produces *no* error, just silence, and a naive system stays green. A dead man's switch (heartbeat) inverts this: it expects a regular ping and alerts on its absence. It's the safeguard that catches "the monitoring is down" and "the backup job hasn't run in a week."

Monitor the monitor: The scariest outages are the ones your dashboards can't see because the dashboards themselves broke. A heartbeat to an external watchdog catches the silence that absence-of-errors would otherwise hide.

📚 **Official docs**

- Prometheus: Alertmanager

Q89 When should you trigger a postmortem, and should near-misses get one? Postmortem

🎙 **Spoken answer · ~74 s**

I define postmortem triggers in advance and in writing, so they aren't a judgement call relitigated under pressure. Always write one for any SEV1 or SEV2 incident, any customer-visible outage, any SLO breach or significant error budget burn, any data loss, and any security incident. Also write them for near-misses and surprising incidents, even if small, such as a failure we only avoided by luck or a change that nearly caused an outage. Near-misses are often skipped, but they're valuable: they offer the same learning without the cost to customers, and they reveal hidden risks before they cause a real incident. A lightweight format for smaller events keeps the burden proportionate, so people don't avoid declaring or reporting. Postmortems should be written soon after, while memory is fresh, reviewed by the team and shared widely, and produce tracked action items. The culture should reward reporting, not punish it. If people fear that writing a postmortem means blame, they'll hide problems. And review the trigger criteria periodically to make sure they capture the right events without creating review fatigue.

Likely follow-ups

- Why write postmortems for near-misses?
- How do you keep postmortem volume manageable?

```
Always: any SEV1/SEV2, any customer-visible outage, any SLO breach
```

```
Also: near-misses ("we got lucky") and surprising incidents, even if small
```

```
Threshold should be written down so it's not a judgement call each time
```

Define postmortem triggers in advance so they aren't relitigated under pressure: all high-severity and customer-facing incidents, and any SLO breach. The often-skipped but high-value case is the near-miss — an incident that almost happened but was caught by luck. Writing those up surfaces the latent weakness before it becomes a real outage, and it's far cheaper to learn from a near-miss than from the failure it predicts.

certs.eknathalabs.com

Near-misses are free lessons: "Where we got lucky" is the section that prevents next quarter's SEV1. A culture that writes up near-misses is one that fixes problems before they bite.

📚 **Official docs**

- Google SRE Book: Postmortem Culture
- Google SRE Workbook: Postmortem Culture

Q90 How do you keep a postmortem genuinely blameless when human action triggered it?

Postmortem

🎙 **Spoken answer · ~78 s**

When a human action, such as running the wrong command, triggered the incident, keeping the postmortem blameless takes deliberate reframing. I change the question from who did it to what allowed this to happen: the missing guardrail, no dry run, no confirmation step, no peer review, a command that looks the same in production and staging, insufficient permissions separation. I assume good intent and reasonable action given the information the person had at the time, and watch for hindsight bias, where the outcome makes the decision seem obviously wrong. I use neutral language: the deploy proceeded, not Alice broke production. I apply a counterfactual test: would a competent peer likely have done the same thing in the same situation? Usually the answer is yes, which shows the system set them up to fail. Then actions target the conditions: add safeguards, make dangerous operations harder, improve tooling and documentation, add automation. Blameless doesn't mean no accountability; it means accountability for improving the system, and the person involved is often the best source of information, so they must feel safe sharing details. Leaders model this from the top.

Likely follow-ups

- What is hindsight bias and how does it distort reviews?
- Does blameless mean no accountability?

```
Reframe "who" into "what allowed this": missing guardrail, no dry-run, no confirm
```

```
Assume good intent + reasonable action given the info at the time (hindsight bias!)
```

```
Use neutral language: "the deploy proceeded" not "Alice broke prod"
```

```
Counterfactual test: would a competent peer likely have done the same? Usually yes
```

When "someone ran the wrong command" is the trigger, blamelessness means treating that as the *start* of the investigation, not the end. The real questions: why did a dangerous command have no confirmation, no dry-run, no guardrail? Guard against hindsight bias — what's obvious after the fact wasn't obvious in the moment with partial information. If a competent engineer would plausibly have made the same choice, the system is the problem, not the person.

certs.eknathalabs.com

Blame destroys your data source: The moment people fear punishment, they stop volunteering the details you need. Blamelessness isn't kindness for its own sake — it's how you get the truth that lets you actually fix the system.

📚 **Official docs**

- Google SRE Book: Postmortem Culture
- Google SRE Workbook: Postmortem Culture
- Atlassian: How to run a blameless postmortem

Q91 What is a 'systems thinking' view of failure vs the 'bad apple' view?

Postmortem

🎙 **Spoken answer · ~76 s**

There are two views of failure. The old bad-apple view says incidents are caused by flawed individuals, so the fix is to find them and remove, punish, or retrain them. The systems view says failure emerges from interactions between components, conditions, pressures, and missing safeguards, and the operator is one component in a system that set them up to fail. People work with incomplete information, time pressure, ambiguous tools, and conflicting goals; usually they did what made sense at the time. So the fix is to change the conditions, not the individual: add guardrails, improve tooling, clarify procedures, reduce complexity, and remove traps. The bad-apple approach has a perverse effect: if people fear blame, they hide mistakes and near-misses, which removes the information needed to improve, making the system less safe. The systems view encourages open reporting and learning. It also recognises that if one person made the error, the next person probably would too unless the system changes. In an interview I'd cite human error as a starting point for investigation, not an end point, and ask why the action seemed reasonable.

Likely follow-ups

- Why does a punitive culture make systems less safe?
- What questions help move from 'human error' to system causes?

```
Bad-apple (old view): failure = a flawed person; fix = remove/punish/retrain them
```

```
Systems view: failure emerges from interactions, conditions, and missing safeguards
```

```
  the operator is one component in a system that set them up to fail
```

```
Fix the conditions, not the individual
```

The "bad apple" theory says incidents are caused by bad individuals and the fix is to find and remove them — but it predicts the next person in the same flawed system will fail the same way, and it teaches everyone to hide mistakes. The systems view (Sidney Dekker, the safety field) sees failure as emerging from the whole system: the conditions, tooling, time pressure, and missing guardrails that made the error likely. You improve reliability by changing those conditions.

certs.eknathalabs.com

The second story: Behind "human error" is always a second story — why that action made sense at the time. Investigating the second story finds the systemic fix; stopping at "human error" guarantees recurrence.

📚 **Official docs**

- Google SRE Book: Postmortem Culture (causes)

Q92 How do you aggregate learnings across many postmortems to find systemic patterns?

Postmortem

🎙 **Spoken answer · ~73 s**

Individual postmortems fix individual incidents; aggregate analysis finds the patterns across them. I tag every incident consistently: cause category, affected system, contributing factors, trigger type, detection method, and whether a detection gap existed. Then I do a regular review, perhaps quarterly, where I cluster incidents: is the same cause recurring, is there a shared blind spot, do many incidents involve the same service or team, is a particular kind of change, like config pushes, the top trigger? If config changes are the top trigger, I invest in config validation and staged rollout; if detection is the issue, in observability and alerting. I also track program health metrics: action item completion rate, repeat incident rate, time to detect and mitigate trends, and the proportion of incidents detected by alerts versus customers. Presenting this to leadership turns incident data into a reliability roadmap with evidence. The review should also check whether past actions actually reduced recurrence. I keep the taxonomy simple so tagging is consistent, and I share the findings widely so teams learn from each other's incidents.

Likely follow-ups

- What would you tag on each incident?
- How do you show that postmortem actions reduced recurrence?

```
Tag incidents: cause category, affected system, contributing factors, detection gap
```

```
Quarterly review: cluster across incidents — same cause recurring? same blind spot?
```

```
Trend: are config changes the top trigger? then invest in config validation
```

```
Track action-item completion + repeat-incident rate as program health
```

Individual postmortems fix individual incidents; aggregate analysis fixes the *system*. By tagging incidents with structured categories and reviewing them in aggregate, you spot the patterns invisible from any single one — e.g. "40% of our SEV2s this quarter were config pushes with no validation," which justifies a specific investment. This turns reactive postmortems into a proactive reliability roadmap and is a hallmark of a mature program.

Look for the meta-pattern: One incident says "fix this bug." Ten incidents tagged the same way say "build the guardrail that prevents this whole class." Aggregate review is where the high-leverage fixes come from.

certs.eknathalabs.com

📚 **Official docs**

- Google SRE Book: Postmortem Culture
- Google SRE Workbook: Postmortem Culture

Q93 What is the difference between proximate cause, root cause, and contributing factors?

Postmortem

🎙 **Spoken answer · ~76 s**

These terms separate the layers of cause. The proximate cause is the immediate trigger, the thing that visibly set off the incident, for example the deploy that broke checkout. Root causes are the deeper systemic reasons that allowed the trigger to cause harm, such as no canary stage, no automated rollback, or no test covering that path. Contributing factors are conditions that combined to make it worse or more likely: peak traffic, a low error budget, a stale runbook, or an on-call engineer new to the service. Stopping at the proximate cause fixes only the surface: we revert the deploy and the next different bad deploy produces the same incident. Root causes are where lasting prevention lives. Contributing factors explain why this happened now and how bad it got, and they often reveal improvements for detection and response. Real incidents usually have several root causes rather than one, so I prefer to speak of causes and contributing factors. A good postmortem lists all three and ensures the action items address each layer: prevent the trigger, reduce the impact, and improve detection and recovery.

Likely follow-ups

- Why do incidents usually have several root causes?
- Give an example of a contributing factor that isn't the cause.

```
Proximate cause: the immediate trigger (the deploy that broke checkout)
```

```
Root cause(s): the deeper systemic reasons (no canary, no rollback automation)
```

```
Contributing factors: conditions that combined (peak traffic + low budget + stale runbook)
```

The proximate cause is what visibly set off the incident, but stopping there fixes only the surface. Root causes are the systemic gaps that let the trigger cause harm. Most real incidents have *multiple contributing factors* that aligned — a latent bug plus a config change plus a missing alert plus peak load. Naming a single "root cause" usually hides the others and breeds false confidence that one fix is enough.

Swiss cheese model: Incidents happen when holes in several layers of defense line up. There's rarely one cause — there's a stack of small gaps. Fix several layers, not just the one nearest the symptom.

📚 **Official docs**

- Google SRE Book: Postmortem Culture (causes)

certs.eknathalabs.com

Q94 How do you balance investing in postmortem action items against new feature work?

Postmortem

🎙 Spoken answer · ~74 s

Reliability work perpetually loses to feature work unless there's a forcing mechanism. So I put reliability actions in the same backlog as features, with real priority, so they compete visibly rather than living in a separate document. High-priority actions, especially those from SEV1 incidents, are scheduled commitments with deadlines, not best effort. The error budget policy is the key lever: when the budget is exhausted, reliability work jumps the queue automatically, and that decision has already been agreed by product, so it isn't a negotiation. I also reserve a regular capacity allocation, for example a percentage of each sprint, for reliability and toil reduction. I track the completion percentage of postmortem actions, and when a repeat incident occurs, I publicly tie it back to the skipped action, which makes the cost of deprioritisation concrete. Presenting the cost of incidents in customer impact and engineering hours helps product weigh them. And I keep action lists short and impactful, because long lists of vague tasks never get done. Leadership support is what makes all of this work in practice.

Likely follow-ups

- How does the error budget policy force reliability work forward?
- Why keep action lists short?

```
Put reliability actions in the SAME backlog as features, with real priority
```

```
High-priority (SEV1-derived) actions: scheduled commitments, not "best effort"
```

```
Error budget policy is the lever: budget gone → reliability work jumps the queue
```

```
Track completion %; tie repeat incidents to skipped actions publicly
```

Reliability work perpetually loses to feature work unless there's a forcing function — and the error budget is exactly that lever: when the budget is exhausted, the agreed policy promotes reliability action items above features automatically, removing the politics. Between freezes, the discipline is treating action items as real prioritized backlog with owners, and making the cost of skipping them visible by tracing each repeat incident back to the fix that was deferred.

The budget ends the argument: Instead of debating "reliability vs features" every sprint, the error budget decides it with data — healthy budget ships features, burned budget ships fixes. That's the whole point of the mechanism.

📚 Official docs

- Google SRE Book: Postmortem Culture
- Google SRE Workbook: Postmortem Culture
- Atlassian: How to run a blameless postmortem

certs.eknathalabs.com

Q95 How do you measure toil so you actually know if you're over the 50% line?

Toil

🎙 Spoken answer · ~73 s

You can't manage toil you don't measure, and it feels like a lot won't justify automation investment. Practical measurement starts with categorising work: tag tickets, interrupts, and on-call tasks as toil, engineering, or overhead. I supplement that with time tracking or periodic surveys, asking engineers what percentage of last week went to manual operational work. I track toil hours per week, the top toil sources ranked by time, and the trend over time, to see if it's growing as the service grows. Tagging on-call interrupts so recurring manual fixes become visible is especially valuable, because the same fix performed repeatedly is a prime automation candidate. Then I compare against the fifty percent guideline, and when it's exceeded I use the data to justify automation work, redirect operational load, or push back on taking on new services. I review it quarterly. A few cautions: measure honestly without turning it into surveillance, define toil consistently so the numbers are comparable, and combine quantitative data with qualitative feedback. The goal is a data-driven conversation about where engineering time goes.

Likely follow-ups

- How do you define toil consistently across a team?
- What data do you need to justify an automation project?

```
Categorise tickets/interrupts: toil vs engineering vs overhead
```

```
Time-tracking or periodic surveys ("what % of last week was manual ops?")
```

```
Track: toil hours/week, top toil sources, trend over time
```

```
Tag on-call interrupts so recurring manual fixes become visible
```

You can't manage toil you don't measure, and "it feels like a lot" won't justify automation investment. Practical measurement: categorize work (toil/engineering/overhead), use lightweight surveys or ticket tagging to estimate the toil fraction, and track the trend. The output is a ranked list of the biggest toil sources — which is exactly your automation backlog — plus an honest read on whether the team is over the 50% cap and drowning.

Make toil visible to justify the fix: "We spend 14 hours/week manually doing X" turns an abstract complaint into a fundable automation project with a clear payback. Measurement is what unlocks the time to escape toil.

📚 Official docs

- Google SRE Book: Eliminating Toil

certs.eknathalabs.com

Q96 What is the 'automation paradox' — when does automating make things worse? Toil

🎙 **Spoken answer · ~76 s**

The automation paradox is that automation removes toil but introduces its own risks. First, automating a flawed or flaky process just makes it fail faster and at larger scale, so I fix the process before automating it. Second, over-automation erodes human skill and situational awareness: when automation handles routine work, operators lose practice and understanding, and when it hits a case it wasn't built for, usually the rare and scary incident, the humans are least prepared to intervene. Third, opaque automation that nobody understands during an incident can make diagnosis harder and cause confusing behaviour, and automation can amplify failures, for example auto-remediation restarting services in a loop, or auto-scaling feeding a retry storm. Mitigations: keep automation simple and well understood, make it observable so humans can see what it is doing, rate-limit it and give it a kill switch, keep manual procedures documented and practise them in game days, and design automation to hand control to a human with clear context when it meets unexpected conditions. The principle is automation that assists humans and fails safe, not automation that replaces understanding.

Likely follow-ups

- How do you keep operators' skills sharp when automation handles routine work?
- Give an example of automation amplifying a failure.

```
Automate a flaky/wrong process → it fails faster and at larger scale
```

```
Over-automation → humans lose the skill + situational awareness to intervene
```

```
  when the automation hits a case it wasn't built for (the rare, scary incident)
```

```
Opaque automation → nobody understands what it does during an incident
```

Automation removes toil but introduces its own risks. Automating a broken process just industrializes the breakage. And the ironies-of-automation problem: the more you automate, the rarer manual operation becomes, so when the automation faces a situation it can't handle, the humans are out of practice and lack context exactly when judgment matters most. Good automation is observable, bounded, and keeps humans able to understand and override it.

Ironies of automation: Automating the routine leaves humans only the hardest, rarest cases — with the least practice. Design automation that explains itself and degrades to a human gracefully, not a black box that fails baffling.

📚 **Official docs**

- Google SRE Book: The Evolution of Automation at Google

certs.eknathalabs.com

Q97 What is 'toil debt' and how is it like technical debt?

Toil

🎙 **Spoken answer · ~74 s**

Toil debt behaves like technical debt. Every manual process I don't automate is a recurring tax that grows with scale. Deferring automation is borrowing time now and paying interest every week, forever. It also compounds: toil consumes the engineering time needed to remove toil, which creates the death spiral. More services means more toil, which leaves less time to automate, which leads to even more toil and, eventually, burnout and attrition, with a team that can only firefight. Like technical debt, small amounts are manageable and sometimes right, such as a quick manual process for something rare, but it needs to be visible, measured, and paid down deliberately. I make it visible by tracking toil sources and their cost, prioritise by frequency and risk, and reserve capacity every sprint to retire some. I use the fifty percent cap as a trigger. When explaining it to management, I describe it in money and time: hours per year consumed, error risk, and opportunity cost. Automating the top few sources typically gives disproportionate returns, which builds the case to continue.

Likely follow-ups

- How do you make toil debt visible to management?
- When is it right to take on toil debt deliberately?

```
Each unautomated manual process is a recurring tax that compounds with scale
```

```
Deferring automation = borrowing time now, paying interest every week forever
```

```
Compounds: toil crowds out the engineering time needed to remove the toil
```

```
The death spiral: more services → more toil → less time to automate → more toil
```

Toil debt behaves like technical debt: every manual process you don't automate is a loan whose interest is paid in operational hours every single week, and it compounds as the system grows. The dangerous feedback loop is that toil consumes the very engineering capacity required to eliminate it — a team past the tipping point can never dig out because there's no time left to build the automation. The 50% cap exists precisely to prevent this spiral.

The spiral is real and one-directional: Once toil exceeds engineering time, the team can only fall further behind. Defend the engineering-time floor aggressively — pushing overflow toil back to product teams if needed — before the spiral starts.

📚 **Official docs**

- Google SRE Book: Eliminating Toil

certs.eknathalabs.com

Q98 How does self-service / platform engineering reduce toil at the org level?

Toil

🎙 Spoken answer · ~76 s

A lot of SRE toil is acting as a manual gateway: provisioning environments, granting access, setting up pipelines, and fulfilling repeated requests from developers. The platform engineering answer is to build a self-service platform with golden paths or paved roads: internal tooling, templates, and automation that let developers do the common things themselves, safely. With guardrails built in, such as policy checks, quotas, standard security settings, and default observability, the safe way is the default way, and the SRE or platform team isn't needed in the loop for routine work. The SRE's job shifts from fulfilling tickets to building and improving the platform. Benefits: it removes a repeated source of toil and waiting, speeds up developers, and improves consistency and security because everything is built from the same vetted templates. The risks: the platform becomes a bottleneck or an unwanted mandate if it's not designed around developers' needs. So I treat it as a product: talk to users, measure adoption and satisfaction, keep an escape hatch for unusual cases, and document well. Success shows as fewer tickets, faster onboarding, and consistent practice.

Likely follow-ups

- How do you stop a platform team becoming a bottleneck?
- What does a golden path include?

```
Toil source: SRE manually fulfilling repeated dev requests (envs, access, deploys)
```

```
Fix: build a self-service platform — golden paths, paved roads, internal tooling
```

```
Devs self-serve common needs safely; SRE builds the platform, not tickets
```

```
Guardrails baked in → safe by default, no SRE in the loop for routine work
```

A lot of SRE toil is acting as a manual gateway: provisioning environments, granting access, running deploys on behalf of developers. Platform engineering eliminates this class wholesale by building self-service "paved roads" — internal developer platforms where teams accomplish common tasks safely without filing a ticket. The SRE/platform team's leverage shifts from doing the work to building the system that lets everyone do it safely. Guardrails are encoded so the easy path is the safe path.

From ticket-takers to platform-builders: The highest-leverage move against org-wide toil is making the common case self-service with safety built in. One paved road removes thousands of future tickets.

📚 Official docs

- Google SRE Book: Eliminating Toil
- CNCF: Platforms white paper

certs.eknathalabs.com

Q99 What is GitOps and how does it reduce operational toil and risk?

Toil

🎙 Spoken answer · ~78 s

GitOps declares desired state in Git, and a controller such as Argo CD or Flux continuously reconciles the cluster toward what's in the repository. Deploying means merging a pull request, rolling back means reverting a commit, and the audit trail is simply the Git log. Drift detection is built in: the controller notices manual changes in the cluster and either flags them or reverts them to the declared state. It reduces toil because deployment steps are automated and consistent, with no hand-run kubectl or bespoke scripts. It reduces risk because there's no manual kubectl against production, so fewer fat-finger incidents; every change is reviewed in a pull request, so there's a second pair of eyes; the cluster credentials stay inside the cluster instead of living in CI; and rebuilding an environment is just pointing the controller at the repo. The costs: you need a pattern for secrets, such as Sealed Secrets or External Secrets; repository structure needs design for multiple environments; and there's a delay between merging and seeing the result, so feedback to developers must be built in. It's a strong fit for reducing operational toil.

Likely follow-ups

- How do you handle secrets in GitOps?
- What does self-heal do in Argo CD?

```
Desired state declared in Git; a controller (ArgoCD/Flux) reconciles cluster → Git
```

```
Deploy = merge a PR; rollback = revert a commit; audit = git log
```

```
Drift detection: controller flags/repairs manual changes back to declared state
```

```
No manual kubectl to prod → fewer fat-finger incidents
```

GitOps makes Git the single source of truth for infrastructure/app state and a controller continuously reconciles reality to it. This cuts toil and risk together: deploys become reviewed PRs, rollbacks become reverts, the full audit trail is the git history, and manual drift is automatically detected and corrected. It removes the error-prone manual `kubectl`-to-prod pattern (a common incident source) and gives you a declarative, repeatable, reviewable change process.

Git as the control plane: Everything goes through review and is recorded, rollback is trivial, and the cluster self-heals toward the declared state. Reviewed-change-only is both less toil and far fewer fat-finger outages.

📚 Official docs

- Google SRE Book: Eliminating Toil
- OpenGitOps principles

certs.eknathalabs.com

Q100 How do you justify reliability/automation investment to management (ROI)?

Toil

🎙 Spoken answer · ~78 s

Management funds business cases, not vibes, so I translate reliability work into money and time. First, quantify the toil: hours per week times the loaded engineering cost gives the recurring spend that automation could recover. Second, quantify incident cost: downtime in lost revenue, engineering hours spent responding, customer support load, SLA credits, and the harder-to-measure churn and reputation damage. Third, calculate the ROI for the project: build cost against annual hours or incident cost saved, giving a payback period. A project that pays back within a few months is an easy yes. Fourth, frame reliability as revenue protection and as velocity: error budgets and safer deployments let teams ship faster with confidence, so reliability enables features. I use real data from our own incidents and error budget burn, with comparisons to the business value of each nine, and I'm honest about uncertainty with ranges rather than false precision. I present options with trade-offs, start with a small pilot to prove value, and report results afterwards, which builds credibility for the next request. Speak the audience's language: finance cares about cost and risk, product about customer trust and velocity.

Likely follow-ups

- How would you estimate the cost of an hour of downtime?
- How do you present uncertainty in an ROI estimate?

```
Quantify the toil: hours/week × loaded cost = recurring spend being wasted
```

```
Quantify incident cost: downtime $ + eng-hours + churn/reputation
```

```
Automation ROI: build cost vs annual hours saved → payback period
```

```
Frame reliability as revenue protection + velocity (budget enables shipping)
```

Management funds business cases, not vibes. Translate reliability work into money and time: the recurring cost of the toil being eliminated (hours × cost), the cost of the incidents being prevented (downtime revenue, engineering time, customer churn), and the payback period of the automation. Reframe reliability not as a cost center but as revenue protection and a velocity enabler — a healthy error budget is what *lets* the business ship features safely.

Speak in payback and risk: "This $X of engineering saves Y hours/week (payback in N months) and reduces our exposure to $Z outages" gets funded. A graph of nines does not.

📚 Official docs

- AWS Well-Architected: Cost optimization (trade-offs with reliability)

certs.eknathalabs.com

Q101 What are the principles of chaos engineering (steady state, blast radius, abort)?

Chaos/Game Day

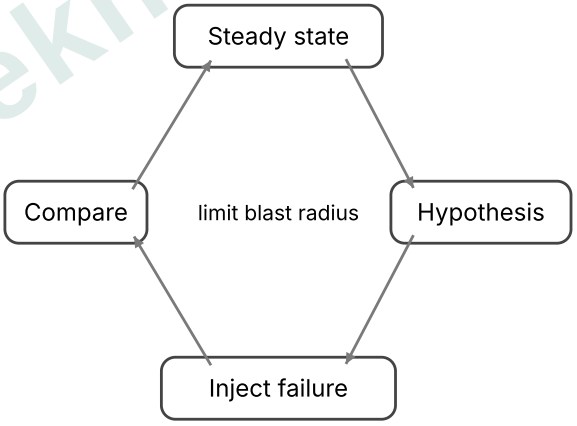
🎙 Spoken answer · ~73 s

Chaos engineering is disciplined experimentation, not random breakage, and it follows a few principles. First, define a steady state as a measurable signal, like an SLI, that shows the system is healthy. Second, form a hypothesis that steady state will hold under a specific fault, for example killing one availability zone won't breach the SLO. Third, minimise blast radius: run the smallest experiment that can test the hypothesis, such as one instance or a small fraction of traffic, and expand only as confidence grows. Fourth, always have an abort or kill switch ready before starting, with clear stop conditions, so a surprise doesn't become an outage. Fifth, eventually run in production, because that's where the real failure modes, traffic patterns, and dependencies live; staging gives only partial confidence. Supporting principles: good observability so I can see the effect, advance notice and agreement from stakeholders, automating experiments to run continuously, and fixing what you find. The goal is to find weaknesses before real outages do, and build confidence in the system's resilience, not to cause chaos.

Likely follow-ups

- Why run chaos experiments in production eventually?
- What stop conditions would you define for an experiment?

🗺 Chaos experiment




```
1. Define steady state as a measurable SLI (the system's "healthy" signal)
```

```
2. Hypothesise steady state HOLDS under a specific fault
```

```
3. Minimise blast radius — smallest experiment that tests the hypothesis
```

```
4. Always have an abort / kill switch ready before you start
```

```
5. Run in production (eventually) — that's where the real failure modes are
```

certs.eknathalabs.com

Chaos engineering is disciplined experimentation, not random breakage. You start from a measurable definition of "healthy" (steady state), form a falsifiable hypothesis ("losing one AZ won't breach the SLO"), inject the smallest fault that tests it with a tight blast radius, and keep an instant abort ready. The aim is to convert unknown failure modes into known, tested ones — and the highest-value experiments run in production, because that's the only place the real failure modes live.

Hypothesis + abort make it safe: Without a steady-state metric and a kill switch, you're not doing chaos engineering — you're just causing an outage. The rigor is what makes injecting failure in prod responsible.

📚 **Official docs**

- AWS: Disaster recovery & availability concepts (Fault isolation boundaries)
- AWS Builders' Library: Workload isolation using shuffle sharding
- Principles of Chaos Engineering

Q102 How do you introduce chaos engineering to a skeptical organization safely?

Chaos/Game Day

🎙 **Spoken answer · ~78 s**

You earn the right to run chaos in production incrementally. I'd start in staging or pre-production to build the tooling, the process, and trust, and to find the cheap, obvious weaknesses. Then the first production experiments are deliberately tiny: a very small blast radius, off-peak, announced in advance, with the on-call team ready and an abort switch. I begin with failures I'm fairly confident the system survives, which proves the safety net works and builds confidence. Showing value early is critical for sceptics: each experiment should find and fix a real gap, and I'd share the results, such as a missing timeout or a fallback that didn't work, as concrete wins that avoided a likely outage. I involve stakeholders from the start, explain the safety measures, align with the error budget so experiments run only when there's budget to absorb surprises, and avoid freezes and peak periods. Game days are a gentle entry point because they're planned and human-led. Over time, increase scope and automate. The message is that this is controlled learning that reduces risk, and I'd measure progress by issues found and incidents avoided.

Likely follow-ups

- Why start with failures you expect to survive?
- How do you get stakeholder buy-in for chaos in production?

```
Start in staging/pre-prod to build the tooling, process, and trust
```

```
First prod experiments: tiny blast radius, off-peak, announced, on-call ready
```

```
Begin with failures you're fairly confident you survive (prove the safety net)
```

```
Show value early: each experiment finds + fixes a real gap → build credibility
```

certs.eknathalabs.com

You earn the right to run chaos in production incrementally. Start in staging to prove the tooling and the abort mechanism work, then run the first production experiments small, off-peak, announced, with on-call standing by — and begin with faults you're reasonably confident the system already survives, so the early runs validate the safety net rather than cause outages. Each experiment that finds and fixes a real weakness builds the organizational trust to go bigger.

Trust is earned per experiment: A few small, well-run experiments that quietly prevent future incidents convert skeptics faster than any slide deck. Grow blast radius only as confidence grows.

📚 **Official docs**

- Principles of Chaos Engineering

Q103 What failure modes are worth injecting (latency, errors, resource, dependency, zone loss)?

Chaos/Game Day

🎙 **Spoken answer · ~75 s**

I choose fault types that map to realistic failure modes and the resilience mechanisms I've built. Latency injection: add delay to a dependency, to check that timeouts and circuit breakers work and callers don't pile up. Error injection: make a dependency return 5xx, to confirm fallbacks and graceful degradation kick in. Resource pressure: CPU, memory, or disk exhaustion, to see whether autoscaling and load shedding respond and whether limits behave as expected. Dependency down: kill a downstream service or database, to check the blast radius is contained. Zone or region loss: take down an availability zone, to verify failover holds the SLO and capacity is enough. Network faults: partitions, packet loss, and DNS failures, which often expose hidden assumptions. Also process kills and node loss, clock skew, expired certificates, and a retry storm or traffic surge. I prioritise by likelihood and impact, based on past incidents, and each experiment ties to a hypothesis about a specific mechanism: this timeout is set correctly, this fallback works. Tools like Chaos Mesh, Litmus, Gremlin, and AWS Fault Injection Service automate these faults.

Likely follow-ups

- Which fault would you inject first in a new microservice?
- Why are network faults especially revealing?

```
Latency: inject delay into a dependency → do timeouts/circuit breakers work?
```

```
Errors: make a dependency return 5xx → does fallback/degradation kick in?
```

```
Resource: CPU/memory/disk pressure → does autoscaling/shedding respond?
```

```
Dependency down: kill a downstream → is the blast radius contained?
```

```
Zone/region loss: kill an AZ → does failover hold the SLO?
```

certs.eknathalabs.com

```
Network: partition, packet loss, DNS failure → split-brain? stale cache?
```

Pick faults that exercise your resilience assumptions. Latency injection is especially valuable because *slow* is often worse than *down* — it ties up resources and reveals missing timeouts. Error injection tests fallbacks and circuit breakers; resource pressure tests autoscaling and shedding; dependency and zone failures test isolation and failover. Network faults (partitions, DNS) surface the subtle split-brain and stale-data bugs that are nearly impossible to find any other way.

Latency is the sneaky one: Systems usually handle a clean "down" but choke on "slow" — a laggy dependency with no timeout cascades into resource exhaustion. Injecting latency finds missing timeouts before prod does.

📚 **Official docs**

- AWS Builders' Library: Timeouts, retries and backoff with jitter

Q104 How do you plan and run an effective game day?

Chaos/Game Day

🎙 **Spoken answer · ~73 s**

A game day is a rehearsed, realistic failure drill that tests people, process, and tooling together. Before: pick a realistic scenario, such as losing the primary database or a region, define what success looks like and the objectives, brief the participants, and set safety bounds and a way to stop. During: inject the failure and let the team respond using the real runbooks, tools, and communication channels, and resist the urge to help, because the point is to find out where they struggle. Observe: assign people to time the response, and note where runbooks were wrong or missing, access was lacking, dashboards didn't show what's needed, communication broke down, or roles were unclear. Scope it so it's challenging but not overwhelming, and consider injecting surprises. After: hold a blameless review, record findings, assign action items to fix the gaps in runbooks, tooling, and training, and schedule the next one. Start in staging and progress to controlled production exercises as confidence grows. Regular game days also train new on-call engineers and make real incidents feel familiar.

Likely follow-ups

- What do you measure during a game day?
- How do you pick a good game day scenario?

```
Before: pick a realistic scenario, define success, brief participants, set safety bounds
```

```
During: inject the failure, let the team respond via real runbooks/tools (don't help)
```

```
Observe: time the response, note where runbooks/tooling/comms broke down
```

```
After: blameless review → fix the gaps → schedule the next one
```

cons.eknathalabs.com

A game day is a rehearsed, realistic failure drill that tests people, process, and tooling together — not just the system. Plan a concrete scenario ("primary region is gone — execute failover"), define what success looks like, and set safety bounds. During the drill, let the team respond with the actual runbooks and tools so you discover the stale step, the expired credential, or the broken failover before a real 3am incident does. The debrief turns those discoveries into fixes.

Game days build muscle memory: The first time someone executes the regional failover should not be during a real outage. Drills make the rare procedure familiar and expose the gaps while the stakes are low.

📚 **Official docs**

- Principles of Chaos Engineering

Q105 What's the relationship between chaos engineering and SLOs/error budgets?

Chaos/Game Day

🎙 **Spoken answer · ~75 s**

SLOs give chaos engineering its definition of success. The steady state is the SLI, and an experiment passes when the SLO is not breached during the fault injection; if it is, I've found a weakness. I run experiments only when the error budget is healthy, because I can afford a surprise, and avoid risky chaos during a freeze or when the budget is depleted. A chaos-induced budget burn is not a failure of the practice; it's a found weakness, and it's far cheaper to discover it in a controlled experiment than in a real outage at peak. I can also budget deliberately for experiments, treating the cost as an investment in learning. Conversely, chaos findings feed SLO work: they show whether the SLO target is realistic given the system's actual resilience, and whether dependencies' failures consume more budget than assumed. Observability tied to SLIs lets me see the impact in real time and abort if the burn rate crosses a threshold, which is a natural abort condition. This connection makes chaos engineering rigorous and ties it to business-level reliability goals.

Likely follow-ups

- Why avoid chaos experiments when the budget is exhausted?
- How can burn rate act as an abort condition?

```
Steady state = your SLI; "experiment passes" = SLO not breached during injection
```

```
Run experiments only when error budget is healthy (you can afford a surprise)
```

```
A chaos-induced budget burn is a found weakness — cheaper now than in a real outage
```

```
Don't run risky chaos during a freeze / depleted budget
```

certs.eknathalabs.com

SLOs give chaos engineering its definition of success: the experiment validates that injecting a fault does not breach the SLO (steady state holds). The error budget governs *when* to run — you experiment while the budget is healthy enough to absorb a surprise, and you pause risky experiments when the budget is depleted or you're in a freeze. A weakness found by chaos burns a little budget on purpose, which is vastly cheaper than discovering it via a real, unplanned outage.

Spend a little budget to save a lot: A controlled experiment that burns 1% of budget to expose a flaw prevents the uncontrolled incident that would have burned the whole month's. Chaos is budget well spent — when you have it.

📚 **Official docs**

- Google SRE Book: Service Level Objectives
- Google SRE Book: Embracing Risk (error budgets)
- Principles of Chaos Engineering

Q106 How do you test disaster recovery and validate your RTO/RPO are real? Chaos/Game Day

🎙 **Spoken answer · ~79 s**

RTO is the recovery time objective, the target time to restore service, and RPO is the recovery point objective, the maximum acceptable data loss. They're just numbers on a slide until you've actually executed recovery and measured. The way to test is to do it: restore from backup into a clean environment, fail over a database or the region, rebuild from infrastructure as code, and measure the actual recovery time and data loss, then compare against the targets. Untested DR is a hope, not a plan, and most plans fail their first real test, for reasons like missing credentials, outdated runbooks, backups that are corrupt or incomplete, undocumented dependencies, or capacity not available in the DR region. I run a progression: tabletop exercises, then partial restores, then full failover drills, in staging and eventually in production for critical services, on a schedule such as quarterly. I measure everything, including detection and decision time, and the time to validate data. Findings feed fixes and updated runbooks. I also verify backups continuously with automated restore tests, and check that dependencies like DNS, secrets, and identity also work in the recovery scenario.

Likely follow-ups

- Why do DR plans usually fail their first test?
- How would you automate backup restore verification?

```
RTO = target time to restore service; RPO = max acceptable data loss
```

```
Test by actually executing recovery: restore from backup, fail over the region
```

```
Measure ACTUAL recovery time + data loss → compare to the targets
```

```
Untested DR is a hope, not a plan — most DR plans fail their first real test
```

certs.eknathalabs.com

RTO and RPO are just numbers on a slide until you've actually executed the recovery and measured the real results. Regularly test DR by truly restoring from backups and performing region failovers, timing the actual recovery and quantifying the actual data loss, then compare against your stated objectives — they're often far apart. The classic discoveries: backups that never restored, a failover runbook with a broken step, or replication lag that means real RPO is hours, not minutes.

An untested backup is not a backup: The first real test of most DR plans is a real disaster, and that's when people learn the restore doesn't work. Rehearse recovery on a schedule and measure it against RTO/RPO.

📚 **Official docs**

- AWS Disaster Recovery of workloads (whitepaper)
- Google SRE Book: Data Integrity

Q107 How do you load test properly, and what's the difference between load, stress, and soak tests?

Capacity

🎙 **Spoken answer · ~76 s**

Different load tests answer different questions. A load test applies expected peak traffic, to verify that the system meets its latency and error SLOs at that load. A stress test ramps past capacity to find the breaking point and, importantly, how it fails: gracefully with shedding and errors, or by collapsing. A soak test runs sustained load for hours or days to find memory leaks, resource exhaustion, fragmentation, connection leaks, and slow degradation. A spike test applies a sudden surge, to see whether autoscaling and load shedding cope with the ramp. To load test properly, use realistic traffic mixes and data, not a single endpoint; test an environment as close to production as possible; include dependencies or realistic stand-ins; generate load from outside the system, with enough generators that they are not the bottleneck; watch the golden signals and saturation to find the bottleneck; avoid coordinated omission, which hides latency under overload; and compare results to baselines in CI to catch regressions. Define pass criteria from the SLOs before running. Tools include k6, Locust, Gatling, and JMeter. Findings feed capacity planning and tuning.

Likely follow-ups

- What is coordinated omission in load testing?
- What does a soak test find that a short load test misses?

```
Load test: expected peak traffic → does it meet latency/error SLOs at that load?
```

```
Stress test: ramp PAST capacity → find the breaking point + how it fails
```

```
Soak test: sustained load for hours/days → find leaks, fragmentation, slow degradation
```

```
Spike test: sudden surge → does autoscaling/shedding cope with the ramp?
```

cors.eknathalabs.com

Different tests answer different questions. A load test validates you meet SLOs at expected peak. A stress test pushes past capacity to find where and *how* the system breaks (graceful degradation vs cliff). A soak test runs for a long time to catch problems that only appear with duration — memory leaks, connection-pool exhaustion, log-disk fill, fragmentation. A spike test checks whether reactive scaling can handle a sudden surge given its lag. Test realistically: real traffic patterns, real data sizes, production-like environment.

Soak tests catch the silent killers: A memory leak or a slowly-filling disk passes every short load test and then takes down production at 2am on day three. Only sustained tests reveal time-dependent failures.

📚 **Official docs**

- Google SRE Book: Testing for Reliability

Q108 What is the difference between vertical and horizontal scaling, and their limits?

Capacity

🎙 **Spoken answer · ~78 s**

Vertical scaling means a bigger machine: more CPU and memory. It's the easy first lever, simple with no architectural change, but it hits a hard ceiling, is often disruptive to apply, and leaves a single point of failure. Horizontal scaling means adding more machines. It's nearly limitless and improves availability, but it requires the application to be stateless or to handle state carefully, needs load balancing, and brings distributed-system complexity: consistency, coordination, and partial failures. Stateful systems, especially databases, are hard to scale horizontally, which is where sharding or partitioning, read replicas, and caching come in, each with trade-offs: cross-shard queries, rebalancing, hot shards. In practice I scale up until it becomes expensive or risky, while designing services to be horizontally scalable from the start, and use managed services that scale for me where it makes sense. I also consider the limits of each approach: vertical scaling is bounded by the largest instance and downtime; horizontal scaling is bounded by the coordination overhead and the bottlenecks that don't scale, like a single database, a shared lock, or a central queue. Finding the real bottleneck comes first.

Likely follow-ups

- Why are databases harder to scale horizontally than web servers?
- What bottlenecks don't improve by adding more instances?

```
Vertical (scale up): bigger machine — simple, but a hard ceiling + single point of failure
```

```
Horizontal (scale out): more machines — near-limitless, but needs statelessness,
```

```
  load balancing, and distributed-system complexity
```

```
Stateful (databases) is hard to scale horizontally → sharding/partitioning
```

ceris.eknathalabs.com

Vertical scaling is the easy first lever — just use a larger instance — but it hits a physical ceiling and leaves you with one big single point of failure. Horizontal scaling adds more instances for effectively unlimited growth and better fault tolerance, but it requires the workload to be stateless (or to externalize state) plus load balancing and the inherent complexity of distributed systems. The genuinely hard case is stateful data stores, which resist horizontal scaling and push you toward sharding/partitioning.

Statelessness is the enabler: Horizontal scaling is trivial for stateless services and brutal for stateful ones. Pushing state into managed datastores/caches is what lets the compute tier scale out freely.

📚 **Official docs**

- Google SRE Book: Software Engineering in SRE (scaling & capacity)

Q109 How do you plan capacity for a known traffic spike (product launch, sale, Super Bowl)?

Capacity

🎙 **Spoken answer · ~73 s**

For a predictable spike like a launch or a big sale, I plan deliberately rather than counting on reactive autoscaling. First, forecast the peak, not the average, using comparable past events and the expected marketing reach, with a safety margin. Second, pre-provision and pre-warm: scale up before the event, because autoscaling reacts with lag and sharp spikes outrun it, and warm caches, connection pools, and load balancers. Third, load test to the forecast peak plus margin, to find the bottleneck before the event, which is often the database, a third-party API, or a quota. Check quotas and limits with cloud providers well in advance. Fourth, prepare mitigations: load shedding, a queue or virtual waiting room, rate limits, and feature flags to turn off non-essential features under pressure. Fifth, operational readiness: a change freeze around the event, a war room with clear roles and an escalation path, dashboards for the key metrics, and a game day rehearsal beforehand. Afterwards, scale down gradually and run a retrospective comparing the forecast with reality, to improve the next one.

Likely follow-ups

- Why isn't autoscaling alone enough for a sharp spike?
- What would you check about third-party dependencies before the event?

```
Forecast the peak (not the average) from comparable past events + marketing reach
```

```
Pre-provision / pre-warm — don't rely on reactive autoscaling for a sharp spike
```

```
Load test to the forecast peak + margin; find the bottleneck before the event
```

```
Prep mitigations: load shedding, queueing, waiting room, feature flags to shed load
```

```
War room + game day rehearsal for the event
```

certs.eknathalabs.com

For a predictable spike, reactive autoscaling alone is a trap — its provisioning lag can't match a sharp surge, so you pre-provision (or pre-warm) capacity ahead of the event sized to the forecast peak plus margin. Load test to that peak beforehand to find the real bottleneck (often the database or a downstream dependency, not the web tier). Have explicit overload defenses ready (shedding, a waiting room, flags to disable non-essential features) and rehearse the event like a game day.

Don't trust autoscaling for sharp spikes: Scaling lag means you're under-capacity exactly when the surge hits. Pre-provision for known events; let autoscaling handle the gentle organic curve.

📚 **Official docs**

- Google SRE Book: Reliable Product Launches at Scale

Q110 What is Little's Law and how is it useful for capacity reasoning?

Capacity

🎙 **Spoken answer · ~80 s**

Little's Law relates throughput, latency, and concurrency with a simple identity: L equals lambda times W. L is the average number of requests in the system, the concurrency; lambda is the arrival rate in requests per second; and W is the average time each request spends in the system, the latency in seconds. For example, a thousand requests per second with a latency of two hundred milliseconds means two hundred requests in flight at any moment. That's directly useful for capacity reasoning: it tells me how big thread pools, connection pools, and queues need to be, and how many instances I need if each can handle a certain concurrency. It also shows why latency increases hurt so much: if a dependency slows from two hundred milliseconds to two seconds at the same arrival rate, the concurrency needed rises tenfold, so pools exhaust and the service stalls. It holds for stable systems regardless of distribution. I use it to size pools, to explain why a slow dependency causes cascading failures, and to estimate the capacity needed for a target throughput, remembering that these are averages and that real systems need headroom for variability.

Likely follow-ups

- Why does a slow dependency exhaust your connection pool?
- How would you size a thread pool using Little's Law?

```
L = λ × W
```

```
L = avg number of requests in the system (concurrency)
```

```
λ = arrival rate (req/s)   W = avg time in system (latency, s)
```

```
e.g. 1000 req/s × 0.2s latency = 200 concurrent in flight
```

```
→ need a thread/connection pool + capacity sized for ~200 concurrent
```

certs.eknathalabs.com

Little's Law relates throughput, latency, and concurrency with a simple identity: the average number of requests in flight equals arrival rate times time-in-system. It's a powerful back-of-envelope tool — if you serve 1000 req/s at 200ms each, you have ~200 requests in flight, so your pools, threads, and capacity must accommodate that concurrency. It also shows why latency spikes are dangerous: if W doubles under load, concurrency doubles, which can exhaust pools and cascade.

Why latency growth cascades: Little's Law says rising latency (W) raises in-flight concurrency (L) for the same traffic — so a slowdown silently consumes thread/connection pools until they exhaust. Slow turns into down.

📚 **Official docs**

- Little's law (queueing theory reference)

Q111 What is utilization vs saturation, and why target ~60-70% utilization not 95%?

Capacity

🎙 **Spoken answer · ~76 s**

Utilisation is the percentage of time a resource is in use; saturation is the amount of queued or waiting work beyond what the resource is serving at the moment. It's tempting to run resources at ninety-five percent for efficiency, but queueing theory shows that latency rises non-linearly as utilisation approaches a hundred percent: at moderate utilisation, queues are short, but near saturation, small variations in arrival rate create long queues and latency climbs steeply, theoretically to infinity. Real traffic is bursty, so a system at high average utilisation spends much of its time overloaded. Headroom absorbs spikes, the extra load from failover when a node or zone is lost, and variance, and gives autoscaling time to react. So I typically target around sixty to seventy percent utilisation for latency-sensitive resources, with the exact figure depending on how bursty the load is and how fast I can scale. Batch or throughput-oriented workloads can run hotter, since they tolerate latency. Cost matters, so I balance efficiency with the risk, and I monitor saturation signals like queue depth, run queue length, and throttling, not just utilisation.

Likely follow-ups

- Why does latency rise non-linearly near full utilisation?
- Which workloads can safely run at higher utilisation?

```
Utilization: % of a resource currently in use
```

```
Saturation: queued/waiting work beyond what's being served right now
```

```
Queueing theory: as utilization → 100%, latency goes to infinity (non-linearly)
```

```
Headroom absorbs spikes, failover load, and variance → target ~60-70%
```

certs.eknathalabs.com

It's tempting to run resources at 95% for efficiency, but queueing theory shows latency rises non-linearly as utilization approaches 100% — the last few percent of utilization cost enormous latency. You also need headroom to absorb traffic spikes, the redundancy load when a peer fails (N+1), and normal variance. Targeting ~60-70% steady-state utilization keeps latency stable and leaves room to survive a failure or surge without tipping into saturation.

The last 10% is the most expensive: Pushing utilization from 70% to 95% saves a little money and buys a lot of latency and fragility. Reliability needs deliberate headroom, not maximal packing.

📚 **Official docs**

- Brendan Gregg: The USE Method (utilization & saturation)

Q112 How do you right-size resource requests/limits in Kubernetes without waste or OOM?

Capacity

🎙 **Spoken answer · ~75 s**

In Kubernetes, requests are the amount guaranteed and used for scheduling, and limits are the hard ceiling. The behaviour at the limit differs: going over a CPU limit means throttling, so the container slows down; going over a memory limit means the container is OOM killed. Setting them well takes measurement. I set requests from observed usage, around the 95th percentile for CPU, and close to peak for memory, using Prometheus data and the Vertical Pod Autoscaler in recommendation mode. For limits, I give memory a limit with headroom above the request, since memory can't be compressed, and consider omitting the CPU limit or setting it generously to avoid unnecessary throttling, relying on requests for fair sharing. Too-high requests waste money and cause pods to be unschedulable or nodes to be underused; too-low requests lead to contention, eviction, and noisy neighbours. I review regularly because usage changes with releases and traffic, and automate recommendations. For critical workloads, setting requests equal to limits gives Guaranteed QoS, which is evicted last. I validate changes under load and watch for throttling and OOM metrics.

Likely follow-ups

- Why might you omit a CPU limit but keep a memory limit?
- What QoS class do you get when requests equal limits?

```
Requests = guaranteed/scheduled amount; Limits = hard ceiling
```

```
CPU over limit → throttled (CFS); Memory over limit → OOMKilled
```

```
Set requests from observed P95-ish usage; limits with headroom (or no CPU limit)
```

```
Too-high requests → wasted/unschedulable; too-low → eviction/contention
```

cors.eknathalabs.com

Kubernetes requests drive scheduling and guarantees; limits cap usage. The behaviors differ critically: exceeding a CPU limit gets you throttled (latency spikes), while exceeding a memory limit gets the container OOMKilled. Right-size requests from real observed usage (around P95) so you neither waste capacity nor starve the pod, and set memory limits with genuine headroom since the penalty is a kill. Many teams avoid CPU limits (to prevent throttling) while keeping memory limits (to contain leaks) and rely on VPA/usage data to tune.

CPU throttles, memory kills: A too-tight CPU limit silently throttles and adds latency; a too-tight memory limit OOMKills the pod outright. Tune from real usage and treat memory headroom as non-negotiable.

📚 **Official docs**

- Kubernetes: Resource management for Pods & containers
- Microsoft: Cloud design pattern – Throttling

Q113 What is N+1 / N+2 redundancy and how does it interact with capacity planning?

Capacity

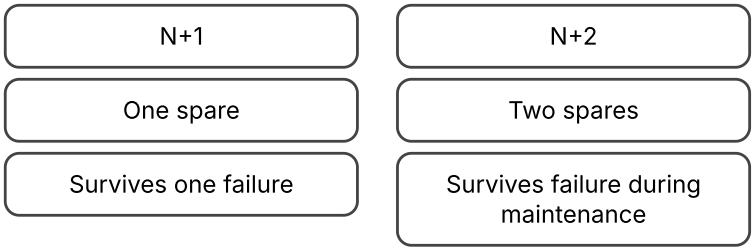
🎙 **Spoken answer · ~83 s**

N is the capacity needed to serve peak load. N plus one means one extra unit, so the system survives losing one node, instance, or zone at peak. N plus two survives losing two at once, for example one failed and one in maintenance. The interaction with capacity planning is that each remaining unit must absorb the failed one's share of the load, so I size for that. For example, with three availability zones and N plus one, I need each zone to run at roughly two thirds of its capacity at peak, so that if one zone is lost the other two can carry everything: that means total provisioned capacity of one and a half times peak, not just peak plus one node. With more zones the overhead is lower, which is one reason to use more of them. Higher redundancy costs more, so I choose by the SLO and the consequence of failure: critical services get N plus two, less critical N plus one. I also consider whether autoscaling can add capacity quickly enough to cover the gap, but I don't rely on it for sudden failures. And I test by actually removing capacity, using chaos experiments.

Likely follow-ups

- Why does a 3-zone N+1 design need more than one extra node?
- How do you validate redundancy actually holds?

🗺 **Redundancy**



cors.eknathalabs.com

```
N = capacity needed to serve peak load
```

```
N+1 = one extra unit so you survive losing one (node/AZ) at peak
```

```
N+2 = survive losing two simultaneously (e.g. one failed + one in maintenance)
```

```
Each remaining unit must absorb the failed one's load → size for that
```

Redundancy planning means provisioning beyond raw demand so a failure doesn't cause an outage. N+1 lets you lose one failure domain at peak load and still serve everyone — but only if the survivors can actually absorb the redirected load, which means each unit runs below 100% (tying back to the utilization headroom rule). N+2 covers the realistic case of a failure happening *during* planned maintenance. The mistake is sizing for N: any single failure at peak then becomes a user-facing outage.

Plan for failure during maintenance: N+1 is fragile if you take a node down for patching and then lose another. N+2 acknowledges that failures don't politely wait for maintenance windows to end.

📚 **Official docs**

- Google SRE Book: Software Engineering in SRE (capacity planning)
- Azure Well-Architected: Performance efficiency

certs.eknathalabs.com

Q114 How do you forecast capacity and detect when you'll run out (trend + lead time)?

Capacity

🎙 **Spoken answer · ~76 s**

Capacity exhaustion should be a forecast, not a surprise. I track usage trends for each constraining resource: CPU, memory, storage, database connections, IOPS, network, and quotas, over time. Then I project the trend to estimate the date I'll hit a limit, accounting for growth rate changes and seasonality, using simple linear or seasonal forecasting, for example Prometheus predict_linear. The critical part is lead time: I alert early enough to act, meaning the warning must come before exhaustion by the provisioning time plus a buffer. If getting more capacity takes two weeks because of procurement or a quota request, the alert must fire at least that far ahead. Storage is the classic example: disk will be full in twelve days is far more useful than disk is full now, and trend-based alerts replace fixed percentage thresholds. I treat the forecast as a ticket-level alert, reviewed in a regular capacity review, with larger events such as launches added manually. I also validate forecasts against actuals to improve accuracy, and consider the cost of overprovisioning. Automation, like auto-expanding volumes, can remove some of the toil.

Likely follow-ups

- Why is a trend-based disk alert better than an 80% threshold?
- What does lead time mean in capacity alerts?

```
Track resource usage trend (CPU, mem, storage, connections, IOPS) over time
```

```
Project the trend → estimate the date you hit a limit
```

```
Alert with LEAD TIME: warn before exhaustion by the provisioning time + buffer
```

```
Storage especially: "disk will be full in ~12 days" beats "disk is full now"
```

Capacity exhaustion should be a forecast, not a surprise. Track usage trends for each constrained resource, project them forward, and alert *predictively* — far enough ahead that you can provision before you hit the wall, accounting for lead time (cloud quota increases, hardware orders, and even autoscaling have non-zero latency). "Storage fills in ~12 days at current growth" is an actionable ticket; "disk full" at 3am is an incident. Predictive capacity alerts are a hallmark of mature operations.

Alert before the cliff, not at it: Lead time is everything — a predictive "you'll run out in N days" gives you a calm provisioning window, while a threshold-at-100% alert gives you an outage. Forecast the constrained resources.

📚 **Official docs**

- Google SRE Book: Software Engineering in SRE (scaling & capacity)

Q115 What is the relationship between SRE, DevOps, and Platform Engineering? Fundamentals

🎙 **Spoken answer · ~75 s**

These overlap but aren't the same. DevOps is the philosophy: break down the wall between development and operations, with shared ownership, automation, and fast feedback. It's a culture and set of practices rather than a role or a prescribed method. SRE is a prescriptive implementation of those ideas that uses specific mechanisms: SLOs and error budgets to manage reliability, toil caps, blameless postmortems, and engineering approaches to operations. Platform engineering builds the internal platform and paved roads that developers self-serve on, treating infrastructure and tooling as a product with developers as customers. All three aim to reduce friction between writing code and running it reliably. In practice they combine: DevOps sets the culture, platform engineering provides the self-service capabilities that remove toil and standardise good practice, and SRE brings the reliability discipline, measuring and defending SLOs, and handling operations at scale. An organisation might have a platform team that builds the paved road, SREs who define reliability standards and support critical services, and development teams who own their services' operations on top of those. The titles matter less than the outcomes.

Likely follow-ups

- How would these three roles collaborate in one organisation?
- What does treating the platform as a product mean?

```
DevOps: culture/philosophy — break the dev↔ops wall (CALMS, shared ownership)
```

certs.eknathalabs.com

```
SRE: a prescriptive implementation of DevOps using SLOs + error budgets + toil caps
```

```
Platform Engineering: build the internal platform/paved roads devs self-serve on
```

```
All three: reduce friction between writing code and running it reliably
```

These overlap but aren't the same. DevOps is the philosophy of shared dev/ops ownership and fast, safe delivery. SRE is one concrete, opinionated way to implement DevOps — with measurable reliability (SLOs), error budgets, and engineering discipline applied to operations. Platform Engineering is the more recent emphasis on building internal developer platforms (paved roads, self-service) so teams can ship reliably without deep ops expertise. In practice they're complementary lenses on the same goal: reliable software, delivered fast.

One framing: "DevOps is the what/why (culture), SRE is a how (prescriptive practices with numbers), and Platform Engineering builds the tooling that makes the good path the easy path." That nuance signals you've seen all three in practice.

📚 **Official docs**

- Google SRE Book: Introduction (what is SRE)
- CNCF: Platforms white paper

Q116 What does 'reliability is a feature' mean, and how do you make that case to product? Fundamentals

🎙 **Spoken answer · ~78 s**

Reliability is a feature means it must be planned, prioritised, and resourced like any feature, not treated as a background hope. The argument to product: an unreliable product loses users regardless of how good its features are, because trust is hard to regain; and reliability competes with features for the same engineering time, so it's a roadmap decision. The error budget makes this trade-off explicit and data-driven rather than political: when reliability is good, ship; when it's burning, fix. I frame it in business terms: churn after outages, lost revenue per hour of downtime, support costs, SLA penalties, and the value of the trust that makes customers renew. I use our own incident data and customer feedback. I also show that reliability work enables velocity: safer deployments, faster rollback, and better observability let teams ship with confidence, so they ship more. And I'm honest that perfection isn't the target: the right reliability is what users need, which protects against over-investing. Practically, I bring SLOs, budget status, and incident costs into planning discussions, and propose specific reliability items with expected outcomes, so product can weigh them next to features.

Likely follow-ups

- How do you quantify the business impact of an outage?
- Why does reliability work increase feature velocity?

certs.eknathalabs.com

```
Unreliable product = users leave regardless of how good the features are
```

```
Reliability competes with features for the same engineering time → it IS a roadmap item
```

```
The error budget makes the trade-off explicit and data-driven, not political
```

```
Frame in business terms: churn, trust, revenue per nine, support cost
```

"Reliability is a feature" means it must be planned, prioritized, and resourced like any other product capability — not treated as invisible plumbing that's only noticed when it breaks. The argument to product: a product users can't depend on loses them no matter how rich the feature set, so reliability directly drives retention and revenue. The error budget operationalizes this by making reliability a quantified, shared decision rather than a recurring argument — and crucially, it also tells you when you're *over*-investing and should ship faster.

It cuts both ways: Reliability-as-a-feature doesn't mean "always maximize uptime" — it means deliberately choosing the right level. 100% is the wrong target; the budget finds the level that keeps users happy while leaving room to ship.

📚 **Official docs**

- AWS Well-Architected: Cost optimization (trade-offs with reliability)

Q117 How do you think about cost vs reliability trade-offs as an SRE?

Fundamentals

🎙 **Spoken answer · ~80 s**

Reliability and cost are in direct tension: each additional nine roughly multiplies cost by ten, through more redundancy, more regions, more headroom, and more engineering effort. So the right target is the fewest nines that keep users happy, decided per service and per user journey, not a blanket rule. The levers are the level of redundancy, multi-AZ versus multi-region, headroom percentage, instance types, retention and replication, and the sophistication of operational processes. I tie reliability spending to the business value of the workload: a payment path deserves far more than an internal report, and different tiers get different SLOs and designs. I make the cost visible: what would moving from three nines to four cost, and what is the avoided downtime worth in revenue and trust? That gives leadership the information to decide. I also look for efficiency, since some reliability is cheap: better deployment practices, timeouts and retries, graceful degradation, and good monitoring improve reliability at low cost, while brute-force redundancy is expensive. And I review over time, because as a service matures its needs change. Overspending on unneeded reliability is also a failure, because that money could improve other things.

Likely follow-ups

- Which reliability improvements are cheap and which are expensive?
- How would you tier services by reliability requirement?

certs.eknathalabs.com

```
Each extra nine ≈ 10x cost (more redundancy, more regions, more headroom)
```

```
Right target = fewest nines that keep users happy (per service/journey)
```

```
Levers: redundancy level, multi-region, headroom %, instance types, retention
```

```
Tie reliability spend to business value of the workload (payment vs internal report)
```

Reliability and cost are in direct tension: every additional nine demands roughly an order of magnitude more investment in redundancy, headroom, and complexity. The SRE job is not to maximize reliability but to find the right level per workload — a payment path may justify four nines while an internal reporting tool is fine at two. Past a point, extra nines are invisible to users (their own device/network is less reliable than your service) yet cost you enormously, so over-reliability is as much a mistake as under-reliability.

Match the spend to the stakes: Don't run a back-office tool active-active multi-region. Reserve expensive reliability for the journeys where failure genuinely costs revenue, trust, or safety.

📚 **Official docs**

- AWS Well-Architected: Cost optimization (trade-offs with reliability)

Q118 What is 'defense in depth' for reliability, and how do the patterns layer together?

Fundamentals

🎙 **Spoken answer · ~78 s**

Defence in depth means no single safeguard is enough, so I layer independent defences, like the Swiss cheese model: each layer has holes, but the holes rarely line up. The patterns layer together roughly like this. Timeouts bound how long I wait. Retries with exponential backoff and jitter handle transient errors. Circuit breakers stop calling a failing dependency. Bulkheads isolate resources so one failure doesn't spread. Load shedding and rate limiting protect against overload. Graceful degradation keeps the core experience alive. Redundancy and failover cover component loss. Progressive rollout with canaries limits the blast radius of a bad change. Monitoring and SLO-based alerting detect problems, and automatic rollback restores service. Each layer catches what the previous one missed, so a failure that slips past one is caught by the next. The independence is important: layers that share a common failure mode, like the same configuration system, don't protect each other. I also test the layers, using chaos experiments, because a defence that has never been exercised often doesn't work. The cost is complexity, so I choose layers based on risk and the importance of the service.

Likely follow-ups

- Why must the layers be independent of each other?
- How do you verify that each layer works?

```
No single safeguard is enough — layer independent defenses:
```

certs.eknathalabs.com

```
  timeouts + retries(backoff+jitter) + circuit breakers + bulkheads
```

```
  + load shedding + graceful degradation + redundancy/failover
```

```
  + progressive rollout + monitoring + auto-rollback
```

```
Each layer catches what the previous one missed (Swiss cheese)
```

Resilience comes from layering independent patterns so a failure that slips past one is caught by the next — the Swiss-cheese model applied to reliability. A timeout bounds the wait; a circuit breaker stops hammering a dead dependency; a bulkhead contains the damage to one pool; graceful degradation keeps the core journey alive; redundancy/failover handles infrastructure loss; progressive rollout + auto-rollback limit bad-change blast radius. No single pattern is sufficient, but stacked, they make catastrophic failure require many simultaneous gaps.

Layers, not silver bullets: Senior answers don't name one pattern — they describe how timeouts, breakers, bulkheads, shedding, and degradation compose so a single fault can't line up holes through every layer at once.

📚 **Official docs**

- Kubernetes: Security overview (4C)

Q119 How do you embed reliability earlier in the lifecycle ('shift left' on reliability)?

Fundamentals

🎙 **Spoken answer · ~76 s**

Shifting reliability left means designing for it from the start instead of retrofitting it after incidents. In design: production readiness reviews before launch, SLOs defined before the service goes live, failure mode analysis asking what happens if each dependency fails, and capacity and dependency mapping, including which dependencies are critical. In build: reliability requirements in the definition of done, load tests and chaos or fault injection in CI where practical, resilience patterns provided by shared libraries, and testing of failure handling. In deploy: canary releases, automatic rollback, and SLO gates built into the pipeline, with feature flags to decouple release from deploy. In run: observability from day one, error budgets, on-call with runbooks, and blameless postmortems, whose lessons feed back into design standards and checklists. SRE engagement can be tiered: a lightweight review for most services, deeper involvement for critical ones. The benefit is that defects found earlier are far cheaper to fix than those found in production. The challenge is not to turn it into bureaucracy: automation, templates, and golden paths make the reliable approach the default and low-friction for developers.

Likely follow-ups

- What belongs in a production readiness review?
- How do you avoid reliability reviews becoming bureaucracy?

certs.eknathalabs.com

```
Design: production readiness reviews, SLOs defined before launch, failure-mode analysis
```

```
Build: reliability requirements + chaos/load tests in CI, dependency/criticality mapping
```

```
Deploy: canary + auto-rollback + SLO gates baked into the pipeline
```

```
Run: observability from day one, error budgets, blameless postmortems → feed back to design
```

Shifting reliability left means designing for it from the start instead of bolting it on after incidents. Concrete practices: production-readiness reviews and defined SLOs before a service launches, failure-mode analysis at design time, load/chaos tests in CI, and observability + canary + auto-rollback built into the delivery pipeline rather than added later. The postmortem feedback loop then carries operational learnings back into design, closing the cycle so each incident makes the next design better.

Production readiness reviews: A checklist before launch — SLIs/SLOs defined, alerts and runbooks in place, dependencies mapped, capacity and failure modes understood — prevents the "launched then immediately on fire" pattern far more cheaply than reacting after.

📚 **Official docs**

- Google SRE Book: Service Level Objectives

Q120 **As an SRE lead, how do you build a reliability culture across engineering, not just an SRE team?** Fundamentals

🎙 **Spoken answer · ~76 s**

Building a reliability culture means reliability becomes everyone's job, not just the SRE team's. First, shared ownership: development teams own their services' SLOs and are on call for them, with SRE enabling by providing tools, expertise, and standards rather than owning every service. Second, make reliability visible and rewarded: SLO dashboards that everyone can see, postmortem learnings shared widely, recognition for people who prevent incidents and improve operability, and reliability work counted in performance and promotion. Third, provide paved roads and platforms so the reliable path is the easy path: templates, libraries, and defaults for observability, deployment, and resilience. Fourth, leadership buy-in: an error budget policy signed off by product leadership, and a blameless culture modelled from the top, so people report problems and near-misses openly. Fifth, teach: game days, postmortem review sessions, embedded SRE rotations or office hours, training for on-call, and sharing patterns. I start with a few willing teams to show results, measure incident trends, and expand. Culture follows incentives: if only features are rewarded, reliability will lose. The test is whether teams make reliability trade-offs without being told.

Likely follow-ups

- How do incentives shape reliability culture?
- How would you start with one team and scale the practice?

certs.eknathalabs.com

```
Shared ownership: dev teams own their service's SLOs + on-call (SRE enables, not owns)
```

```
Make reliability visible + rewarded: SLO dashboards, postmortem learnings shared widely
```

```
Provide paved roads/platform so the reliable path is the easy path
```

```
Leadership buy-in: error budget policy signed by product; blameless culture from the top
```

```
Teach via game days, postmortem reviews, embedded SREs
```

A reliability culture can't live in a single SRE silo — if SRE is the only team that cares, every other team optimizes purely for features and SRE drowns. The lead's job is to spread ownership: dev teams own their SLOs and on-call, with SRE providing the platform, tooling, and expertise (paved roads make the reliable path the path of least resistance). It needs genuine leadership buy-in — product signing the error budget policy, blamelessness modeled from the top — and continuous teaching through shared postmortems, game days, and embedded SREs. Culture is the hard 80%; tooling is the easy 20%.

SRE enables; teams own: The end state isn't "SRE keeps everyone reliable" — it's every team owning its reliability with SRE providing leverage. That scales; a central team being everyone's safety net does not.

📚 **Official docs**

- Google SRE Book: Introduction (what is SRE)

Q121 **What are the SRE principles from Google's book, and which ones do you apply first in a new team?** Fundamentals

🎙 **Spoken answer · ~63 s**

The core ideas are that reliability is a feature measured by users, that perfect reliability is the wrong goal because it is expensive and slows change, so I manage risk with explicit targets and error budgets. Other principles are limiting toil so engineers spend time on engineering, monitoring symptoms rather than causes, automating repetitive work, keeping systems simple, treating release engineering as a first-class discipline, and learning from failure through blameless postmortems. In a new team I do not roll out everything. I begin with what creates shared language and decisions: define service level indicators and objectives for the most critical user journeys, agree an error budget policy with product, set up a clear on-call and incident process with severity levels, and start writing postmortems. Once that foundation exists, I measure toil and pick the biggest automation target, and then address capacity, release safety and resilience testing in order of risk.

Likely follow-ups

- Why is perfect reliability the wrong goal?
- What would you do in the first 30 days?

certs.eknathalabs.com

Embrace risk with explicit SLOs, eliminate toil, monitor what matters, automate, simplicity, release engineering, and blameless learning. Start with SLOs and an incident process since they create shared language and priorities.

```
# first 30 days
```

```
1. Define 2-3 SLIs per critical user journey
```

```
2. Agree SLOs and an error budget policy
```

```
3. Set up on-call, severity levels, postmortems
```

```
4. Measure toil, pick the top automation target
```

Interview tip: Do not try to adopt every principle at once. Pick the ones that remove the biggest current pain.

📚 **Official docs**

- Google SRE Book: Introduction (what is SRE)

Q122 How do you decide which services deserve SRE attention and which should stay with the development team?

Fundamentals

🎙 **Spoken answer · ~58 s**

SRE capacity is limited, so I allocate it where it reduces the most risk. I tier services by business criticality, number of users affected, revenue or safety impact, dependency fan-out, and how operationally painful they are, measured through incident counts, pages and toil. Critical tier services get dedicated SRE engagement, including shared on-call. Other services get lighter touch: consulting, reviews, shared tooling and templates, with development teams owning operations. Before taking on a service, it must pass a production readiness review covering monitoring, SLOs, capacity, rollback, runbooks and documented dependencies. If the service is poor, SRE can give the team a list of improvements and engage only once they are done. Engagements also have exit criteria: when a service is stable and its toil is low, SRE can reduce support and hand operations back, keeping the model sustainable.

Likely follow-ups

- What is a production readiness review?
- When do you hand a service back?

Prioritise by business criticality, user impact, change rate and operational burden. Use readiness reviews and engagement tiers: full SRE support for critical services, consulting and tooling for the rest, and clear criteria to hand back.

```
# service tiering
```

certs.eknathalabs.com

```
Tier 0: revenue/safety critical -> SRE on-call
```

```
Tier 1: important -> shared on-call + SRE consulting
```

```
Tier 2: internal/low -> dev team owns, self-service tooling
```

Interview tip: A production readiness review is the gate. Services that fail it stay with the dev team until they improve.

📚 **Start here (official docs)**

- Google SRE Book

Q123 What are common failure modes of an SRE team itself, and how do you avoid them?

Fundamentals

🎙 **Spoken answer · ~58 s**

A common failure is turning into a ticket queue: work arrives from others, nothing gets automated, and engineers burn out. A related one is lacking authority, so SRE raises reliability concerns but cannot slow risky launches because there is no agreed error budget policy with leadership backing. Hero culture is another, where a few people fix everything and knowledge stays in their heads, creating burnout and single points of failure. Teams can also drift apart from developers, so they inherit systems they did not design and cannot change. To avoid these I protect engineering time with a toil cap, track toil and pages as metrics, secure executive agreement on the error budget policy, rotate people through on-call and documentation duties, embed SREs with product teams for design reviews, and make reliability improvements visible work with goals, not invisible heroics.

Likely follow-ups

- How do you protect engineering time?
- What does hero culture cost?

Becoming a ticket-taking ops team, no authority to say no, hero culture, ignoring toil, or being separated from developers. Avoid with an engineering mandate, toil cap, error-budget policy backed by leadership and shared ownership.

```
# health checks for the team
```

```
toil % of time (target < 50%)
```

```
pages per shift (target <= 2 actionable)
```

```
% postmortem actions closed in 30 days
```

```
engineering project hours vs ticket hours
```

certs.eknathalabs.com

Interview tip: If SRE only reacts to tickets, it is just operations with a new name.

📚 **Official docs**

- Google SRE Book: Introduction (what is SRE)

Q124 What is the difference between availability, reliability, resilience and durability?

Fundamentals

🎙 **Spoken answer · ~62 s**

People use these words loosely, but they measure different things. Availability is whether the service responds successfully when users need it, typically expressed as a percentage of good requests or uptime. Reliability is broader: the service does what it should, correctly and consistently, over time, so it includes latency, correctness and freshness, not only being up. Resilience is the system's ability to absorb faults such as an instance failure or a dependency outage, degrade gracefully and recover quickly, which I validate through fault injection and failover tests. Durability applies to data: the probability that stored information is not lost or corrupted, for example through replication and backups; object stores quote very many nines for it. A service can be available but unreliable, returning wrong results, or available yet not durable if a failure loses writes. So I define separate indicators and targets for each property that users care about.

Likely follow-ups

- Can a system be available but not durable?
- How is availability related to MTTR?

Availability is the fraction of time the service works; reliability is consistent correct behaviour over time; resilience is the ability to withstand and recover from faults; durability is that stored data is not lost. They need different measures.

```
availability = good_events / valid_events
```

```
durability: 11 nines = 1 object lost per 10^11 per year
```

```
MTTR + MTBF -> availability = MTBF / (MTBF + MTTR)
```

```
resilience: test via fault injection and failovers
```

Interview tip: A system can be highly available yet lose data. Durability and correctness need their own SLIs.

📚 **Start here (official docs)**

- Google SRE Book

certs.eknathalabs.com

Q125 What is the difference between monitoring, observability and telemetry, and why does the distinction matter for SRE?

Fundamentals

🎙 Spoken answer · ~62 s

Telemetry is simply the data a system emits: metrics, logs, traces and events. Monitoring is the practice of watching specific signals I decided in advance matter, and alerting when they cross thresholds, which is perfect for known failure modes. Observability is a property of the system: whether I can understand its internal state from its outputs well enough to debug situations I did not anticipate. In a distributed system, failures often involve an unexpected combination, such as one customer on one version in one region, so I need rich, high-cardinality context attached to events and the ability to slice and compare dimensions quickly. For SRE this matters because dashboards and alerts for known causes are not enough during a novel incident. I invest in structured events, consistent trace context, service-level instrumentation and fast query tools, so that engineers can form and test hypotheses in minutes rather than guessing.

Likely follow-ups

- What does high cardinality enable?
- Can you have monitoring without observability?

Telemetry is the raw data (metrics, logs, traces). Monitoring checks known conditions against thresholds. Observability is the ability to ask new questions of a system and understand unknown failures from its outputs.

```
# known unknowns -> monitoring: alert when error_rate > 1%
```

```
# unknown unknowns -> observability: slice by version, region, customer, endpoint
```

```
histogram_quantile(0.99, sum by (le, version) (rate(http_duration_bucket[5m])))
```

Interview tip: High-cardinality, well-structured events let you debug failures you did not predict.

📚 Start here (official docs)

- Google SRE Book

certs.eknathalabs.com

Q126 How do you write an SLO document that stakeholders actually use?

SLI/SLO

🎙 Spoken answer · ~63 s

A good SLO document is short and unambiguous. It names the service and the user journey it protects, defines the indicator precisely, including what counts as a good event, what counts as valid, and where it is measured, for example at the load balancer or from the client. It states the target and the window, such as ninety-nine point nine percent over a rolling twenty-eight days, lists explicit exclusions like planned maintenance and why, and links to the dashboard and alerts that implement it. Crucially, it includes the error budget policy: what the team does when the budget is low or exhausted, agreed in advance with product and engineering leadership. The document has an owner and a review schedule, because targets should be revisited as the product and user expectations change. I get product sign-off, because the SLO is a business decision about acceptable risk, not only a technical number.

Likely follow-ups

- What belongs in the exclusions?
- Who must sign off the SLO?

State the user journey, SLI definition and measurement point, target and window, exclusions, error budget policy, owner and review cadence, in one page agreed with product. Link it to dashboards and alerts.

```
Service: checkout-api
```

```
SLI: 2xx/3xx responses under 500ms at the load balancer / all valid requests
```

```
SLO: 99.9% over rolling 28 days
```

```
Policy: budget < 25% -> freeze risky releases
```

```
Owner: payments-sre   Review: quarterly
```

Interview tip: If product has not signed the SLO, it is not an SLO. It is an engineering wish.

📚 Official docs

- Google SRE Book: Service Level Objectives

certs.eknathalabs.com

Q127 How do you handle SLOs for dependencies, and why does a chain of 99.9% services not give 99.9%?

SLI/SLO

🎙 Spoken answer · ~58 s

When a request must pass through several components in series, the success probability is the product of each component's availability. Three dependencies at ninety-nine point nine percent give about ninety-nine point seven percent overall, which is roughly a day of downtime in a year, noticeably worse than any single one. So when I set an SLO for my service, I look at my critical dependencies' real performance, not just their promised numbers, and calculate the ceiling. If the target is higher than the ceiling, I have options: reduce the number of hard dependencies in the request path, add caching or stale-serving fallbacks, make calls asynchronous through a queue, or degrade gracefully when a dependency fails. Redundant parallel paths raise availability because both must fail. I also agree on SLOs with dependency owners and monitor their contribution to my error budget.

Likely follow-ups

- How does redundancy change the maths?
- How do you reduce hard dependencies?

Serial dependencies multiply: three 99.9% services give about 99.7%. Compute the composite, add redundancy or fallbacks for critical paths, and set your SLO below what dependencies allow or mitigate their failures.

```
0.999 * 0.999 * 0.999 = 0.997  (~26 h/year)
```

```
parallel redundancy: 1 - (1-0.99)*(1-0.99) = 0.9999
```

```
# decide: tighten deps, cache, queue, or degrade gracefully
```

Interview tip: You cannot promise more availability than your hard dependencies unless you add fallbacks that tolerate their failure.

📚 Official docs

- Google SRE Book: Service Level Objectives
- Google SRE Book: Embracing Risk (capacity & availability math)

certs.eknathalabs.com

Q128 How do you implement SLOs with Prometheus recording rules and Sloth or OpenSLO?

SLI/SLO

🎙 Spoken answer · ~63 s

I start by defining the service level indicator as a ratio of good events to valid events, computed by a recording rule over a short window like five minutes, so downstream queries are cheap. Longer windows, such as thirty minutes, six hours and the full slo window, are derived from it. From those, I compute the error budget remaining and burn rates, then the multi-window multi-burn-rate alerts that page for fast burns and ticket for slow burns. Writing all this by hand per service is repetitive and error-prone, so I use a declarative spec, for example in the OpenSLO format or the Sloth and Pyrra tools, which generate the recording rules, alerts and dashboards consistently. The spec lives in Git alongside the service, is reviewed, and validated with rule linting in CI. I then verify alerts by replaying historical incidents to see if they would have fired at the right time.

Likely follow-ups

- Why record at a short window?
- Why generate rules from a spec?

Define SLI ratios as recording rules over multiple windows, compute error budget and burn rates from them, and generate alerts and dashboards from a declarative SLO spec with tools like Sloth or Pyrra to avoid hand-written errors.

```
record: sli:http_availability:ratio_rate5m  expr: sum(rate(http_requests_total{code!~"5.."}[5m])) / sum(rate(http_requests_total[5m]))
```

```
sloth generate -i slo.yml -o rules.yml
```

```
promtool check rules rules.yml
```

```
1 - avg_over_time(sli:http_availability:ratio_rate5m[28d])
```

Interview tip: Generate rules from a spec. Hand-written multi-window burn rate alerts are easy to get subtly wrong.

📚 Official docs

- Google SRE Book: Service Level Objectives
- Prometheus: Recording rules
- Sloth – SLO generator

certs.eknathalabs.com

Q129 What are common mistakes when setting SLOs?

SLI/SLO

🎙 Spoken answer · ~60 s

A frequent mistake is aiming for too many nines because it sounds impressive, when users would not notice the difference and the cost and slowdown are large. Another is choosing indicators that measure resource health, like CPU or queue length, rather than what users experience, such as successful and fast responses. Averages hide the slow tail, so percentile or threshold-based latency indicators are better. Teams also write SLOs without product agreement or without a consequence, so when the budget is spent nothing changes, and the SLO becomes decoration. Windows can be wrong: too short makes the number noisy, too long makes it slow to react. Finally, targets get set once and never reviewed, even as the product, traffic and user expectations evolve. I check each SLO against questions: does it reflect user pain, can the team act on it, and does leadership honour the policy.

Likely follow-ups

- Why avoid too many nines?
- What makes an SLO actionable?

Too many nines, measuring server health instead of user experience, a single average-based SLI, no business sign-off, SLOs no one acts on, windows too short or long, and never revisiting targets.

```
# anti-patterns
```

```
SLO 99.999% for a service whose users tolerate minutes of downtime
```

```
SLI = CPU < 80%  (not user-facing)
```

```
SLI = average latency (hides tail)
```

```
No policy for when the budget is spent
```

Interview tip: An SLO nobody uses to make decisions is decoration. Tie it to alerts, release gating and planning.

📚 Official docs

- Google SRE Book: Service Level Objectives
- Google SRE Workbook: Implementing SLOs

certs.eknathalabs.com

Q130 How do you set SLOs for asynchronous systems like queues, event streams and email delivery?

SLI/SLO

🎙 **Spoken answer · ~59 s**

Asynchronous systems have no request-response cycle to time, so the user experience is about whether work gets done, how soon and whether it is right. I use three kinds of indicator. Freshness or end-to-end latency: the time from event creation to completion, targeted at a percentile, such as ninety-five percent of messages processed within two minutes. Completeness or durability: the share of messages that are eventually processed without loss or unrecoverable failure, tracked by comparing produced and processed counts and dead-letter queue volume. Correctness for results, like valid emails or accurate aggregates. For monitoring, the age of the oldest unprocessed message is more meaningful than queue depth, since a large queue that drains quickly is fine, while a small one stuck is not. Alerts consider the trend of the backlog and consumer lag, so I catch a growing delay before users do.

Likely follow-ups

- Why age of oldest message over queue depth?
- How do you measure message loss?

Measure freshness (age of oldest message or end-to-end delay), completeness (messages processed without loss), and correctness, not request latency. Use percentile delay targets and monitor the backlog trend.

```
histogram_quantile(0.95, sum(rate(msg_e2e_delay_seconds_bucket[5m])) by (le))
```

```
kafka consumer lag: kafka_consumergroup_lag
```

```
processed / (produced) over 28d >= 0.9999
```

```
oldest_unacked_message_age_seconds < 120
```

Interview tip: Queue depth alone is misleading. Age of the oldest item tells users' real wait time.

📚 **Official docs**

- Google SRE Book: Service Level Objectives
- Google SRE Workbook: Implementing SLOs

certs.eknathalabs.com

Q131 How do you use error budgets to negotiate with product teams about launches and risk?

Error Budget

🎙 Spoken answer · ~58 s

Without a shared measure, reliability debates become opinion versus opinion. The error budget gives both sides a common number. At the start of a quarter we agree the SLO and the policy. During planning, I show how much budget remains and how fast it burned recently. If there is plenty of budget, product can launch more aggressively, run experiments, or do risky migrations, because reliability has headroom. If budget is nearly spent, I recommend prioritising stability work, smaller releases or delaying a risky launch, and explain the numbers. I try to offer alternatives, not a flat refusal: use a feature flag, a canary and a staged rollout, schedule the launch for lower traffic, or do the reliability work that buys more budget first. This keeps the relationship collaborative, and the decisions follow rules the business agreed in advance.

Likely follow-ups

- How do you avoid being the blocker?
- What if product disagrees with the policy?

Show the remaining budget as shared data. If healthy, product can take more risk and ship faster; if low, prioritise reliability work. The budget turns an argument into a pre-agreed rule, not a personality clash.

```
error_budget_remaining = 1 - (bad_events / (SLO_budget_fraction * total_events))
```

```
budget > 50%: ship freely, run experiments
```

```
budget 10-50%: extra care, smaller changes
```

```
budget < 10%: freeze risky changes, fix reliability
```

Interview tip: Present options and trade-offs, such as launching with a feature flag and staged rollout, rather than a flat no.

📚 Official docs

- Google SRE Book: Embracing Risk (error budgets)
- Google SRE Book: Reliable Product Launches at Scale

certs.eknathalabs.com

Q132 What counts against the error budget and what should be excluded?

Error Budget

🎙 Spoken answer · ~62 s

The budget should reflect user pain, so I count responses that failed or were too slow from the user's perspective. That includes server errors, timeouts, and requests slower than the latency threshold, whether caused by my own code or a dependency I choose to rely on. I exclude events that are not the service's fault or not real users: client errors like bad requests or unauthorised calls, health check traffic, synthetic load tests, and traffic from known abusive clients. Planned maintenance is contentious: excluding it can be justified only if users were notified and had alternatives, and if the SLO document states it. Otherwise it should count, because users still experienced downtime. Every exclusion makes the metric look better than reality, so I keep the list short, written down and reviewed, and I compare the SLI against external signals like support tickets to check that it reflects reality.

Likely follow-ups

- Should 4xx errors count?
- Why be careful with planned maintenance exclusions?

Count user-visible failures and slow responses from your service or its hard dependencies. Exclude client errors (4xx caused by users), synthetic probes, load tests, and agreed planned maintenance only if users were warned and the SLO document says so.

```
good: status < 500 AND latency < 400ms
```

```
bad: 5xx, timeouts, latency >= 400ms
```

```
excluded: 4xx caused by client, health checks, internal test traffic
```

```
sum(rate(http_requests_total{job="api",code=~"5.."}[28d]))
```

Interview tip: Be careful with exclusions. Every exclusion makes the number look better than users' experience.

📚 Official docs

- Google SRE Book: Embracing Risk (error budgets)

certs.eknathalabs.com

Q133 How do you handle a single huge incident that consumes the entire quarterly error budget on day one?

Error Budget

🎙 Spoken answer · ~58 s

A large incident can use up the whole budget, and then the policy says to slow down feature work and focus on reliability. I follow it, because the rule was agreed in advance, but I apply judgement about scope. The freeze concerns risky changes, while fixes that improve reliability continue. The team completes the postmortem, delivers the highest-impact preventive actions, and demonstrates improvement before returning to normal. If the window is a calendar quarter, the team would be frozen for months on day one, which is demotivating and unrealistic, so this is a signal to consider a rolling window, such as twenty-eight days, where the bad period gradually ages out and the budget recovers. I would also check whether the SLO is calibrated correctly and whether one outage this large indicates a single point of failure that needs architectural work.

Likely follow-ups

- Why does the window type matter?
- What is frozen and what continues?

Follow the policy: freeze risky changes, prioritise remediation, but avoid punishing the team for the whole window. Consider whether the window is too long, analyse root causes, and use shorter rolling windows and burn-rate alerts to regain balance.

```
budget consumed: 100% in 40 min outage
```

```
action: freeze non-critical releases
```

```
action: complete postmortem actions
```

```
review: rolling 28d window vs calendar quarter
```

Interview tip: Calendar windows reset all at once; rolling windows recover gradually as the bad period ages out.

📚 Official docs

- Google SRE Book: Embracing Risk (error budgets)

certs.eknathalabs.com

Q134 How do you use error budget burn data in capacity and roadmap planning?

Error Budget

🎙 Spoken answer · ~56 s

Over a quarter I tag each incident and burn event with its category, such as bad deploys, dependency failures, capacity saturation, configuration changes or unknown. Summing the consumed budget by category shows where reliability is actually lost. If thirty-eight percent of the burn comes from deployments, investing in progressive delivery, automated rollback and better pre-release testing pays off more than anything else. If a particular dependency causes a quarter of the burn, I discuss its SLO with the owning team or add fallbacks and caching. If capacity saturation is the driver, I prioritise load testing and autoscaling improvements. This evidence goes into roadmap planning as proposals with estimated budget savings, so leadership can compare reliability projects against features on a common basis. It also helps me retire work that does not move the numbers.

Likely follow-ups

- Why categorise burn by cause?
- How do you justify a reliability project?

Track which causes consume budget (deploys, dependencies, capacity, bugs), then allocate roadmap time proportionally to the top drivers. Reliability work is justified by budget evidence instead of intuition.

```
sum by (cause) (budget_burn_total)
```

```
deploy-related: 38%
```

```
dependency X: 25%
```

```
capacity saturation: 20%
```

```
unknown: 17%
```

Interview tip: Label incidents with a cause category so you can sum the budget burn per category each quarter.

📚 Official docs

- Google SRE Book: Embracing Risk (error budgets)

certs.eknathalabs.com

Q135 How would you set up fast and slow burn alerts, and what are typical window and threshold pairs?

Error Budget

🎙 **Spoken answer · ~65 s**

Burn rate is how fast I consume budget relative to the rate that would exactly exhaust it over the SLO window. A burn rate of one means I will end the window at zero. The recommended approach pairs a long window with a short one. For fast, severe problems, I page when the burn rate over one hour exceeds fourteen point four, which would consume two percent of a thirty-day budget in that hour, and the five-minute rate also exceeds it, confirming the problem is still ongoing so the alert resets quickly after recovery. A second page uses six times over six hours. For slow burns that need attention but not urgently, I create tickets at three times over one day and one times over three days. This design gives high precision, good recall, and fast detection for big problems, while catching sneaky slow leaks, without relying on fixed error-rate thresholds that either flap or miss.

Likely follow-ups

- Why a short confirming window?
- What does burn rate of 1 mean?

Use multi-window, multi-burn-rate alerts: page on a 14.4x burn over 1 hour (confirmed by 5 min), also 6x over 6 hours; ticket on 3x over 1 day and 1x over 3 days. The short window confirms it is still happening.

```
expr: (sli:burn_rate1h > 14.4 and sli:burn_rate5m > 14.4)
```

```
expr: (sli:burn_rate6h > 6 and sli:burn_rate30m > 6)
```

```
expr: (sli:burn_rate1d > 3 and sli:burn_rate2h > 3)
```

```
expr: (sli:burn_rate3d > 1 and sli:burn_rate6h > 1)
```

Interview tip: A 14.4x burn over one hour consumes 2% of a 30-day budget, which justifies waking someone.

📚 **Official docs**

- Google SRE Workbook: Implementing SLOs

certs.eknathalabs.com

Q136 How do you handle on-call compensation, fairness and burnout prevention?

On-Call

🎙 Spoken answer · ~61 s

On-call is a real burden, even when pages are rare, because it limits personal freedom. I treat it as work that deserves recognition through pay, time off in lieu or both, with clear policies. I keep frequency sustainable, typically no more than one week in four or five, with enough people in the rotation, and ensure people can rest after being woken at night, with a protected morning or day off. Fairness means tracking pages and out-of-hours interruptions per person and rebalancing, using follow-the-sun coverage where teams span time zones. Equally important is reducing the load: each page is a defect to fix, so I allocate time in the sprint for removing noisy alerts and repeating causes. I monitor wellbeing through surveys and one-to-ones, allow swaps and exemptions for personal circumstances, and ensure there is a backup and a clear escalation path so nobody feels alone.

Likely follow-ups

- How do you measure fairness?
- How do you give recovery time?

Compensate on-call time (pay or time off), limit frequency, ensure rest after night pages, distribute load fairly across time zones, track page load per person, and give engineers time to fix causes of pages.

```
pages per person per week
```

```
night pages (22:00-07:00)
```

```
on-call weeks per engineer per quarter
```

```
interrupted-sleep follow-up days off
```

Interview tip: If the same person is paged most often, the rotation is not fair and the system is not yet healthy.

📚 Official docs

- Google SRE Workbook: On-Call

certs.eknathalabs.com

Q137 What makes a good alert, and what is the test for deciding whether something should page a human?

On-Call

🎙 **Spoken answer · ~59 s**

I apply a simple filter before any alert pages a person. First, urgency: does it require action now, or can it wait until working hours. Second, impact: are users affected or about to be, ideally measured through symptoms like error rate and latency rather than internal causes. Third, actionability: is there something a human can usefully do, with a runbook describing diagnosis and mitigation. Fourth, novelty: if the same thing happens routinely and the fix is mechanical, automation should handle it, not a person. Alerts that fail these tests become tickets, dashboards or logs. The page itself includes the service, the impact, a link to the dashboard and the runbook, and recent changes, so the responder can begin immediately. I review alert quality in regular reviews, deleting those that never led to action and tuning those with a high false positive rate.

Likely follow-ups

- What turns a page into a ticket?
- What goes in the alert message?

A good page is urgent, actionable, user-impacting, novel and needs human judgement now. If it can wait until morning, is not actionable, or can be auto-remediated, make it a ticket, a dashboard item or automation.

```
test 1: does it need action in the next few minutes?
```

```
test 2: is there a known action to take (runbook link)?
```

```
test 3: is the user impact real or imminent?
```

```
test 4: could automation handle it?
```

Interview tip: Every page should come with a runbook link and a clear description of impact. Otherwise the responder starts from zero.

📚 **Start here (official docs)**

- Google SRE Book

certs.eknathalabs.com

Q138 How do you structure handoffs between on-call shifts and between teams?

On-Call

🎙 Spoken answer · ~56 s

Handoffs are where context gets lost. I use a standard template that the outgoing engineer fills in: incidents still open and their current status, mitigations that are in place and must be removed, alerts that have been silenced temporarily with the reason and expiry, planned deployments or maintenance during the next shift, flaky alerts and what has been tried, and anything the person suspects might get worse. The handoff takes place in a short live conversation, not only a document, so the incoming engineer can ask questions. A running on-call log or channel keeps a history of decisions. Between teams, such as follow-the-sun, the same applies, plus explicit confirmation of ownership of any incident in progress. I use a secondary on-call as backup for continuity, and review handoff notes in retrospectives so the template improves.

Likely follow-ups

- Why a live conversation, not just a document?
- What is dangerous to forget?

A written handoff covering open incidents, active mitigations, risky changes upcoming, noisy alerts, and follow-ups, delivered in a short sync. Use a shared log so context is not lost. Pair with a primary and secondary for continuity.

```
# handoff template
```

```
Open incidents / ongoing mitigations:
```

```
Temporary silences and why:
```

```
Changes planned this week:
```

```
Known flaky alerts and tickets:
```

```
Things I'd watch:
```

Interview tip: Temporary silences and manual workarounds are the most dangerous items to forget in a handoff.

📚 Official docs

- Google SRE Book: Being On-Call
- Google SRE Workbook: On-Call

certs.eknathalabs.com

Q139 How do you design runbooks and automated remediation so on-call engineers do less manual work?

On-Call

🎙 Spoken answer · ~63 s

I think of a ladder. First there is a written runbook for each actionable alert, explaining what it means, how to diagnose, and how to mitigate. When engineers repeat the same steps, I convert them to a script that can be run with one command, with a dry run mode. The next step is a button or chat command with approvals and audit logging, usable by anyone on call. For well-understood, low-risk cases, I go to automatic remediation, such as restarting a stuck worker or rotating logs when a disk fills. Automation needs guardrails: it must be idempotent, limited in rate with a maximum number of actions per time window, produce notifications, and stop and escalate to a human when it does not work. Otherwise a flawed automation can amplify an outage. Each automated action is reviewed after use, and the underlying cause is still tracked for a permanent fix.

Likely follow-ups

- Why limit automation rates?
- How do you decide what to automate first?

Start with a runbook per alert, then script the repeated steps, then wire safe, idempotent automation with guardrails, rate limits and audit logs. Promote steps from runbook to script to automatic action as confidence grows.

```
alert: DiskFull -> runbook: /runbooks/disk-full
```

```
script: ./clean-old-logs.sh --dry-run
```

```
automation: rundeck job with approval
```

```
auto-remediate: rotate logs when disk > 85% (max 3 per hour)
```

Interview tip: Add a circuit breaker to automation. A remediation that loops can make an incident much worse.

📚 Official docs

- Google SRE Workbook: On-Call
- Atlassian: Incident management handbook (lifecycle)

certs.eknathalabs.com

Q140 How should a team run an effective on-call retrospective or weekly alert review?

On-Call

🎙 Spoken answer · ~57 s

A weekly thirty-minute review keeps alert quality from decaying. The previous on-call engineer brings data: how many pages, which alerts fired most, which were actionable and which were noise, how long they took to resolve, and how many came out of hours. For each top item the team picks one of a few outcomes: delete the alert because it is useless, tune the threshold or add duration to reduce flapping, automate the response, or fix the underlying cause with a ticket and owner. I track the trend in pages per shift and in the percent of pages that led to meaningful action, with a goal such as at most two actionable pages per twelve-hour shift. The meeting also shares knowledge from tricky incidents and updates runbooks. Recognising and celebrating improvements reinforces the habit and keeps morale up.

Likely follow-ups

- What metrics do you bring?
- What are the possible outcomes for a noisy alert?

Review last week's pages: classify actionable vs noise, identify top sources, assign owners to fix or delete alerts, track trends in pages per shift, and celebrate reductions. Keep it short and recurring.

```
pages by alert name, last 7 days
```

```
% actionable vs non-actionable
```

```
time to acknowledge / resolve per page
```

```
action: delete | tune | automate | fix cause
```

Interview tip: Delete aggressively. Alerts that never lead to action cost sleep and attention.

📚 Official docs

- Google SRE Workbook: On-Call

certs.eknathalabs.com

Q141 How do you run an incident when the root cause is unknown and the clock is ticking?

Incident Mgmt

🎙 Spoken answer · ~57 s

When the cause is unclear, I focus on reducing user impact, not on winning the debugging puzzle. The first question is what changed recently, because deployments, config updates, feature flags and infrastructure changes explain most incidents, and reverting is a fast, low-risk mitigation. If nothing changed, I look at the golden signals and dependency health to localise the problem, and use general mitigations like failing over to another region, scaling up, shedding load or disabling a non-essential feature. I split the team: one person investigates logs and metrics, another checks dependencies, with the incident commander coordinating and keeping a timeline. I time-box each hypothesis, say ten minutes, and rotate if it leads nowhere. Communication to stakeholders goes out regularly. Once service is restored, the deeper root cause analysis moves to the postmortem, with the evidence preserved.

Likely follow-ups

- Why roll back before diagnosing?
- How do you time-box hypotheses?

Stabilise first: check recent changes, roll back, shift traffic, scale or shed load. Form hypotheses, split investigators, time-box each, and keep a timeline. Mitigation matters more than finding the cause during the incident.

```
git log --since='2 hours ago' --oneline  # what changed?
```

```
kubectl rollout undo deploy/api
```

```
kubectl top pods; kubectl get events --sort-by=.lastTimestamp
```

```
# traffic shift to healthy region
```

Interview tip: Most incidents correlate with a recent change. Roll back first, ask why later.

📚 Start here (official docs)

- Google SRE Book

certs.eknathalabs.com

Q142 What does an effective incident communication cadence look like for customers and executives?

Incident Mgmt

🎙 Spoken answer · ~59 s

During an incident, silence creates anxiety and duplicate questions. I assign a communications lead, separate from the technical responders, so engineers can focus. The first public message goes out quickly, acknowledging the problem, describing the user-visible impact in plain language, and saying when the next update will come. Then I update on that cadence, usually every thirty minutes for severe incidents, even if the update is only that we are still working on it. Messages state what we know, what we are doing, and avoid guessing about cause or promising a time to fix. Internal executives get a slightly more detailed update with business impact, the status of mitigation, and any decisions needed. When resolved, I send a clear closure message and later share a summary or postmortem. A status page, templates and a defined owner make this repeatable under stress.

Likely follow-ups

- Why a separate communications lead?
- What should you avoid saying mid-incident?

Acknowledge quickly, state impact and what you know, give regular updates even without news, name the next update time, avoid speculation, and close with resolution and follow-up. Separate technical channels from stakeholder updates.

```
T+10m: 'We are investigating elevated errors in checkout. Next update 30 min.'
```

```
T+40m: 'Cause identified, mitigation in progress.'
```

```
T+90m: 'Resolved. Monitoring. Postmortem to follow.'
```

Interview tip: Promise the time of the next update, then keep it, even if there is nothing new.

📚 Official docs

- Google SRE Workbook: Incident Response
- Atlassian: Incident management handbook (lifecycle)

certs.eknathalabs.com

Q143 How do you handle an incident that spans multiple teams or vendors, with unclear ownership?

Incident Mgmt

🎙 **Spoken answer · ~60 s**

With many parties, the main risk is confusion: duplicated work, conflicting changes and nobody in charge. I establish a single incident commander who coordinates and has the authority to make decisions, a single channel and call for everyone, and named leads for each affected area. A scribe keeps one timeline so decisions and observations are recorded. Tasks are assigned to specific people with a time to report back. For a vendor, I open the highest-priority support case immediately, escalate through our account contact, supply logs and a clear description of symptoms and our evidence, and ask for a named engineer, while preparing my own mitigation in case they are slow, such as failing over to another provider. If it is unclear who owns a component, the commander assigns someone to take it for now, and the ownership question is addressed in the postmortem.

Likely follow-ups

- How do you escalate with a vendor?
- Who decides when ownership is unclear?

Declare a single incident commander, open one shared channel and bridge, list involved parties and owners, assign tasks explicitly, escalate vendors through support contracts, and keep one timeline. Resolve ownership debates after the incident.

```
#inc-1042 channel, single bridge
```

```
roles: IC, Ops lead, Comms lead, Scribe
```

```
vendor: open P1 case, request named engineer, share evidence
```

```
timeline doc: who/what/when
```

Interview tip: The incident commander has authority to assign work. Arguing over ownership is for the postmortem.

📚 **Official docs**

- OpenTelemetry: Signals (traces, metrics, logs)
- Atlassian: Incident management handbook (lifecycle)

certs.eknathalabs.com

Q144 What is the difference between an incident commander and a technical lead, and why separate them?

Incident Mgmt

🎙 Spoken answer · ~59 s

The incident commander manages the response as a whole: understanding the current state, deciding priorities, assigning tasks, requesting additional help, ensuring communication happens, and deciding when to escalate or declare the incident over. The technical lead, or ops lead, works hands-on with the systems: investigating logs, testing hypotheses, performing mitigations. Combining the roles is a common failure, because when the same person goes deep into a technical problem, nobody tracks the broader picture, updates stop, and other responders wait for direction. In small incidents one person may hold several roles, but as severity grows I split them. The commander should be the person best at coordination and staying calm, not necessarily the most senior engineer or the one who knows the system best. Regular practice in game days builds this muscle, and handing off the commander role during long incidents prevents fatigue.

Likely follow-ups

- Who should be the commander?
- How do you hand off command?

The incident commander coordinates, decides, delegates and communicates; the technical lead dives into diagnosis and fixes. Separating roles prevents the person debugging from losing the big picture and keeps coordination continuous.

```
IC: owns process, priorities, decisions, delegation
```

```
Ops/Tech lead: hands-on diagnosis and mitigation
```

```
Comms lead: internal/external updates
```

```
Scribe: timeline and decisions
```

Interview tip: If you are the IC, resist the urge to debug. Your job is to keep everyone effective.

📚 Official docs

- Google SRE Book: Managing Incidents
- PagerDuty Incident Response documentation

certs.eknathalabs.com

Q145 How do you decide to roll back versus roll forward during a bad deploy?

Incident Mgmt

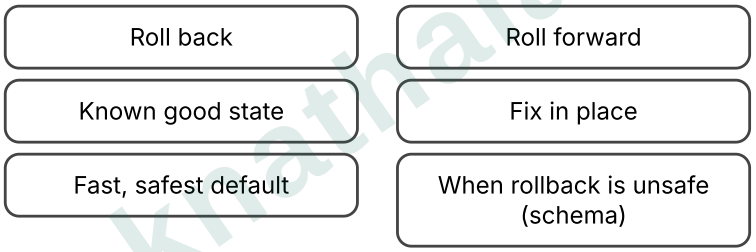
🎙 Spoken answer · ~64 s

My default is to roll back, because it restores a known good state quickly and is predictable. Before I do it, I check the rollback is safe: has a database migration already run that the old version cannot handle, has data been written in a new format, have other services started depending on the new behaviour. If so, rolling back might make things worse, and a forward fix may be safer. Roll forward is also reasonable when the cause is clear and the fix is a trivial, well-tested change that can pass through the pipeline faster than a rollback, though in a real incident I am cautious about untested rushed fixes. I compare estimated time to restore for each option and the risk of making things worse. To keep rollback always possible, I design changes to be backward compatible, using expand-and-contract migrations and feature flags, so the old version works with the new schema.

Likely follow-ups

- When is rollback unsafe?
- What is expand and contract?

🗺 Rollback vs roll forward



Roll back when the change is clearly implicated and rollback is safe and fast. Roll forward when rollback is risky (schema changes, data migrations) or the fix is tiny and well understood. Decide on time to restore and risk.

```
kubectl rollout undo deployment/api
```

```
argocd app rollback api 42
```

```
git revert <sha> && deploy
```

```
# data migration applied? check backward compatibility first
```

Interview tip: Design deploys to be backward compatible so rollback is always possible. Expand, then migrate, then contract.

📚 Official docs

- AWS Builders' Library: Ensuring rollback safety during deployments

certs.eknathalabs.com

Q146 How do you write postmortem action items that actually reduce recurrence?

Postmortem

🎙 **Spoken answer · ~62 s**

Weak action items like be more careful or send a reminder do not change a system. Good ones address how the failure happened and are concrete: a code change, an alert, a guardrail in the pipeline, a test, or a design change. I classify them as preventing recurrence, improving detection, and reducing impact when it happens again, and prioritise by the risk they reduce versus effort. Each item has a named owner, a due date and a ticket in the team's normal backlog, not a separate document that gets forgotten, and the postmortem links to them. I limit the number to what the team can actually complete, and mark the one or two critical items as must-do. Progress is reviewed in a regular meeting, and completion rates are reported. If an action keeps slipping, I escalate or explicitly decide not to do it and record the accepted risk.

Likely follow-ups

- What are the three types of actions?
- What do you do with slipping actions?

Make actions specific, owned, time-bound and prioritised by impact: prevent, detect and mitigate. Prefer systemic fixes and automation over reminders and training. Track them in the same backlog as feature work.

```
P0 prevent: add schema validation in CI (owner: Priya, due: 2 wks)
```

```
P1 detect: alert on queue age > 5 min (owner: Sam, due: 1 wk)
```

```
P2 mitigate: automate rollback on error spike (owner: Lee, due: 6 wks)
```

```
bad: 'be more careful' / 'retrain team'
```

Interview tip: If the action is 'tell people to be careful', the system is still unsafe. Fix the system, not people.

📚 **Official docs**

- Google SRE Book: Postmortem Culture
- Google SRE Workbook: Postmortem Culture
- Atlassian: How to run a blameless postmortem

certs.eknathalabs.com

Q147 How do you facilitate a blameless postmortem meeting when emotions are high or someone made an obvious error?

Postmortem

🎙 Spoken answer · ~61 s

I open the meeting by restating that the goal is learning and that we assume everyone acted with good intent given what they knew. I walk through the timeline from logs and chat records, rather than memories, so the discussion is factual. When someone made a mistake, I ask questions about context: what information did you have, what did the dashboards show, what made that choice reasonable, what was confusing. This shifts attention to missing guardrails, unclear tooling, poor documentation or pressure. I avoid asking why did you, which sounds accusatory, and I replace phrases like human error with the system gap that allowed it. If senior leaders attend, I brief them beforehand not to assign fault. The person involved is invited, not forced, to share their perspective. When people feel safe, they share the full picture, which is the only way to find real contributing factors.

Likely follow-ups

- Why avoid asking 'why did you'?
- How do you handle leaders who want accountability?

Set ground rules, focus on how the system allowed the error, use neutral language and timeline reconstruction, ask what made the action reasonable at the time, and keep leaders from assigning blame. Safety encourages honesty.

```
ask: 'What did you know at the time?'
```

```
ask: 'What made this decision seem reasonable?'
```

```
avoid: 'Why did you...?'
```

```
replace 'human error' with the missing guardrail
```

Interview tip: If people fear punishment, they hide details, and you lose the information needed to prevent recurrence.

📚 Official docs

- Google SRE Book: Postmortem Culture
- Google SRE Workbook: Postmortem Culture
- Atlassian: How to run a blameless postmortem

certs.eknathalabs.com

Q148 What is a good timeline in a postmortem and how do you build it from evidence?

Postmortem

🎙 Spoken answer · ~58 s

I build the timeline from objective sources: monitoring graphs, deployment and change logs, alert histories, and the incident channel transcript, all converted to a single time zone. I record key events in order: when the problem began, when it was detected and how, when people were paged, hypotheses tried, mitigations applied, and the recovery time. Importantly I add the responders' beliefs and decision points, such as at fourteen twenty we thought the database was the cause, because that explains why certain steps were taken and what information was missing. I mark uncertain items and gaps instead of guessing. The timeline allows me to compute durations like time to detect, time to engage and time to mitigate, which show where the process can improve. Reviewing it with the people involved corrects errors and often uncovers details that were not recorded anywhere.

Likely follow-ups

- What metrics come from a timeline?
- Why record what responders believed?

A timestamped sequence in one time zone covering detection, escalation, key decisions, mitigations and recovery, built from logs, metrics, chat and deploy records, with gaps and uncertainty marked. It shows both what happened and what people believed.

```
14:02 deploy 2026.10.1 rolled to 25%
```

```
14:09 error rate 2% -> 11%
```

```
14:12 page fired (detected after 10 min)
```

```
14:20 hypothesis: DB connections. Wrong.
```

```
14:41 rollback started; 14:48 recovery
```

Interview tip: Include decision points and what responders believed, not only system events. That reveals why actions made sense.

📚 Official docs

- Google SRE Book: Postmortem Culture
- Google SRE Workbook: Postmortem Culture

certs.eknathalabs.com

Q149 How do you handle postmortems for incidents caused by a third-party vendor or cloud provider outage?

Postmortem

🎙 Spoken answer · ~58 s

Even when the root cause lies with a provider, the incident revealed something about my system: it depended on that component in a way that turned the failure into user impact. So the postmortem examines my side. Did we detect the problem quickly ourselves or only learn from the vendor status page? Could we have degraded gracefully, served cached data, switched to a secondary region or provider, or queued work? Was our communication to customers clear and timely? What did the dependency's published SLA promise versus our own commitments? I request the provider's root cause analysis and follow up on contractual remedies such as service credits, but I treat those as secondary. Then I evaluate whether investment in redundancy, such as multi-region or multi-provider, is justified by the cost of outages and the likelihood, rather than reflexively adding complexity.

Likely follow-ups

- Should you always go multi-provider?
- What do you ask the vendor for?

Focus on what you control: detection, your dependency design, fallback options, communication and contract follow-up. Examine why a single provider failure had that impact and what resilience investments are justified.

```
question: why was this a single point of failure?
```

```
question: did our alerts detect it before the vendor status page?
```

```
question: could we have failed over or degraded gracefully?
```

```
action: request provider RCA, review SLA credits
```

Interview tip: Blaming the vendor ends learning. The question is how your system should behave when a dependency fails.

📚 Official docs

- Google SRE Book: Postmortem Culture
- Google SRE Workbook: Postmortem Culture

certs.eknathalabs.com

Q150 How do you share postmortem learnings across a large organisation so other teams benefit?

Postmortem

🎙 Spoken answer · ~57 s

Learning only helps if it travels. I publish every significant postmortem in a central, searchable repository with consistent tags for cause, affected component and customer impact, so people can look up similar failures before designing something. A regular cross-team reading or review session, perhaps monthly, discusses a few of the most interesting incidents, which builds a shared culture and spreads knowledge informally. I also analyse across postmortems to find patterns, such as repeated configuration-change failures, and report themes to leadership with suggested investments. The most scalable step is converting lessons into defaults: add a check to the deployment pipeline, update service templates and production readiness checklists, build a library that handles retries safely, or enforce a policy. Then every team benefits without needing to read the document. Summaries and newsletters help keep awareness high, without blame.

Likely follow-ups

- How do you find cross-incident patterns?
- What is the most scalable form of sharing?

Publish postmortems in a searchable internal repository, hold regular review sessions, summarise themes in newsletters, tag by cause and component, and turn recurring lessons into checklists, templates and platform guardrails.

```
postmortems/2026-10-12-checkout-outage.md
```

```
tags: deploy, config, dependency, capacity
```

```
monthly review: top 3 themes
```

```
release checklist updated with lesson #7
```

Interview tip: The best sharing mechanism is a guardrail in the platform that prevents the same failure for everyone.

📚 Official docs

- Google SRE Book: Postmortem Culture
- Google SRE Workbook: Postmortem Culture

certs.eknathalabs.com

Q151 Give examples of toil and non-toil, and explain the criteria that define toil.

Toil

🎙 **Spoken answer · ~58 s**

Toil has several characteristics: it is manual, repetitive, could be automated, is tactical and interrupt-driven rather than strategic, produces no lasting improvement, and grows in proportion to the size or traffic of the service. Examples include manually rotating certificates on a schedule, handling routine access requests by hand, restarting a leaking process every night, manually copying data between systems, and responding to alerts with the same fixed procedure each time. Work that looks like operations but is not toil includes designing and writing the automation that eliminates the above, performing a one-off migration, writing a postmortem, tuning capacity models, or on-call incident investigations that teach something new. Overhead like meetings and administration is also not toil, though it is not engineering. Identifying toil precisely matters because the aim is to keep it below a threshold and systematically remove it.

Likely follow-ups

- Is a one-off migration toil?
- Why does scale-linearity matter?

Toil is manual, repetitive, automatable, tactical work with no enduring value that scales linearly with service growth. Examples: manual restarts, ticket-driven provisioning. Not toil: writing automation, design, postmortems, one-off migrations.

```
toil: manually rotating certificates every quarter
```

```
toil: approving access requests by hand
```

```
toil: restarting a leaking service nightly
```

```
not toil: building self-service access tooling
```

Interview tip: The key question is whether the work leaves the system permanently better. Toil does not.

📚 **Official docs**

- Google SRE Book: Eliminating Toil

certs.eknathalabs.com

Q152 How do you quantify the ROI of automating a task, and when is it not worth it?

Toil

🎙 Spoken answer · ~62 s

I estimate the recurring cost of the manual work: how often it happens, how long it takes, and how many people are involved, converted into hours per year. I add intangible benefits such as fewer errors, faster response for users, and less interruption. Then I estimate the cost to build the automation plus its ongoing maintenance, because scripts rot. If the payback period is short, say under a year, and the task is stable, automation is a clear win. It is not worth it when the task is rare, takes minutes, or the underlying process is likely to change soon. Also, I first ask whether the task should exist at all: sometimes removing an approval step or fixing the root cause of why the task is needed gives a bigger gain than automating it. Cheap partial automation, like a script with a checklist, can capture most of the value.

Likely follow-ups

- Why include maintenance cost?
- When should you eliminate rather than automate?

Compare annual time saved (frequency x duration x people) plus error reduction against development and maintenance cost. Skip automation for rare, quick or unstable tasks, or when removing the need is better than automating it.

```
saved = freq/yr * minutes * people  = 52 * 30 * 2 = 52 h/yr
```

```
cost = build 40 h + maintain 8 h/yr
```

```
payback = 40 / (52 - 8) ≈ 0.9 years
```

```
also: risk reduction, faster lead time
```

Interview tip: Sometimes the best fix is to eliminate the task entirely by changing the process or design, not automate it.

📚 Start here (official docs)

- Google SRE Book

certs.eknathalabs.com

Q153 What is a self-service platform's role in reducing toil, and how do you measure adoption?

Toil

🎙 **Spoken answer · ~57 s**

Many operational tasks are requests from developers: create a database, add a queue, get a new environment, change DNS. If each needs a ticket and manual work from SRE, toil grows with the number of teams. A self-service platform, such as templates, a portal or declarative APIs with guardrails built in, lets developers do these tasks safely themselves in minutes, and encodes security and reliability best practice by default. I measure its success with the number of related tickets, the time to provision before and after, the percentage of services using the standard path, and developer satisfaction. I treat the platform as a product: talk to users, publish documentation, keep a roadmap and measure reliability of the platform itself. Without adoption, the effort is wasted, so I prioritise the most requested and most painful workflows first.

Likely follow-ups

- How do you measure platform success?
- What guardrails belong in self-service?

Self-service lets developers provision and operate resources through APIs or templates without tickets, removing SRE from the critical path. Measure adoption, ticket volume, time to provision and satisfaction.

```
ticket volume for 'new database' requests
```

```
time to provision: 3 days -> 10 minutes
```

```
% services on the paved road
```

```
developer satisfaction (survey)
```

Interview tip: A self-service tool nobody uses just moved the toil to its builders. Treat the platform as a product.

📚 **Official docs**

- Google SRE Book: Eliminating Toil
- CNCF: Platforms white paper

certs.eknathalabs.com

Q154 How do you tackle toil that comes from a poorly designed or legacy system that nobody dares to change?

Toil

🎙 **Spoken answer · ~56 s**

Fear of changing a legacy system usually stems from poor visibility and no tests. So I start by making the cost visible: record the hours spent per week and the incidents it causes, so I can build a business case. Next I create a safety net: add monitoring to understand real behaviour, and write characterisation tests that capture current outputs, even if imperfect, so I can detect unintended changes. Then I attack the most painful manual steps with small automations that wrap the system without modifying its core, and gradually replace components behind stable interfaces using a strangler approach, routing a growing share of work to the new implementation. I avoid a big-bang rewrite, which tends to be late and risky. Each increment must deliver measurable toil reduction, which keeps stakeholder support and demonstrates progress.

Likely follow-ups

- What is a characterisation test?
- Why avoid big-bang rewrites?

Quantify the cost, add observability and tests as a safety net, carve out the worst pain with strangler-style improvements, get management support with data, and avoid big-bang rewrites. Reduce toil incrementally.

```
toil log: 14 h/week on manual reconciliation
```

```
step 1: add monitoring + characterisation tests
```

```
step 2: automate the top 2 manual steps
```

```
step 3: replace one module behind an interface
```

Interview tip: Characterisation tests record what the legacy system does today, giving courage to change it.

📚 **Official docs**

- Google SRE Book: Eliminating Toil

certs.eknathalabs.com

Q155 How do you design retry policies safely, including retry budgets and idempotency keys?

Reliability Patterns

🎙 Spoken answer · ~62 s

Retries help with transient errors but can turn a small problem into an outage. I retry only when the error is likely temporary, such as a connection timeout or a service unavailable response, never for client errors. Each retry uses exponential backoff with random jitter, so clients spread out instead of synchronising, with a small maximum attempt count and an overall deadline. To avoid amplification, I use a retry budget, which limits retries to a fraction of normal traffic, say ten percent, so that a struggling service is not flooded. I also watch for retries at multiple layers multiplying, so I set them at one layer where possible. For operations that change state, such as creating a payment, retries are only safe if the operation is idempotent, so I require an idempotency key that the server uses to deduplicate. Circuit breakers stop retries entirely when failure is persistent.

Likely follow-ups

- Why does jitter matter?
- What is a retry budget?

Retry only transient, idempotent operations with exponential backoff and jitter, cap attempts, and use a retry budget (percentage of requests) to prevent amplification. Use idempotency keys for non-idempotent writes like payments.

```
max_attempts: 3  backoff: 100ms * 2^n  jitter: full
```

```
retry_budget: 10% of requests
```

```
POST /payments  Idempotency-Key: 7f3a-...
```

```
retry on: 503, 504, connect timeout; never on 400, 401, 404
```

Interview tip: Retries across several layers multiply: three layers with three attempts each make 27 calls for one failure.

📚 Official docs

- RFC 9110: HTTP Semantics (status codes, methods)
- AWS Builders' Library: Timeouts, retries and backoff with jitter
- Microsoft: Cloud design pattern – Retry

certs.eknathalabs.com

Q156 What is a service mesh's contribution to reliability, and what costs does it add?

Reliability Patterns

🎙 **Spoken answer · ~60 s**

A service mesh puts a proxy next to each workload, so cross-cutting reliability features are implemented uniformly without changing every application: timeouts, retries with limits, circuit breaking and outlier ejection, load balancing, canary traffic splitting, fault injection for testing, mutual TLS between services, and rich metrics and traces for every call. That consistency is valuable in organisations with many languages and teams. The costs are real: each hop adds proxy latency and memory and CPU overhead, the control plane becomes critical infrastructure whose outage or bad configuration can break everything, upgrades are complex, and debugging requires understanding another layer. Teams must learn its configuration model. I adopt a mesh when there is a clear need such as zero-trust encryption or uniform traffic management at scale, start with a limited rollout, have strong observability of the mesh itself, and keep an escape hatch to bypass it.

Likely follow-ups

- What new failure modes does it introduce?
- When is a mesh overkill?

A mesh provides consistent retries, timeouts, circuit breaking, mTLS, traffic shifting and telemetry outside application code. Costs: extra latency and resource use, operational complexity, and another critical failure domain.

```
istioctl proxy-status
```

```
kubectl get virtualservice,destinationrule -A
```

```
outlierDetection: consecutive5xxErrors: 5 interval: 10s baseEjectionTime: 30s
```

```
kubectl top pods -n istio-system
```

Interview tip: Adopt a mesh for a clear problem, such as mTLS or uniform traffic policy, not because it is fashionable.

📚 **Official docs**

- Istio documentation

certs.eknathalabs.com

Q157 How do you design a graceful shutdown and connection draining for zero-downtime deploys?

Reliability Patterns

🎙 **Spoken answer · ~61 s**

Deployments fail in subtle ways when pods are killed while serving requests. A correct shutdown sequence has several parts. When the orchestrator decides to stop a pod, it removes it from the service endpoints and sends a termination signal, but these propagate asynchronously, so some traffic may still arrive for a few seconds. A short pre-stop delay gives load balancers time to stop routing. On receiving the signal, the application stops accepting new connections, marks itself not ready, completes in-flight requests within a deadline, flushes buffers and closes database connections, then exits. The termination grace period must be longer than the longest request plus the delay, or the process is force-killed. Long-lived connections like websockets need a mechanism to ask clients to reconnect elsewhere. I test this under load during rollouts, watching for error spikes, and verify that jobs and consumers also stop cleanly and release leases.

Likely follow-ups

- Why a preStop sleep?
- What happens to long-lived connections?

On SIGTERM, fail readiness so traffic stops, finish in-flight requests within a grace period, close connections cleanly, and use preStop delay to let load balancers deregister. Match terminationGracePeriod to the longest request.

```
lifecycle: preStop: exec: command: ["sleep","10"]
```

```
terminationGracePeriodSeconds: 45
```

```
server.shutdown(timeout=30)  # stop accepting, drain
```

```
kubectl rollout status deploy/api
```

Interview tip: Endpoint removal and SIGTERM race each other. The preStop sleep covers the propagation delay.

📚 **Official docs**

- Kubernetes: Pod lifecycle & termination
- Kubernetes: Safely drain a node

certs.eknathalabs.com

Q158 How do you build and test a disaster recovery plan for a stateful service, including RTO and RPO targets?

Reliability Patterns

🎙 Spoken answer · ~62 s

I start from business requirements: how long can the service be down, the recovery time objective, and how much data can we lose, the recovery point objective. Those numbers drive the strategy and cost: backup and restore is cheapest but slowest; pilot light keeps minimal infrastructure ready; warm standby runs a scaled-down copy; active-active gives near-zero downtime at the highest cost and complexity. For stateful data, replication and continuous log shipping set the recovery point, while automation of provisioning and failover sets the recovery time. I document runbooks, keep infrastructure as code so the environment can be rebuilt, store backups in a separate account or region with immutability, and monitor backup success. The key is testing: scheduled drills that restore from backups, run verification queries and a failover to the secondary, measuring actual times against targets and fixing gaps. I include failback and dependency ordering in the exercise.

Likely follow-ups

- How do RTO and RPO differ?
- Why store backups in a separate account?

Define business RTO/RPO, choose a strategy (backup-restore, pilot light, warm standby, active-active) by cost, automate failover and restore, and test regularly with real restores. An untested backup is not a backup.

```
pg_basebackup + WAL archiving -> RPO ~ minutes
```

```
aws rds restore-db-instance-to-point-in-time ...
```

```
time ./restore-and-verify.sh  # measures real RTO
```

```
dr drill: failover region, run smoke tests, fail back
```

Interview tip: Measure the real restore time and data loss in a drill, then compare it with the target.

📚 Official docs

- AWS Disaster Recovery of workloads (whitepaper)
- Google SRE Book: Data Integrity

certs.eknathalabs.com

Q159 What is a feature flag's role in reliability, and what are the risks of flags themselves?

Reliability Patterns

🎙 Spoken answer · ~58 s

Feature flags decouple deploying code from enabling behaviour. For reliability this offers a gradual rollout to a small percentage of users with monitoring, immediate disabling of a problematic feature without a rollback or deployment, and operational switches that shed non-essential load during overload. The risks come from the flags themselves. They add code paths whose combinations are rarely all tested; they accumulate as dead code if not removed; the flag management service can become a dependency, so the application needs cached values and safe defaults if it is unreachable; and a mistaken change to a flag is itself a production change that can cause an outage, so it needs review, audit logs and ideally staged propagation. I assign each flag an owner and expiry date, remove it after full rollout, and treat flag changes with the same care as deployments.

Likely follow-ups

- What are safe defaults?
- Why give flags an expiry?

Flags allow gradual rollout, kill switches and fast mitigation without redeploys. Risks: stale flags, untested combinations, flag service outages, and config errors. Mitigate with defaults, expiry, ownership and testing.

```
flag: new-recommendations  rollout: 5% -> 25% -> 100%
```

```
kill switch: disable feature in seconds
```

```
default if flag service down: safe/off
```

```
lint: flags older than 90 days
```

Interview tip: Define safe defaults so a flag service outage does not take your application down.

📚 Official docs

- Martin Fowler: Feature Toggles

certs.eknathalabs.com

Q160 How do you run chaos experiments in Kubernetes using tools like Chaos Mesh or Litmus?

Chaos/Game Day

🎙 Spoken answer · ~61 s

I begin by defining the hypothesis and steady state, for example that when one payments pod is killed, error rate stays below one percent and latency within target. Then I write the experiment declaratively with a tool like Chaos Mesh or Litmus: choose the fault type such as killing pods, adding network latency or loss, stressing CPU or memory, or partitioning a dependency, and restrict the blast radius with namespace and label selectors, a small percentage of targets, and a short duration. I run first in a staging environment, then in production with care, during working hours, with the team watching dashboards, and with automatic abort conditions linked to alerts. After the run, I compare observed behaviour with the hypothesis, record findings, fix weaknesses such as missing retries or bad timeouts, and repeat the experiment to confirm. Over time I schedule experiments continuously as regression tests.

Likely follow-ups

- What is a good first experiment?
- How do you limit blast radius?

Define steady-state metrics, write an experiment manifest (pod kill, network delay, CPU stress), scope by label and namespace, run during working hours with abort conditions, and observe whether SLOs hold. Start in staging.

```
kubectl apply -f podchaos.yaml
```

```
apiVersion: chaos-mesh.org/v1alpha1 kind: NetworkChaos spec: action: delay delay: {
latency: 200ms } selector: { labelSelectors: { app: payments } } duration: 5m
```

```
kubectl get podchaos,networkchaos -n chaos-testing
```

```
kubectl delete -f podchaos.yaml   # abort
```

Interview tip: Always have a one-command abort and automatic stop conditions tied to SLO alerts.

📚 Official docs

- Principles of Chaos Engineering
- AWS Fault Injection Service

certs.eknathalabs.com

Q161 What kinds of failures reveal the most in distributed systems: latency injection or crashes? Why is slowness harder?

Chaos/Game Day

🎙 Spoken answer · ~58 s

A crashed dependency produces immediate errors, which are easy to detect and handle with retries or fallbacks. A slow dependency is far more dangerous: callers wait, holding threads, connections and memory; queues build; retries pile on more load; health checks may still pass because the process is alive. This can exhaust resources in upstream services and cascade through the system. So latency injection reveals missing or too generous timeouts, absent bulkheads that should isolate resource pools, lack of circuit breakers, and poor backpressure. Packet loss, partial failures and intermittent errors reveal similar weaknesses. I prefer to inject gradual slowness at the network or proxy level, for example adding hundreds of milliseconds to a database call, and observe thread pool usage, queue depth and tail latency. The fix is usually layered: timeouts, bulkheads, load shedding, circuit breaking and graceful degradation.

Likely follow-ups

- Why is slowness worse than failure?
- What protects against slow dependencies?

Slowness is harder than crashes because it ties up threads, queues and connections, triggers retries and cascades, and evades health checks. Latency injection exposes missing timeouts, bulkheads and backpressure.

```
tc qdisc add dev eth0 root netem delay 500ms 100ms
```

```
toxiproxy-cli toxic add -t latency -a latency=1000 db_proxy
```

```
kubectl exec pod -- tc qdisc del dev eth0 root
```

```
# watch: thread pool saturation, queue depth, p99
```

Interview tip: A dead dependency fails fast. A slow one holds your resources hostage.

📚 Official docs

- OWASP Top Ten

certs.eknathalabs.com

Q162 How do you verify that failover works for databases and caches without causing a real outage? Chaos/Game Day

🎙 **Spoken answer · ~57 s**

Failover is only trustworthy if exercised. I schedule controlled failover drills, first in a production-like staging environment and then in production during low-risk windows with the team ready. I trigger the managed failover or kill the primary, then observe: how long until the replica is promoted, how clients behave when connections break, whether connection pools detect dead connections and reconnect, whether DNS time-to-live and client-side caching delay the switch, and whether any writes were lost or duplicated. I measure the real recovery time against the target and check dashboards for error spikes and queue build-up. Typical findings include applications with infinite retries, no connection validation, long DNS caching, or sessions that are lost. I fix them, repeat the drill, and then keep it on a regular schedule so that the team stays practised and regressions get caught.

Likely follow-ups

- What commonly goes wrong in failover?
- Why repeat the drill regularly?

Rehearse in staging and then production in a controlled window: trigger managed failover, measure client reconnect behaviour, confirm no data loss, and check connection pool and DNS cache handling. Run regularly so it is routine.

```
aws rds reboot-db-instance --db-instance-identifier db --force-failover
```

```
redis-cli -h primary debug sleep 30
```

```
dig +short db.internal   # watch DNS update
```

```
psql -c 'select pg_is_in_recovery()'
```

Interview tip: Applications often fail not because failover is slow but because they cache DNS or hold dead connections.

📚 **Start here (official docs)**

- Google SRE Book

certs.eknathalabs.com

Q163 How do you build a resilience scorecard or maturity model for services? Chaos/Game Day

🎙 Spoken answer · ~63 s

A scorecard turns resilience from a vague aspiration into a checklist that teams can improve. I define a set of criteria drawn from experience, such as a documented SLO with burn alerts, timeouts and retries configured properly, graceful degradation for non-critical dependencies, tested failover, backups with verified restores, a load test within the last quarter, runbooks linked from alerts, a dependency map, and periodic chaos experiments. Each service is scored against the criteria, perhaps in levels from basic to advanced, and results are presented on a dashboard. Leadership uses it to see where risk is concentrated and to prioritise investment; teams use it as a roadmap of concrete next steps. I make the criteria proportionate to service tier, since an internal tool needs less than a payment system. To keep it constructive, I focus on trends and support, and use it as a production readiness gate for new critical services.

Likely follow-ups

- Why scale criteria by tier?
- How do you avoid shaming teams?

Define criteria like SLOs, tested failover, timeouts, runbooks, capacity tests, dependency maps and chaos testing, score each service, share results, and use them to prioritise investment and gate launches.

```
[x] SLO + burn alerts
```

```
[x] Timeouts/retries reviewed
```

```
[ ] Failover tested in last 6 months
```

```
[ ] Load tested at 2x peak
```

```
[x] Runbooks linked from alerts
```

```
score: 3/5
```

Interview tip: Use scorecards to guide improvement, not to shame teams. Show progress over time.

📚 Official docs

- OpenSSF Scorecard

certs.eknathalabs.com

Q164 How do you perform capacity planning for a database, including connection limits, IOPS and storage growth?

Capacity

🎙 **Spoken answer · ~64 s**

Databases are hard to scale quickly, so I plan early. I monitor the main resources: CPU and memory, disk IOPS and throughput, connection counts, replication lag, cache hit ratio, and storage consumption. I project trends, for example linear growth of storage over the past thirty days into the next ninety, and define alert thresholds that give enough lead time to act, considering how long a resize, migration or new replica takes. Connection limits are an easy trap because each application instance opens pools; a connection pooler such as PgBouncer reduces pressure. I load test with realistic queries to find the point of saturation and to validate that indexes and queries are efficient before simply adding hardware. Scaling options include vertical upgrades, read replicas and caching for read-heavy workloads, partitioning, and eventually sharding for write scale. I also plan for peak events and keep headroom for failover, when one node must carry the load.

Likely follow-ups

- Why do connection limits matter?
- What gives you scaling lead time?

Track CPU, memory, IOPS, connections, replication lag and storage growth; project trends with headroom; load test; plan scaling steps (vertical, read replicas, sharding) well before limits, considering lead time.

```
select count(*) from pg_stat_activity;
```

```
aws cloudwatch get-metric-statistics --metric-name ReadIOPS ...
```

```
predict_linear(db_storage_used_bytes[30d], 90*86400)
```

```
pgbouncer: pool_mode=transaction default_pool_size=20
```

Interview tip: Connection limits are often hit before CPU. Pooling is usually the first and cheapest capacity fix.

📚 **Official docs**

- Google SRE Book: Software Engineering in SRE (capacity planning)
- PgBouncer
- Azure Well-Architected: Performance efficiency

certs.eknathalabs.com

Q165 What are the pitfalls of autoscaling on CPU alone, and which better signals exist?

Capacity

🎙 **Spoken answer · ~59 s**

CPU is a convenient default but often a poor trigger. It lags behind demand, so by the time CPU is high, users already see latency. It also misrepresents workloads that are I/O bound, waiting on databases, or driven by queues, where CPU stays low while latency climbs. Better signals reflect work arriving or waiting: requests per second per instance, concurrent in-flight requests, queue depth or the age of the oldest message, or consumer lag for streaming workloads. Event-driven autoscalers can scale on these metrics, and even down to zero for sporadic jobs. I also consider the mechanics: startup and warm-up time, image pull time, scale-up rate limits, and stabilisation windows to avoid flapping. I keep minimum replicas for baseline resilience, set maximums to protect dependencies and budgets, and load test to confirm that the autoscaler keeps latency within target during a ramp.

Likely follow-ups

- Why is CPU a lagging indicator?
- What is KEDA used for?

CPU lags and doesn't reflect I/O-bound or queue-driven work. Better signals: request rate per pod, queue depth or age, concurrency, latency, or custom business metrics, plus scale-up speed and warm-up considerations.

```
kubectl autoscale deploy api --min=3 --max=30 --cpu-percent=60
```

```
keda: ScaledObject triggers: - type: kafka metadata: { lagThreshold: "100" }
```

```
metrics: - type: Pods pods: { metric: { name: requests_per_second }, target: { type:
AverageValue, averageValue: "50" } }
```

Interview tip: Scale on a leading indicator of load, not a lagging indicator of pain.

📚 **Start here (official docs)**

- Google SRE Book

certs.eknathalabs.com

Q166 How do you use load shedding priority and admission control to keep critical traffic working during overload?

Capacity

🎙 Spoken answer · ~60 s

When demand exceeds capacity, trying to serve everything means serving nothing well: queues grow, latency rises and requests time out, wasting work. Load shedding deliberately rejects some work to keep the rest healthy. I classify traffic by importance: user-facing checkout is critical, background sync and analytics are lower. Detection uses saturation signals such as concurrent requests, queue length, memory pressure or latency. When thresholds are crossed, the service rejects the lowest priority requests first with a quick error and a hint to retry later, then progressively more, while reserving capacity for critical flows. Clients are expected to honour the retry hint with backoff so that shedding does not trigger a storm. Adaptive concurrency limits can set thresholds automatically. I test with load generators that push beyond capacity to confirm that the service degrades gracefully, and publish metrics on shed requests so I can tune.

Likely follow-ups

- What should the client do when shed?
- How do you pick what to shed first?

Classify requests by criticality, reject or queue lower-priority work first when saturation signals appear, return fast errors with retry hints, and protect resources for critical paths. Overload control should be automatic and tested.

```
X-Request-Priority: critical | normal | batch
```

```
if inflight > limit: shed batch, then normal
```

```
HTTP 503 Retry-After: 5
```

```
envoy: overload_manager resource_monitors: fixed_heap
```

Interview tip: Failing fast with a clear error is better than slow timeouts that hold resources and make everything worse.

📚 Official docs

- Google SRE Book: Handling Overload
- AWS Builders' Library: Using load shedding to avoid overload

certs.eknathalabs.com

Q167 How do you account for cost in capacity decisions, such as spot instances, reservations and rightsizing?

Capacity

🎙 Spoken answer · ~58 s

Capacity is a trade-off between reliability and cost. I separate demand into a steady baseline, which I cover with reserved or committed capacity at a discount, and variable load, which uses on-demand instances or autoscaling. Fault-tolerant, interruptible workloads such as batch jobs, CI runners and stateless workers can run on spot capacity at a large discount, as long as they handle interruptions and are spread across instance types and zones. Rightsizing uses real utilisation data to shrink oversized instances and requests, but I preserve headroom for traffic spikes, deployments and the loss of a zone, so that I do not trade resilience for savings. I track unit economics, such as cost per thousand requests, to see efficiency as the system grows, and review commitments regularly. Cost data is shared with teams, creating accountability, and reliability targets stay the guardrail.

Likely follow-ups

- What workloads suit spot instances?
- Why track cost per request?

Match purchase model to workload: reserved or committed for baseline, on-demand for variable, spot for fault-tolerant batch. Rightsize using utilisation data, but keep headroom for failure and spikes, and track unit costs.

```
kubectl top nodes; kubectl top pods -A --sort-by=cpu
```

```
aws ce get-reservation-utilization --time-period Start=2026-09-01,End=2026-10-01
```

```
karpenter: capacityType: [spot, on-demand]
```

```
cost per 1000 requests = monthly_cost / (requests/1000)
```

Interview tip: Cost per unit of work (per request, per user) is more useful than the raw bill when traffic grows.

📚 Official docs

- AWS Well-Architected: Cost optimization pillar

certs.eknathalabs.com

Q168 Your service's error rate jumps to 30% right after a deploy, but the canary looked healthy. What could explain this and what do you do?

Incident Mgmt

🎙 Spoken answer · ~63 s

First I restore service by rolling back, since the deploy is the obvious suspect and rollback is fast. Then I work out why the canary gave false confidence. Possibilities include: the canary received too small a share of traffic or too short an observation time, so rare paths or heavy users were not exercised; the failing code path depends on a specific customer segment, endpoint or data shape not in the sample; the effect is delayed, such as a cache entry expiring, a memory leak, a connection pool filling, or a scheduled job running later; the canary and full rollout differ in configuration or in the dependencies they hit; or a downstream system only saturates at full volume. I examine the metrics comparing canary and baseline, check the failing requests, and reproduce in staging. Improvements include longer bake time, automated analysis against SLO metrics, traffic mirroring, and tests covering the failing path.

Likely follow-ups

- Why can a canary pass but full rollout fail?
- How would you improve canary analysis?

Mitigate: roll back. Then investigate why the canary missed it: low traffic share, wrong traffic type, delayed effect (cache expiry, connection pool, background job), config differences, or a dependency that only fails at full volume.

```
kubectl rollout undo deploy/api
```

```
kubectl argo rollouts abort api
```

```
compare canary vs prod: traffic %, endpoints hit, data sets
```

```
check cache TTLs, cron jobs, feature flag segments
```

Interview tip: A canary is only as good as the traffic it sees. If it never exercised the failing path, it proved nothing.

📚 Official docs

- Google SRE Workbook: Canarying Releases
- Martin Fowler: CanaryRelease

certs.eknathalabs.com

Q169 You get paged at 3 a.m. for a database CPU alert on a service you do not know well. How do you proceed?

Incident Mgmt

🎙 Spoken answer · ~62 s

At 3 a.m. I start by assessing whether users are impacted: error rate, latency and the service's SLO dashboards. The alert description should link to a runbook, so I follow it. I check what changed recently, deployments, migrations or traffic spikes, and look at the database's active queries to see whether a few expensive or runaway statements are driving CPU. If I find an obvious offender, I can cancel the query, and if a recent change is implicated, I roll it back. If load is simply high, I consider mitigation like scaling the instance, adding capacity to read replicas, enabling caching or shedding non-critical traffic. If I do not know the service and the runbook is insufficient, I escalate to the secondary or the service owner without hesitation. I write down my actions in the incident channel, and after resolution I improve the runbook and file follow-up tickets.

Likely follow-ups

- When do you escalate?
- What do you record while mitigating?

Check user impact first, open the runbook, look at recent changes and top queries, mitigate (kill runaway queries, scale, shed load), escalate to the owning team if needed, and document what you did for handoff.

```
select pid, state, now()-query_start as dur, left(query,80) from pg_stat_activity where state<>'idle' order by dur desc limit 10;
```

```
select pg_cancel_backend(<pid>);
```

```
git log --since='6 hours ago' --oneline
```

```
escalate: page secondary / service owner
```

Interview tip: Escalating early when you lack context is a strength. The cost of waking a colleague is lower than a prolonged outage.

📚 Start here (official docs)

- Google SRE Book

certs.eknathalabs.com

Q170 A marketing campaign will send 10x normal traffic next Tuesday and you were told on Friday. How do you prepare in four days?

Capacity

🎙 **Spoken answer · ~61 s**

First I clarify expectations with marketing: expected peak requests, user journeys, timing, and the risks if we fail. Then I translate that into capacity needs and identify the critical path, such as landing page, search and checkout. I load test those paths in a production-like environment up to and beyond twelve times current traffic to find bottlenecks, and fix the biggest ones: database queries, cache usage, connection pools. I check hard limits that cannot be changed quickly, cloud quotas, third-party API rate limits and payment provider capacity, and request increases now. Before the event I pre-scale instances and databases, warm caches and CDN, and configure autoscaling with higher maximums. I add protections: rate limiting, priority load shedding, and a feature kill switch for non-essential components. On the day, on-call is staffed with a war room, dashboards and a rollback plan, and a clear communication channel with marketing.

Likely follow-ups

- What is the hidden ceiling in a surge?
- How do you decide what to shed?

Quantify expected load, load test critical paths, pre-scale and raise limits, warm caches, protect dependencies, add load shedding and a kill switch, brief on-call, set up a war room and rehearse rollback.

```
k6 run --vus 2000 --duration 15m checkout.js
```

```
kubectl scale deploy/api --replicas=40
```

```
aws service-quotas list-service-quotas ...
```

```
pre-warm: CDN cache, DB connection pool, JIT
```

Interview tip: Check quotas and third-party rate limits. They are the usual hidden ceiling in a surge.

📚 **Start here (official docs)**

- Google SRE Book

certs.eknathalabs.com

Q171 A chaos experiment accidentally causes a customer-visible outage. How do you respond and what changes?

Chaos/Game Day

🎙 Spoken answer · ~63 s

The first step is to stop the experiment and restore service, using the abort mechanism I prepared and any standard incident response. I treat it like any other incident: communicate with affected customers and stakeholders honestly, noting that it was caused by our own testing. Afterwards I run a blameless postmortem to answer specific questions. Was the blast radius larger than planned, and why? Did the abort conditions or monitoring fail to trigger? Was the experiment approved and scheduled appropriately, with the right people watching? Did it reveal a genuine hidden dependency or weakness, which is in effect the purpose of the exercise but at too high a cost? Based on the findings I strengthen guardrails: smaller scope, automatic aborts tied to SLO burn, stricter review, running at quieter times, and more testing in staging. I keep the practice going, because abandoning it leaves such weaknesses to be discovered by real failures.

Likely follow-ups

- What guardrails would you add?
- Why not stop chaos engineering?

Abort the experiment immediately, restore service, communicate honestly, run a blameless postmortem, review blast radius, abort conditions and approvals, and keep the practice with stronger safeguards rather than abandoning it.

```
kubectl delete -f experiment.yaml   # abort
```

```
verify steady state restored: error rate, latency
```

```
postmortem: why was blast radius larger than planned?
```

```
new guard: auto-abort on SLO burn > 2x
```

Interview tip: The experiment revealed a real weakness. The failure is valuable if you learn and tighten the process.

📚 Official docs

- Principles of Chaos Engineering

certs.eknathalabs.com

Q172 Your team's toil is at 70% and leadership wants new features supported. How do you restore balance?

Toil

🎙 **Spoken answer · ~60 s**

I start by quantifying: for a couple of weeks the team records toil by category and hours, so I have data rather than feelings. I identify the few sources that account for most of the time, for example access requests, certificate renewals and manual deployments, and estimate the cost of automating each against hours saved. I take this to leadership as a trade-off. At seventy percent toil, the team has little capacity for engineering, so new feature support will degrade reliability and increase burnout. I propose a time-boxed plan to automate the top items, with expected payback, and ask for temporary measures: pause onboarding of new services, push back or hand back low-value operational work, or add headcount. I protect engineering time in each sprint, track the toil percentage each month, and share wins. If the trend does not improve, I escalate with evidence.

Likely follow-ups

- How do you gather toil data?
- What do you ask leadership for?

Measure and categorise toil, pick the top two sources by hours, propose time-boxed automation with expected payback, negotiate to cap new onboarding, hand back or stop supporting low-value services, and report progress.

```
toil log by category (hours/week)
```

```
top: access requests 12h, cert renewals 8h, manual deploys 7h
```

```
plan: self-service access (3 wks), cert-manager (2 wks)
```

```
negotiate: pause new service onboarding until toil < 50%
```

Interview tip: Present it as a business trade-off with numbers: hours lost, incidents caused, and what automation will return.

📚 **Official docs**

- Google SRE Book: Eliminating Toil

certs.eknathalabs.com

Q173 Your SLO dashboard shows 99.95% but customers are complaining about slowness and errors. What could be wrong?

SLI/SLO

🎙 Spoken answer · ~61 s

A healthy dashboard with unhappy customers means the indicator is not capturing reality. I check several possibilities. The measurement point may be too far from the user, such as inside the service, missing problems at the CDN, DNS, network or client; real user monitoring would show them. Aggregation may hide a segment: a specific region, large customer or endpoint can be failing badly while the global ratio looks fine, so I break the data down. Latency may be measured as an average or against a threshold that is too generous compared with what users tolerate. Exclusions or the valid-events filter may remove traffic that users care about, like retries or certain error codes. The definition of good may count a response as successful even if the content is wrong. I compare with support tickets and synthetic probes, correct the SLI definition, and recalculate the historical numbers.

Likely follow-ups

- Why compare with RUM?
- How can aggregation hide problems?

The SLI does not reflect user experience: wrong measurement point, averaged latency, excluded traffic, missing client-side failures, regional or segment issues hidden in the aggregate, or good-event definition too lenient.

```
compare: server-side vs client-side (RUM) success rate
```

```
break down SLI by region, customer, endpoint, device
```

```
check latency threshold vs real user tolerance
```

```
review exclusions and valid-event filters
```

Interview tip: Cross-check SLIs against support tickets and RUM. If they disagree, trust the customers and fix the metric.

📚 Official docs

- Google SRE Book: Service Level Objectives
- Google SRE Workbook: Implementing SLOs
- Google SRE Book: Embracing Risk (capacity & availability math)

certs.eknathalabs.com